

✦ DESIGN PATTERNS ✦

Final Exam Study & Memorization Guide

SE 506 / CSE 2215 — Design Patterns

Institute of Information Technology, University of Dhaka

Revised edition — aligned with the teacher's final suggestions

🔗 OOP & Good Design

📋 SOLID

👤 Smells: Effects & Fixes

🔗 22 Pattern Signatures

🔗 Pattern Comparisons

📄 UML Scenarios

📄 Cheat Sheet

Built from 11 past question papers (2013–2024) plus the teacher's final suggestion note. Every past-paper question is kept (with how many times it appeared). New sections cover what the note asks for: **good design & abstraction, what each SOLID principle really targets, smell effects & fixes**, and — since design patterns are ~60% — each pattern's **identifying signature, when / when not**, look-alike **comparisons**, and **scenario** → **UML** practice. Pattern content follows **refactoring.guru**; diagrams are real UML class diagrams, because the exam asks for design, not code.

Sources: Finals 2013, 2014, 2017, 2020, 2024 (SE 506 & CSE 2215) · Midterm 2016 · Teacher's final suggestion note · refactoring.guru

🎯 Teacher's Exam Focus → Where It's Covered

Expected weight: Design Patterns ≈ 60% of questions (per the teacher's pre-moderation paper).

1. Basics of OOP

When is one design better than another; applying abstraction properly (inheritance vs abstract class). → **Part 1** (1.1, 1.2) + OOP Q&A.

2. SOLID

What each principle actually targets; the real meaning of Single Responsibility; why "closed for modification" doesn't apply to every file. → **Part 2**.

3. Code Smells

Don't memorize — focus on effects and solutions; use "smells in the wild" insights. → **Part 3** (effect → fix tables, 3.3) + **Part 4** exercises.

4. Design Patterns (~60%)

Differences & similarities; when / when not to use among alternatives; scenario questions → suggest pattern(s) + core UML (no code); each pattern's identifying signature. → **Part 5** (signatures), **Part 6** (comparisons), **Part 7** (past-paper scenarios), **Part 8** (new scenarios).

★ **Teacher focus** marks every question added or rewritten for the note. **asked 3x** shows how often a question appeared in past papers.

Scenario answer template: Problem → Pattern → Why (map scenario to pattern roles) → UML → Why not the look-alike.

Contents

1 OOP Basics & Good Design

better design, abstraction, inheritance vs abstract class vs interface, OOP Q&A

2 SOLID Principles

what each targets · real SRP · limits of OCP · LSP · ISP · DIP

3 Code Smells: Effects & Solutions

smell families, effect → fix, smells in the wild, refactoring theory Q&A

4 Code Smell Exercises

8 real exam code snippets with formatted question code & refactors

5 Pattern Catalog — 22 Patterns

intent, ⚡ signature, UML, when / when not, look-alikes

6 Similar Patterns Compared

Proxy/Decorator/Adapter/Facade, Composite/Decorator, Strategy/State/Template, factories ...

7 Past-Paper Design Questions

all scenario questions answered with UML

8 Scenario Practice

11 new scenarios with UML + 22-item quick-fire table

9 Memory Tricks

mnemonics, signature pictures, tie-breakers

10 Final Cheat Sheet

22 patterns + signatures, keyword → pattern, smell → fix, exam strategy

PART 1 — OOP Basics & Good Design

1.1 When can we call one design better than another? ★ Teacher focus

A: A design is better when it is **cheaper and safer to change**. Code is read and changed far more often than it is written, so we judge a design by what happens when requirements change. Refactoring Guru names two features of good design: **code reuse** and **extensibility**. In practice, check these points:

- **Low coupling** — classes know as little as possible about each other. One change does not ripple into many classes.
- **High cohesion** — each class has one clear job, and everything inside it belongs to that job.
- **Change is local** — a new requirement means *adding* a class, not editing many old ones (Open/Closed).
- **Easy to understand** — a new developer can find where things happen. Names explain intent.
- **Easy to test** — you can test a class alone by giving it a fake dependency (depends on interfaces, not concrete classes).
- **No duplication** — each piece of knowledge lives in one place.
- **Simple enough** — no "just in case" layers (Speculative Generality). A pattern that solves no real problem makes a design *worse*.

How to compare two designs in an exam: pick a *likely change* (new type, new channel, new rule) and walk it through both designs. The one where that change touches fewer existing classes — and doesn't need `if/else` edits — is the better design.

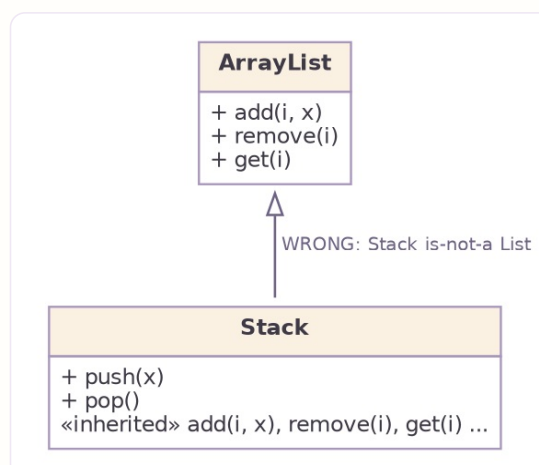
Refactoring Guru's design principles behind this: **Encapsulate what varies, Program to an interface, not an implementation, Favor composition over inheritance, and SOLID.**

1.2 How do we apply abstraction properly? Inheritance vs abstract class vs interface ★ Teacher focus

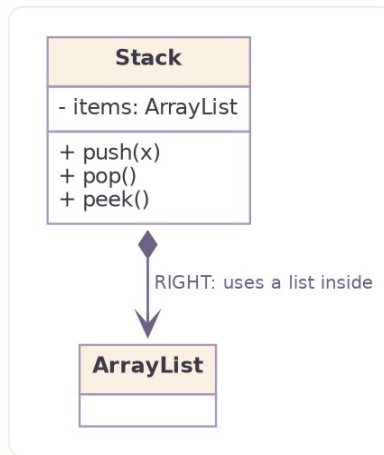
A: Abstraction means **showing only what matters and hiding the rest**. Applying it properly means choosing the right tool for the *relationship* you're modelling:

Tool	Use it when	Signal it's wrong
Inheritance (extends a concrete class)	Child truly is-a parent and can replace it everywhere (LSP holds)	Child overrides methods to throw errors or do nothing (Refused Bequest)
Abstract class	Several classes share identity + common code/state , but some steps differ	You only needed a contract, no shared code
Interface	Unrelated classes share a capability ("can fly", "can be printed")	You keep copy-pasting the same code into every implementer
Composition	You want to reuse behaviour without an is-a relationship, or swap it at runtime	— (usually the safe default)

Classic mistake — inheriting just to reuse code:



Wrong: Stack inherits every List method (add at index, remove at index) — it breaks the stack idea.



Right: Stack HAS a list inside and exposes only push/pop/peek.

Rules of thumb:

1. Ask "is-a and can always substitute?" — if not, don't inherit.
2. Share code → abstract class. Share only a promise → interface.
3. Want behaviour that can change at runtime → composition (this is the heart of Strategy, State, Decorator, Bridge).
4. Keep hierarchies shallow. Deep trees break easily when the top changes.

1.3 What are the three main categories of design patterns? Briefly describe the purpose of each.

asked 2x

A: Refactoring Guru groups its 22 patterns by intent:

- **Creational** — *how objects are created*. They make creation flexible so code doesn't depend on concrete classes. (Factory Method, Abstract Factory, Builder, Prototype, Singleton)
- **Structural** — *how classes and objects are assembled into bigger structures* while staying flexible and efficient. (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy)
- **Behavioral** — *how objects communicate and share responsibilities*. (Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor)

Memory hook: **Creational = birth, Structural = body, Behavioral = brain.**

1.4 Why is access to non-static variables not allowed from static methods?

asked 2x

A: A **static method belongs to the class**, not to any object. It can run before any object exists (e.g. `main()`). A **non-static variable lives inside an object**. When a static method runs, there may be no object at all — and if there are many objects, the method can't know *whose* variable to use. So the compiler blocks it. (A static method can still use instance data if you pass it an object: `obj.field`.)

1.5 What does encapsulation mean? Why is it a vital OOP concept?

asked 2x

A: **Encapsulation** = bundling data and the methods that work on it into one class, and **hiding the internal state** (private fields, public methods).

Why it's vital:

- **Protects data** — no one can put an object into an invalid state from outside.
- **Hides implementation** — you can change the inside without breaking users of the class.
- **Reduces coupling** — others depend on a small public interface only.
- It's the base of "**Encapsulate what varies**", the first principle on Refactoring Guru — nearly every pattern hides a changing part behind a stable interface.

1.6 What are the basic features of OOP? Which one is most important and why?

A: The four pillars:

1. **Abstraction** — model only the essential details.
2. **Encapsulation** — hide internal state behind methods.
3. **Inheritance** — build new classes on existing ones.
4. **Polymorphism** — one interface, many implementations.

Most important (with justification): Polymorphism is a strong answer for this course, because it's what lets us *swap behaviour without `if/else`* — every design pattern (Strategy, State, Factory Method, Observer...) depends on calling a method through an interface and letting the object decide what happens. **Abstraction** is also acceptable if argued as "you must first decide the right interface". Any answer is fine if justified — the reasoning earns the marks.

1.7 What do object inheritance and object composition mean? Pros and cons for reusability?

A:

- **Inheritance ("is-a")** — a subclass reuses the parent's code automatically.
- ✓ Quick reuse, natural polymorphism.
- ✗ Tight coupling: parent changes can break children. Fixed at compile time. Subclass gets *all* parent methods, even unwanted ones. Deep hierarchies explode in size (e.g. `Car × Engine × Color` subclasses).
- **Composition ("has-a")** — a class holds another object and delegates work to it.
- ✓ Loose coupling, swap parts at runtime, combine behaviours freely.
- ✗ A bit more code (delegating methods), more objects to wire.

Refactoring Guru: "**Favor composition over inheritance.**"

1.8 Can we replace an interface with an abstract class? Justify. asked 3x

A: Not fully.

- An abstract class can act like an interface (only abstract methods), **but** Java/C# allow only **one** parent class and **many** interfaces. A class that must play several roles (e.g. `Comparable`, `Serializable`, `Printable`) can't do that with abstract classes.
- Forcing unrelated classes (`Bird`, `Airplane`) to extend one abstract `Flyer` creates a fake is-a relationship.
- Abstract classes carry state and code — sometimes that's unwanted coupling when you only need a contract.

So: replace only if the class needs just one role and the implementers really share identity/code. Otherwise keep the interface.

1.9 What is the philosophical difference between an abstract class and an interface? When to use which?

A:

	Abstract class	Interface
Says	what an object is (identity)	what an object can do (capability/contract)
State	can have fields	no instance state
Code	can share implemented methods	contract only (default methods are an exception, not the idea)
Inheritance	one	many

Scenarios:

- `Employee` as **abstract class**: all employees share `name`, `id`, `calculateTax()`; each subtype defines `calculatePay()`.
- `Payable` as **interface**: `Employee`, `Invoice`, and `Vendor` are unrelated but all can be paid.

1.10 "Talk to the interface, not to the concrete class" — explain.

A: Code should depend on **abstractions**, not specific classes. Write `List<String> list = new ArrayList<>()` instead of `ArrayList<String> list = ...`. Then you can swap `ArrayList` for `LinkedList` without touching the code that uses `list`.

This is Refactoring Guru's principle "**Program to an interface, not an implementation**", and it's the core of the **Dependency Inversion Principle**. Benefits: lower coupling, easy testing with fakes, and new implementations can be plugged in — exactly how Strategy, Factory Method, and Observer work.

1.11 Anything can be built with other paradigms — why do we need OOP?

A: True — procedural or functional code can compute anything. OOP isn't about what's *possible*; it's about **managing change in large systems**:

- **Models the real domain** (Customer, Order) so design is easier to reason about.
- **Encapsulation** limits how far a change or bug can spread.
- **Polymorphism + inheritance** let you add behaviour without rewriting old code (Open/Closed).
- **Design patterns** — proven, shareable solutions — are expressed with objects and interfaces.
- Teams can split work along class boundaries.

In short: OOP trades a little upfront structure for long-term **flexibility, reuse, and maintainability**.

PART 2 — SOLID Principles ★ Teacher focus

Teacher's focus: know **what each principle actually targets** — not just the slogan. Two points discussed in class: the *real* meaning of Single Responsibility, and that "closed for modification" does **not** apply to every file.

2.1 SOLID at a glance — what does each principle target? ★ Teacher focus

A:

Principle	Slogan	What problem it targets	Smell when violated	Patterns that help
S — Single Responsibility	A class should have one reason to change	Changes from different people/areas colliding in one class	Divergent Change, Large Class	Extract Class, Facade
O — Open/Closed	Open for extension, closed for modification	Editing tested, working code every time a new variant appears	Switch Statements, Shotgun Surgery	Strategy, Decorator, Factory Method, Template Method, Observer
L — Liskov Substitution	Subtypes must be usable wherever the parent is	Broken inheritance — subclasses that surprise callers	Refused Bequest, instanceof checks	Composition instead of inheritance
I — Interface Segregation	Clients shouldn't depend on methods they don't use	Fat interfaces forcing dummy implementations	Refused Bequest, empty methods	Adapter, splitting interfaces
D — Dependency Inversion	Depend on abstractions, not concretions	High-level policy tied to low-level details (DB, email API)	Hard-to-test code, new everywhere	Abstract Factory, Factory Method, Strategy, Bridge

2.2 What does "Single Responsibility" actually mean? ★ Teacher focus

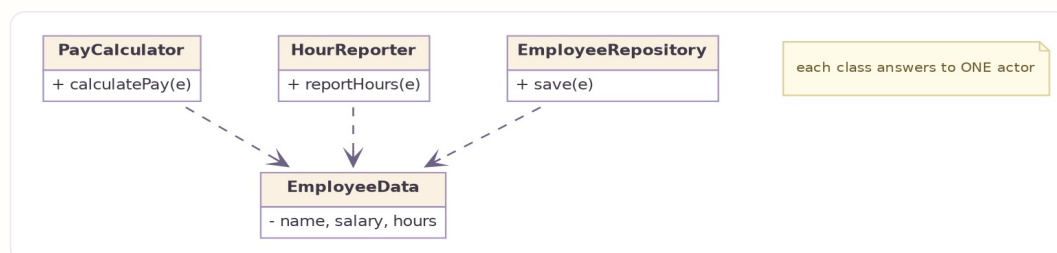
A: It does **not** mean "a class does only one thing" or "a class has only one method". It means:

A class should have only one reason to change — it should serve one actor (one group of people/one area of the business).

If the accountants, the HR team, and the database admins can all ask for changes to the same class, that class has **three** responsibilities — even if it's small.



Violates SRP: three different actors can force changes into one class. A change for Accounting can accidentally break HR's report.



Follows SRP: each class answers to one actor. Changes stay isolated.

Why it matters: when two responsibilities share a class, a change for one can silently break the other, and merges between teams conflict. SRP is really about **high cohesion** — keep together what changes together, separate what

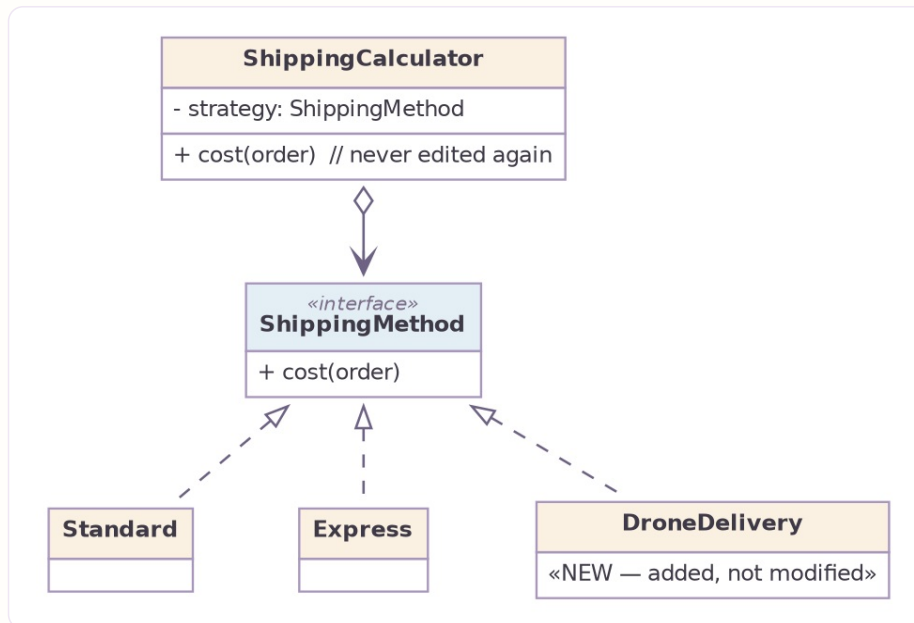
changes for different reasons.

Common mistakes: splitting a class into dozens of one-method classes (over-fragmentation), or thinking a 50-line class automatically violates SRP. Size isn't the test — *reasons to change* are.

2.3 Explain Open/Closed. Is "closed for modification" applicable to every file?

★ Teacher focus

A: OCP: you should be able to add new behaviour by **adding new code** (new classes), not by **editing existing, tested code**.



OCP in action: a new delivery method is a new class. ShippingCalculator is never edited.

"Closed for modification" is NOT applicable to all files. It's a goal for the *stable core* of the system along the *likely axes of change*, not a rule for every file:

- **Composition root** / **main** / **configuration** / **wiring code** — somebody has to say `new DroneDelivery()` and plug it in. That place *must* change. OCP pushes modification to these few spots instead of spreading it everywhere.
- **Factories** — a simple factory with a registry or `switch` is edited when a new type arrives. That's acceptable: the edit is in one place.
- **Bug fixes** — a wrong result must be fixed by modifying the code. OCP never forbids fixing bugs.
- **Data classes** / **DTOs** / **UI layouts** / **scripts** / **config files** — they change with requirements; wrapping them in abstractions would be over-engineering.
- **Unpredicted changes** — you can only be "closed" against changes you foresaw. You can't design for every possible change (that leads to Speculative Generality).
- **Early-stage code** — until you see real variation, keep it simple; refactor toward OCP when the second or third variant appears.

Key sentence for the exam: OCP is strategic — we choose which kinds of change to close against, and we accept modification in wiring/configuration code.

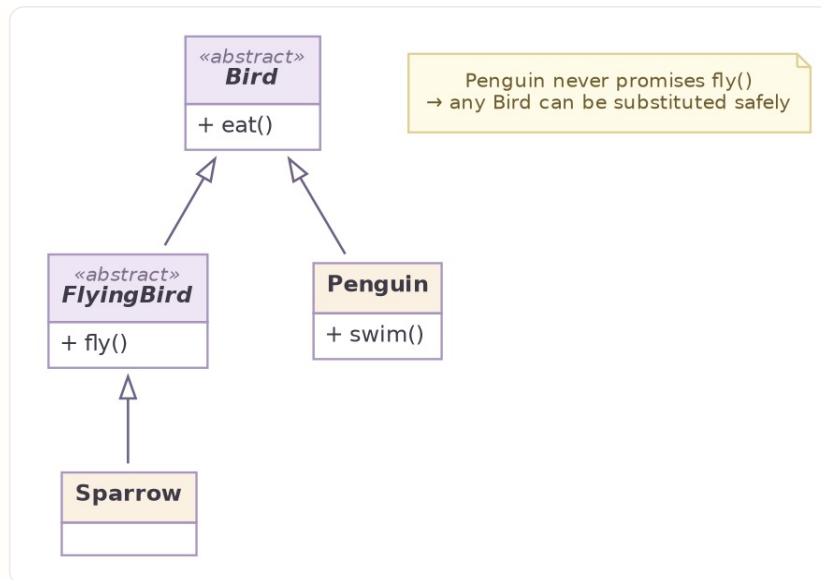
2.4 Explain Liskov Substitution with an example.

A: LSP: if code works with a parent type, it must still work correctly when given **any** subtype — no surprises.

Rules a subclass must follow:

- Don't **demand more** (stronger preconditions) than the parent.
- Don't **promise less** (weaker postconditions/results).
- Don't throw **new kinds of exceptions** the caller doesn't expect.
- Keep the parent's **invariants** (rules that are always true).

Classic violations: `Penguin extends Bird` where `fly()` throws an exception; `Square extends Rectangle` where `setWidth()` also changes the height (breaks code that expects width and height to be independent).

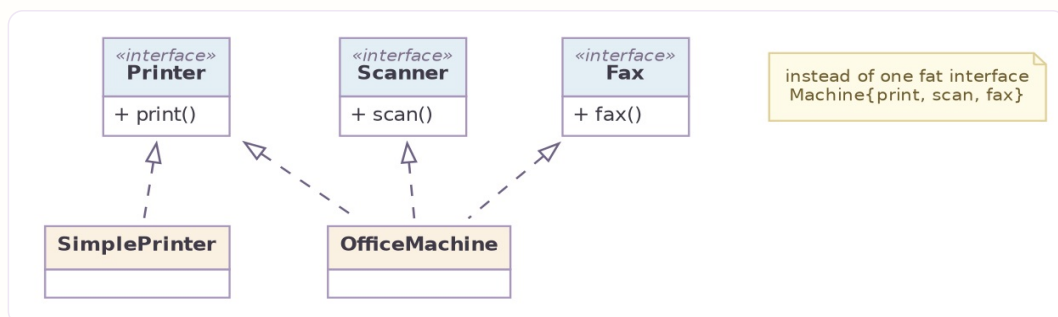


Fix: only birds that can fly promise fly(). Every Bird is now safely substitutable.

LSP is what makes **polymorphism safe**. Violations show up as `if (x instanceof Penguin)` checks — a smell.

2.5 Explain Interface Segregation.

A: ISP: many small, specific interfaces are better than one big general one. A client should not be forced to implement or depend on methods it doesn't use.



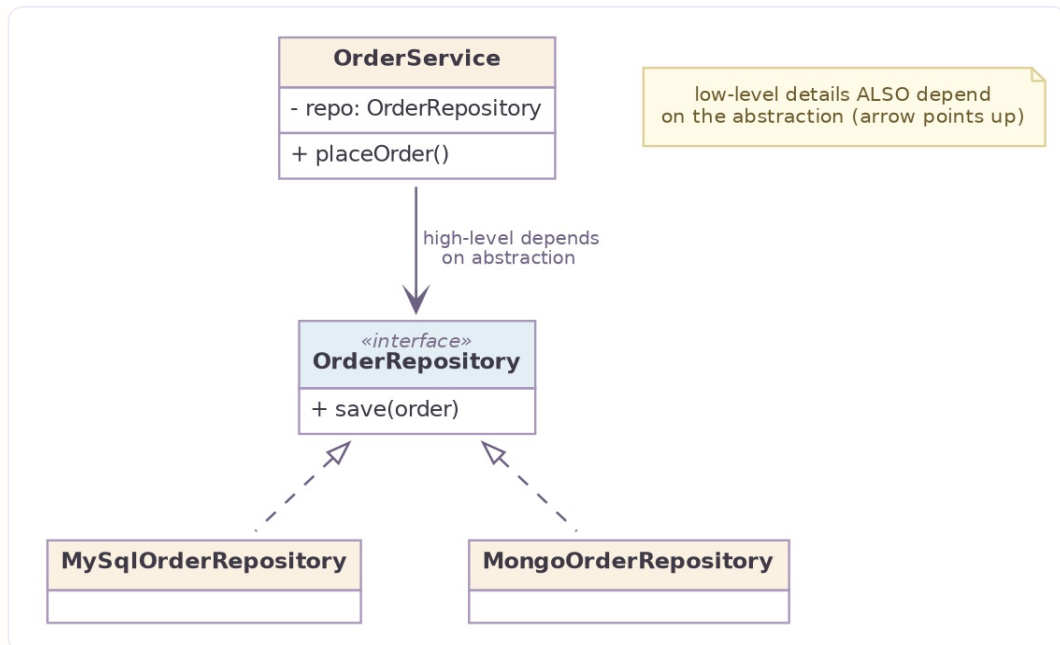
A simple printer implements only Printer. The office machine implements all three. Nobody writes an empty fax() method.

Targets: fat interfaces that force classes into empty or exception-throwing methods (which also breaks LSP), and needless recompiles/changes when an unrelated method changes.

2.6 Explain Dependency Inversion. How is it different from Dependency Injection?

A: DIP:

1. High-level modules (business rules) should not depend on low-level modules (DB, email, files). **Both** should depend on abstractions.
2. Abstractions should not depend on details; details depend on abstractions.



The high-level *OrderService* owns the *OrderRepository* interface. *MySQL* and *Mongo* implementations depend upward on it — the dependency arrow is "inverted".

DIP vs Dependency Injection: DIP is a **design principle** (which way dependencies point). Dependency Injection is a **technique** (passing the dependency in through a constructor/setter instead of creating it inside). DI is a common way to achieve DIP.

2.7 How do SOLID, code smells, and design patterns relate?

A: They're three views of the same goal — code that is easy to change:

- **Code smells** are the *symptoms* (e.g. Switch Statements).
- **SOLID** names the *principle being broken* (e.g. Open/Closed).
- **Refactoring** is the *process* of fixing it, and a **design pattern** is often the *destination* (e.g. Replace Conditional with Polymorphism → Strategy/State).

Example chain: `switch(type)` in 5 methods → smell: Switch Statements + Shotgun Surgery → violates OCP → refactor: Replace Type Code with Subclasses/State → pattern: **State** or **Strategy**.

PART 3 — Code Smells: Effects & Solutions

Teacher's focus: **don't memorize the list — understand the effect of each smell and how to fix it.** Use the insights from your "smells in the wild" project. The tables below are built around *effect* → *fix* so you can reason your way to an answer even for a smell you've never seen.

3.1 Code smells by effect — what goes wrong and how to fix it ★ Teacher focus

A: Refactoring Guru groups smells into five families. Learn the **family's effect** — the individual smells follow from it.

Family	Effect on the project	Smells in it	Main fixes
Bloaters	Code grows so big it's hard to read, test, and change; bugs hide inside	Long Method, Large Class, Long Parameter List, Primitive Obsession, Data Clumps	Extract Method, Extract Class, Introduce Parameter Object, Replace Data Value with Object
OO Abusers	OOP features used wrongly → type-checking <code>if/switch</code> everywhere, broken inheritance	Switch Statements, Refused Bequest, Temporary Field, Alternative Classes with Different Interfaces	Replace Conditional with Polymorphism, Replace Inheritance with Delegation, Extract Class
Change Preventers	One change forces edits in many places → slow, risky changes	Divergent Change, Shotgun Surgery, Parallel Inheritance Hierarchies	Extract Class (split), Move Method/Field (gather), merge hierarchies
Dispensables	Useless code adds noise and maintenance cost	Comments (as deodorant), Duplicate Code, Lazy Class, Data Class, Dead Code, Speculative Generality	Extract Method, Inline Class, Collapse Hierarchy, delete it
Couplers	Classes know too much about each other → changes ripple	Feature Envy, Inappropriate Intimacy, Message Chains, Middle Man	Move Method, Hide Delegate, Remove Middle Man

3.2 The most-tested smells: effect → fix (one line each)

A:

Smell	Effect (why it hurts)	Fix
Long Method	Hard to understand; logic duplicated inside; hard to test pieces	Extract Method, Decompose Conditional, Replace Temp with Query
Large Class	Many reasons to change (SRP broken); merge conflicts	Extract Class / Extract Subclass
Long Parameter List	Calls are confusing, easy to pass args in wrong order	Introduce Parameter Object, Preserve Whole Object
Primitive Obsession	Validation/format rules scattered and duplicated	Replace Data Value with Object, Replace Type Code with Class/Subclasses
Data Clumps	Same group of fields travels everywhere	Extract Class / Parameter Object
Switch Statements	Every new type = edit every switch (breaks OCP)	Replace Conditional with Polymorphism → Strategy/State
Refused Bequest	Subclass rejects parent's methods → LSP broken	Push Down Method, Replace Inheritance with Delegation
Divergent Change	One class changed for many different reasons	Extract Class
Shotgun Surgery	One change = many small edits across classes	Move Method/Field into one class
Parallel Inheritance	Adding a subclass forces a matching subclass elsewhere	Move methods to merge the two hierarchies
Duplicate Code	Fix a bug in one copy, forget the other	Extract Method, Pull Up Method, Form Template Method
Comments (deodorant)	Comment hides unclear code; comments go stale	Extract Method + rename so code explains itself
Lazy Class	Costs maintenance, adds nothing	Inline Class, Collapse Hierarchy
Speculative	Unused "future-proof" code confuses readers	Collapse Hierarchy, Inline Class, Remove Parameter

Generality		
Feature Envy	Method uses another class's data more than its own	Move Method (+ Extract Method)
Message Chains	<code>a.getB().getC().getD()</code> — client breaks if chain changes	Hide Delegate
Middle Man	Class only forwards calls	Remove Middle Man

3.3 Insights from "smells in the wild" — how smells behave in real projects

★ Teacher focus

A: Points to use when the exam asks about smells in real code (add your own project's examples next to these):

- **Smells come in groups.** A Long Method usually also has Duplicate Code and "deodorant" Comments; a Large Class usually has Divergent Change. Fixing the root one (Extract Method/Class) removes several at once.
- **A smell is a symptom, not a bug.** The code works — the damage shows up later as slow, risky change. Always explain the *effect*.
- **Context matters.** A long but linear, well-named method may be fine; a `switch` that appears once in a factory is acceptable. Judge by how often that code changes and how many places a change touches.
- **Fix the hotspots first.** Refactor code that changes often or has many bugs; leave stable, rarely-touched code alone.
- **Smells hide real bugs.** Tangled conditions (like `getTitle().contentEquals(author)` in the findBooks exam question) survive because nobody can read them.
- **Refactor in small steps with tests** — refactoring must never change behaviour.
- **Many smells point to a pattern.** Switch on type → Strategy/State; null checks → Null Object; duplicated algorithm steps → Template Method; many objects talking to each other → Mediator.

3.4 What is refactoring? Why do we refactor?

asked 2x

A: Refactoring = improving the **internal structure** of code **without changing its external behaviour**. Done in small steps, running tests after each.

We refactor to: make code easier to read, find bugs faster, add features faster (clean code is cheaper to extend), and pay off technical debt before it grows.

3.5 What advice must we keep in mind before refactoring?

A:

1. **Have tests first** — they prove behaviour didn't change.
2. **Small steps** — one change, run tests, repeat.
3. **Don't mix refactoring with new features** — separate commits.
4. **Code must stay working** after each step.
5. **Know when to stop** — refactor to what's needed now, not to an ideal.
6. **Communicate** with the team when touching shared code.

3.6 Define code smell. Name 10 common code smells.

asked 2x

A: A **code smell** is a surface sign in code that usually points to a **deeper design problem**. It isn't a bug; the code runs, but it's harder to understand and change.

Ten common ones: **Long Method, Large Class, Long Parameter List, Duplicate Code, Switch Statements, Feature Envy, Data Clumps, Primitive Obsession, Shotgun Surgery, Message Chains**. (Also: Refused Bequest, Lazy Class, Speculative Generality, Comments, Divergent Change, Middle Man, Parallel Inheritance Hierarchies.)

3.7 What is Feature Envy? Remedies?

asked 3x

A: A method that is **more interested in another class's data than its own** — it keeps calling another object's getters.

Effect: logic about `Customer` is scattered in other classes; changing `Customer` breaks them.

Remedy: Move Method to the class whose data it uses. If only part of it envies, **Extract Method** first, then move that part. (Exception: Strategy/Visitor intentionally separate behaviour from data.)

3.8 Describe Parallel Inheritance Hierarchies, Speculative Generality, Feature Envy, and Message Chains, and the problems they cause.

A:

- **Parallel Inheritance Hierarchies** — every time you add a subclass in one hierarchy, you must add a matching one in another (e.g. `Engineer` needs `EngineerReport`). *Problem:* change is doubled and easy to forget. *Fix:* make one hierarchy refer to the other, then move methods and delete the extra hierarchy.
- **Speculative Generality** — classes, parameters, or hooks created "for the future" that nothing uses. *Problem:* more code to read and maintain for zero benefit. *Fix:* Collapse Hierarchy, Inline Class, Remove Parameter.
- **Feature Envy** — see 3.7. *Problem:* behaviour lives far from its data; high coupling.
- **Message Chains** — `order.getCustomer().getAddress().getCity()`. *Problem:* the client depends on the whole navigation path; any change in the chain breaks it. *Fix:* **Hide Delegate** (`order.getCustomerCity()`), or Extract + Move Method.

3.9 Probable causes of Long Method and Large (Long) Class.

A: Long Method: adding "just one more line" is easier than creating a new method; copy-paste instead of reuse; one method handling many steps and cases; fear of "too many small methods".

Large Class: a class keeps absorbing new features ("it's already there"); a vague "Manager/Util" class becomes a dumping ground; fear of splitting working code; no reviews catching growth. Both are **Bloaters** and grow slowly over time.

3.10 "A method is long when it has more than 20 lines" — comment. When is a method long? Remedies? asked 2x

A: Line count is only a rough hint. A method is **long** when it's hard to understand at a glance — it does **several things**, mixes levels of detail, or needs **comments to explain its sections**. Refactoring Guru's advice: if you feel the need to comment a block, extract it into a method. A 25-line straight sequence of named steps can be fine; an 8-line nested puzzle may not.

Remedies: Extract Method, Replace Temp with Query, Introduce Parameter Object, Replace Method with Method Object, Decompose Conditional.

3.11 "Comments are often used as deodorant" — justify. Remedies? asked 4x — most repeated

A: Deodorant hides a bad smell without removing its cause. Likewise, a comment is often written to explain confusing code instead of making the code clear. **Effect:** the code stays hard to read, and comments go out of date and start to lie.

Remedies:

- Comment explains a block → **Extract Method** with a name that says it.
- Comment explains an expression → **Extract Variable** with a clear name.
- Comment explains a method → **Rename Method**.
- Comment states a required condition → **Introduce Assertion**.

Keep comments only for **why** (business reasons, workarounds), never for **what** the code does.

3.12 What is semantic duplication? Example.

A: Two pieces of code that **look different but do the same thing**. Text-matching tools miss it.

Example: `validateTitle()` returns `!title.isEmpty() && !title.contains("\n")`, while `validateAuthor()` does the same checks with three `if` statements. Different shape, same meaning → merge into one `isValid(String field)`.

3.13 What is Parameter Object? Example.

A: A refactoring that replaces a **group of parameters that always travel together** with one object.

Before: `createEvent(int day, int month, int year, int startHour, int endHour)` After: `createEvent(EventTime time)` where `EventTime` holds date + start + end.

Removes: Long Parameter List and Data Clumps. Bonus: validation (end after start) moves into `EventTime`.

3.14 What are the different levels of duplication? Example for each. asked 2x

A:

1. **Within a method** — the same expression repeated → Extract Variable.
2. **Between methods in one class** — e.g. `GetCsv()` and `GetCsvWithHeader()` repeat the same loop → Extract Method.
3. **Between sibling subclasses** — two subclasses have the same method → Pull Up Method; similar-but-different steps → Form Template Method.
4. **Between unrelated classes** — → Extract Class/Superclass.
5. **Semantic duplication** — different code, same logic (see 3.12) → Substitute Algorithm, then unify.

3.15 What is Refused Bequest? Example and refactor. asked 3x

A: A subclass **inherits methods/data it doesn't want** — it ignores them, or overrides them to throw errors. **Effect:** the inheritance is a lie; code expecting the parent breaks (LSP violation).

Example: `Penguin extends Bird` and overrides `fly()` to throw `UnsupportedOperationException`.

Refactor: if inheritance makes sense otherwise, **Push Down Method** (`fly()` moves into `FlyingBird`). If the child isn't really a parent type, **Replace Inheritance with Delegation** (hold the other class as a field and use only what you need). See the LSP diagram in Part 2.

3.16 What is Primitive Obsession? Example, refactor, remedies. asked 4x

A: Using **basic types** (`int`, `String`, `double`) for **domain concepts** that deserve a class — money, phone numbers, ranges, type codes. **Effect:** validation and formatting logic is copied everywhere the primitive is used.

Example: `double amount; String currency;` scattered across `Order`, `Invoice`, `Refund`. **Refactor:** create `Money { amount; currency; add(); convert(); }` and use it everywhere.

Remedies: Replace Data Value with Object, Introduce Parameter Object, Replace Type Code with Class / Subclasses / State-Strategy, Replace Array with Object.

3.17 What is a Lazy Class? Example and refactor. asked 2x

A: A class that **doesn't do enough to earn its place** — maybe it was cut down by refactoring, or created for future use that never came. **Effect:** another file to read and maintain, for nothing.

Example: `PersonName { first; last; getFull(); }` used in one place only. **Refactor: Inline Class** (move its members into `Person`), or **Collapse Hierarchy** if it's a near-empty subclass.

3.18 What is Oddball Solution? Example.

A: The **same problem is solved in different ways** in different parts of the code. Example: the project has a `Cache` class, but one module builds its own `HashMap` + expiry logic. **Effect:** two things to learn, test, and fix. **Fix:** pick the best solution (Substitute Algorithm) and use it everywhere; an Adapter can help if interfaces differ.

3.19 Which of these are refactoring? (yes/no) asked 2x

A: i. Refactor the code to support logging — **No** (new behaviour) ii. Refactor to authenticate against LDAP instead of DB — **No** (behaviour change) iii. Refactor to eliminate duplicate code — **Yes** iv. This class is too big, refactor it — **Yes** v. Implement caching to improve performance — **No** (new behaviour / optimization)

Rule: **adds or changes behaviour → not refactoring. Only restructures → refactoring.**

3.20 What is the relation between design patterns and refactoring? asked 2x

A: Refactoring is the **process**; patterns are often the **destination**. You notice a smell, refactor step by step, and the code ends up in a known pattern shape — e.g. a type-code `switch` refactored with Replace Conditional with Polymorphism becomes **Strategy/State**; duplicated algorithm steps become **Template Method** (Form Template Method); null checks become **Null Object**. Patterns tell you *where* to go; refactoring gets you there safely without breaking behaviour. Also: don't force patterns in upfront — refactor toward them when the need appears.

3.21 What is "Collapse Hierarchy"? Example. Which smells does it remove? asked 3x

A: When a **subclass and its superclass are practically the same**, merge them into one class.

Example: `Manager extends Employee` but after refactoring `Manager` adds nothing → move anything left into `Employee` and delete `Manager`.

Removes: **Lazy Class** and **Speculative Generality** (hierarchies made "just in case"). Also reduces Parallel Inheritance Hierarchies.

3.22 What does Extract Method mean? Which smells does it remove? asked 2x

A: Take a fragment of code that can be grouped, move it into a **new method with a name that explains its purpose**, and call it from the old place.

Removes: **Long Method**, **Duplicate Code** (the new method is reused), **Comments** (the name replaces the comment), and helps with **Feature Envy** (extract, then move).

PART 4 — Code Smell Exercises (real exam code)

For each: the **original question code**, the **smells + their effect**, and the **refactor**. Focus on *why* each smell hurts — that's what the teacher marks.

4.1 Employee with `_type` and `switch` in `PayAmount()` (2013)

Original question

```
public class Employee {
    private int _type;
    private const int ENGINEER = 0;
    private const int SALESMAN = 1;
    private const int MANAGER = 2;
    private double _monthlySalary;
    private double _commission;
    private double _bonus;

    public Employee(int type, double salary, double commission, double bonus) { ... }

    public double PayAmount() {
        switch (_type) {
            case ENGINEER: return _monthlySalary;
            case SALESMAN: return _monthlySalary + _commission;
            case MANAGER: return _monthlySalary + _bonus;
            default: throw new Exception("Incorrect Employee");
        }
    }
}
```

Refactor the code and mention the approach of refactoring you applied.

A — Smells & effects:

- **Primitive Obsession** (`int _type` + magic constants) → the compiler can't stop `new Employee(7, ...)`; type rules are scattered.
- **Switch Statements** → every new employee type means editing this switch (and any other switch on `_type`) — breaks **OCP**, risks **Shotgun Surgery**.
- **Temporary Field** → `_commission` is meaningless for engineers, `_bonus` for salesmen.

Refactoring applied: **Replace Type Code with Subclasses** + **Replace Conditional with Polymorphism**.

```
abstract class Employee {
    protected double monthlySalary;
    abstract double payAmount();
}
class Engineer extends Employee {
    double payAmount() { return monthlySalary; }
}
class Salesman extends Employee {
    private double commission;
    double payAmount() { return monthlySalary + commission; }
}
class Manager extends Employee {
    private double bonus;
    double payAmount() { return monthlySalary + bonus; }
}
```

New type = new subclass; no existing code changes. (If an employee's type can change at runtime, use **Replace Type Code with State/Strategy** instead of subclasses.)

4.2 Validator / ValidatorFactory asked 4x: 2013, 2014, 2017, 2024

Original question

```
class Validator {
    private bool result;
    public bool IsValid(object obj) {
        // in this method we have a code that can validate
        // string/int/double/float objects
        // result is set here using some code
        return result;
    }
}

class ValidatorFactory {
    public Validator CreateValidator(string type) {
        if (type == "string" || type == "int" ||
            type == "double" || type == "float")
            return new Validator();
        return null;
    }
}

// client
Validator validator = new ValidatorFactory().CreateValidator(args[0]);
if (validator != null)
    validator.IsValid(args[1]);
```

What smells do you get? Refactor the code to remove the smells. (Don't treat the comment as a smell.)

A — Smells & effects:

- **Primitive Obsession** — `type` is a raw string; typos like `"Int"` fail silently.
- **Conditional complexity / Switch Statements** — long `||` chain; adding a type means editing it.
- **One class validates four types** (`IsValid` does everything) → Divergent Change; a change for strings risks floats.
- **Temporary Field** — `result` is a field only used inside one call; should be a local return value.
- **Null return** → every client must write `if (validator != null)` (null-check smell).
- **Lazy/pointless factory** — it always returns the same `Validator` class, so the "factory" doesn't really choose anything.

Refactor — one validator per type, a lookup instead of the if-chain, and a Null Object instead of `null`:

```
interface Validator { boolean isValid(Object obj); }

class StringValidator implements Validator { public boolean isValid(Object o) { /* ... */ return true; } }
class IntValidator implements Validator { public boolean isValid(Object o) { /* ... */ return true; } }
class DoubleValidator implements Validator { public boolean isValid(Object o) { /* ... */ return true; } }
class FloatValidator implements Validator { public boolean isValid(Object o) { /* ... */ return true; } }

class NullValidator implements Validator { // replaces 0
    public boolean isValid(Object o) { return false; }
}

class ValidatorFactory {
    private static final Map<String, Validator> validators = Map.of(
        "string", new StringValidator(),
        "int", new IntValidator(),
        "double", new DoubleValidator(),
        "float", new FloatValidator());

    public Validator createValidator(String type) {
        return validators.getOrDefault(type, new NullValidator());
    }
}

// client — no null check any more
new ValidatorFactory().createValidator(args[0]).isValid(args[1]);
```

Techniques named: **Replace Conditional with Polymorphism**, **Replace Temp Field with local/return value**, **Introduce Null Object**, lookup map instead of `||` chain.

4.3 CsvGenerator — three CSV methods (2014, 2017)

Original question

```
public class Person {
    public string Name;
    public string Email;
    public int Age;
}

public class CsvGenerator {
    public string GetCsvWithHeader(List<Person> persons) {
        string str = "Name,Email,Age\n";
        for (int i = 0; i < persons.Count; i++)
            str += persons[i].Name + "," + persons[i].Email + "," + persons[i].Age + "\n";
        return str;
    }

    public string GetCsv(List<Person> persons) {
        string str = "";
        for (int i = 0; i < persons.Count; i++)
            str += persons[i].Name + "," + persons[i].Email + "," + persons[i].Age + "\n";
        return str;
    }

    public string GetCsvWithHeaderAndFooter(List<Person> persons) {
        string str = "Name,Email,Age\n";
        foreach (var person in persons)
            str += person.Name + "," + person.Email + "," + person.Age + "\n";
        str += "Total no of persons: " + persons.Count;
        return str;
    }
}
```

What smells? Refactor and mention the approach.

A — Smells & effects:

- **Duplicate Code** — the row loop appears 3 times; adding a column means 3 edits (and one will be forgotten).
- **Data Class + Feature Envy** — `Person` is just public fields; `CsvGenerator` reaches in to format a person. Formatting a person belongs to `Person`.
- **Public fields** break encapsulation.
- Oddball loop styles (`for` vs `foreach`) for the same job.

Refactoring applied: **Extract Method**, **Move Method** (row formatting into `Person`), **Encapsulate Field**.

```
class Person {
    private String name, email;
    private int age;
    String toCsvRow() { return name + "," + email + "," + age + "\n"; }
}

class CsvGenerator {
    private static final String HEADER = "Name,Email,Age\n";

    private String rows(List<Person> persons) {
        StringBuilder sb = new StringBuilder();
        for (Person p : persons) sb.append(p.toCsvRow());
        return sb.toString();
    }

    public String getCsv(List<Person> persons) { return rows(persons); }
    public String getCsvWithHeader(List<Person> persons) { return HEADER + rows(persons); }
    public String getCsvWithHeaderAndFooter(List<Person> persons) {
        return HEADER + rows(persons) + "Total no of persons: " + persons.size();
    }
}
```

4.4 foundPerson(String[] people) (2013)

Original question

```
String foundPerson(String[] people) {
    for (int i = 0; i < people.length; i++) {
        if (people[i].equals("Don")) { return "Don"; }
        if (people[i].equals("John")) { return "John"; }
        if (people[i].equals("Kent")) { return "Kent"; }
    }
    return "";
}
```

Find the smells. Name the refactoring process. Write the refactored code.

A — Smells & effects: Duplicate Code (same `if/return` three times) and hard-coded data mixed with logic — adding a name means another copy-pasted block.

Refactoring process: Substitute Algorithm (replace the algorithm with a clearer one — this is Fowler's own example for it).

```
String foundPerson(String[] people) {
    List<String> candidates = Arrays.asList("Don", "John", "Kent");
    for (String person : people)
        if (candidates.contains(person))
            return person;
    return "";
}
```

4.5 toStringHelper(StringBuffer result) (2014)

Original question

```
private String toStringHelper(StringBuffer result) {
    result.append("<");
    result.append(name);
    result.append(attributes.toString());
    result.append(">");
    if (!value.equals(""))
        result.append(value);
    Iterator it = children().iterator();
    while (it.hasNext()) {
        TagNode node = (TagNode) it.next();
        node.toStringHelper(result);
    }
    result.append("</");
    result.append(name);
    result.append(">");
    return result.toString();
}
```

Find the smells. Name the refactoring process. Write the refactored code.

A — Smells & effects: Long Method doing four jobs (open tag, value, children, close tag) at mixed levels of detail — hard to read and change one part safely.

Refactoring process: Extract Method (Compose Method).

```

private String toStringHelper(StringBuffer result) {
    writeOpenTagTo(result);
    writeValueTo(result);
    writeChildrenTo(result);
    writeEndTagTo(result);
    return result.toString();
}
private void writeOpenTagTo(StringBuffer r) { r.append("<").append(name).append(attributes).append(">"); }
private void writeValueTo(StringBuffer r) { if (!value.equals("")) r.append(value); }
private void writeChildrenTo(StringBuffer r) {
    for (Object child : children()) ((TagNode) child).toStringHelper(r);
}
private void writeEndTagTo(StringBuffer r) { r.append("</").append(name).append(">"); }

```

Now the main method reads like a table of contents.

4.6 Customer / BillingPlan null check (2017, 2024)

Original question

```

Customer customer = site.getCustomer();
BillingPlan plan;
if (customer == null)
    plan = BillingPlan.basic();
else
    plan = customer.getPlan();

```

Refactor to remove the null checking. Write complete code with the necessary classes.

A — Smell & effect: repeated **null checks** — every caller must remember them; forgetting one causes a `NullPointerException`.

Refactoring: Introduce Null Object.

```

class Customer {
    private BillingPlan plan;
    public BillingPlan getPlan() { return plan; }
    public boolean isNull() { return false; }
}

class NullCustomer extends Customer {
    @Override public BillingPlan getPlan() { return BillingPlan.basic(); }
    @Override public boolean isNull() { return true; }
}

class Site {
    private Customer customer;
    public Customer getCustomer() {
        return (customer == null) ? new NullCustomer() : customer;
    }
}

// client — no null check
BillingPlan plan = site.getCustomer().getPlan();

```

4.7 findBooks / validateTitle / validateAuthor (2020, 2024)

Original question

```

public List<Book> findBooks(String title, String author,
                           LocalDate firstDate, LocalDate endDate) {
    List<Book> foundBooks = new ArrayList<Book>();
    if (validateTitle(title) && validateAuthor(author)) {
        for (int i = 0; i < books.size(); i++) {
            Book book = books.get(i);
            if (book.getPublishedOn().isAfter(firstDate) &&
                book.getPublishedOn().isBefore(endDate)) {
                foundBooks.add(book);
            } else if (book.getTitle().contentEquals(title) &&
                       book.getTitle().contentEquals(author)) {
                foundBooks.add(book);
            }
        }
    }
    return foundBooks;
}

public boolean validateTitle(String title) {
    return !title.isEmpty() && !title.contains("\n");
}

public boolean validateAuthor(String author) {
    if (author.isEmpty()) return false;
    if (author.contains("\n")) return false;
    return true;
}

```

a) Identify the code smells with brief explanation. b) Refactor to eradicate them.

A — Smells & effects:

- **Long Method / nested conditionals** (if → for → if/else-if) — hard to follow; this is how the bug below survived.
- **Duplicate Code (semantic)** — both validators do the same check in different shapes.
- **Long Parameter List + Data Clump** — `firstDate, endDate` always travel together → a `DateRange` object.
- **Duplicate branch body** — both branches do `foundBooks.add(book)` → merge conditions.
- **Hidden bug** — `getTitle().contentEquals(author)` should compare the author.

Refactoring: Extract Method, Decompose/Consolidate Conditional, Introduce Parameter Object, Replace Nested Conditional with Guard Clause.

```

public List<Book> findBooks(String title, String author, DateRange range) {
    List<Book> found = new ArrayList<>();
    if (!isValidText(title) || !isValidText(author)) return found; // guard clause
    for (Book book : books)
        if (range.contains(book.getPublishedOn()) || matches(book, title, author))
            found.add(book);
    return found;
}

private boolean isValidText(String s) { return !s.isEmpty() && !s.contains("\n"); }
private boolean matches(Book b, String title, String author) {
    return b.getTitle().contentEquals(title) && b.getAuthor().contentEquals(author);
}

class DateRange {
    private final LocalDate start, end;
    boolean contains(LocalDate d) { return d.isAfter(start) && d.isBefore(end); }
}

```

4.8 TextEditor / UserInterface tight coupling → Command (2024)

Original question

```
// Business Logic
public class TextEditor {
    public void save() { System.out.println("Saving file..."); }
    public void print() { System.out.println("Printing file..."); }
}

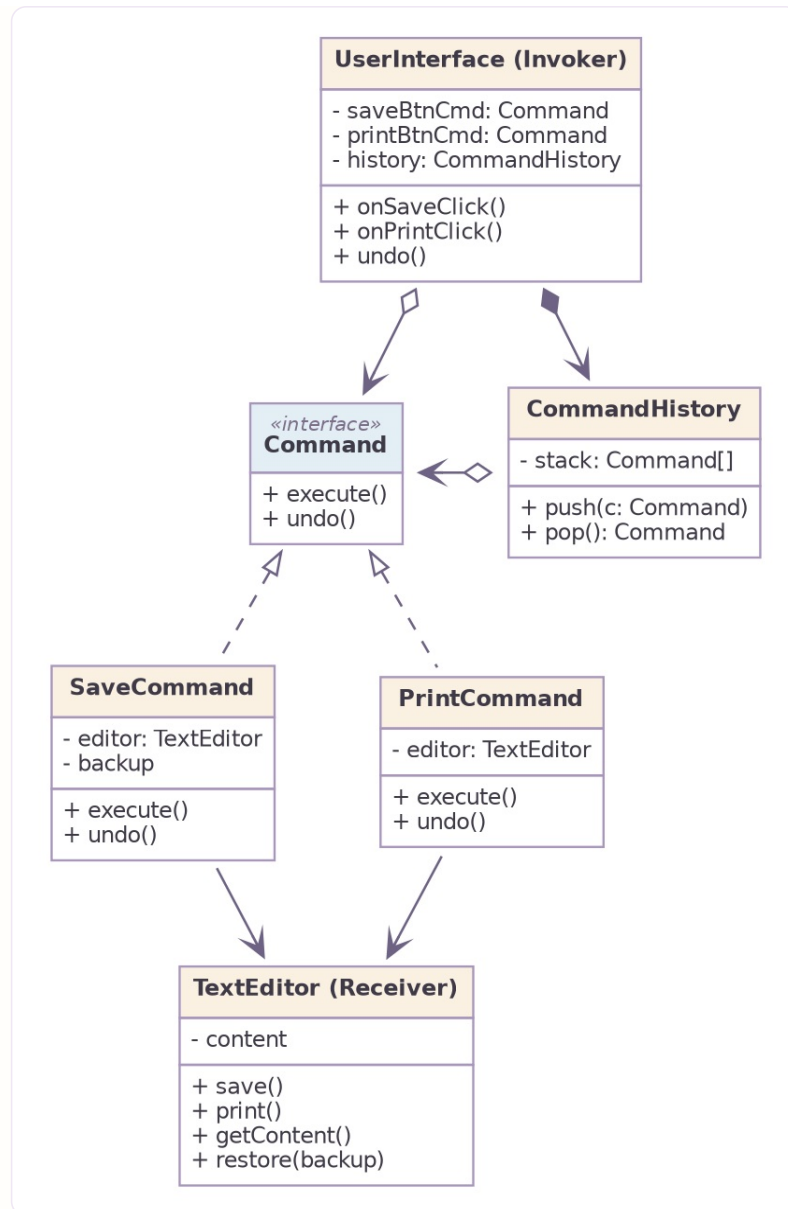
// UI Layer (Simplified)
public class UserInterface {
    private TextEditor editor = new TextEditor();

    public UserInterface() {
        // --- Tight Coupling ---
        // saveButton.addActionListener(e -> editor.save());
        // printButton.addActionListener(e -> editor.print());
    }
    public void simulateSaveClick() { editor.save(); } // Direct call
    public void simulatePrintClick() { editor.print(); } // Direct call
}
```

a) Why is the tight coupling a problem? What happens if you add "undo"? b) How does Command decouple invoker from receiver? Draw the UML. c) How would you implement a command history for undo?

a) The UI knows the concrete `TextEditor` and its exact methods, and creates it itself (`new`). Effects: UI can't be reused with another receiver; buttons can't be reconfigured; actions can't be **stored, queued, logged, or undone** because they're just direct calls. Adding **undo** would force the UI to track "what did I just do and how do I reverse it" with flags and switches — the UI becomes a Large Class mixing presentation and business logic (SRP broken), and every new action edits the UI (OCP broken).

b) Command turns each request into an **object** with `execute()` . The **Invoker** (UI/button) holds a `Command` reference and only calls `execute()` . The **ConcreteCommand** knows the **Receiver** (`TextEditor`) and which method to call. So the invoker never sees the receiver.



Command design for the text editor. The UI depends only on the Command interface; each command knows the receiver; CommandHistory enables undo.

c) Add `undo()` to `Command`. Before executing, each command saves the state it needs to reverse itself (a backup of the text). After a successful `execute()`, the invoker pushes the command onto `CommandHistory` (a stack). **Undo** = pop the last command and call its `undo()`. For big/complex states, combine with **Memento**: the command stores an editor snapshot and undo restores it.

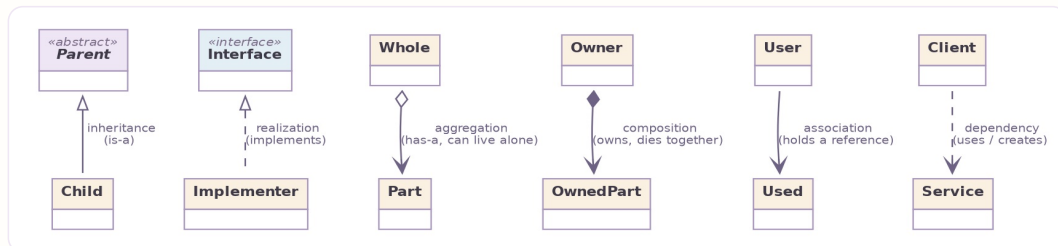
PART 5 — Pattern Catalog: Signatures, UML, When / When Not

★ Teacher focus

Teacher: "While studying a pattern, focus on what part of its design is the identifying signature." Each card below has a 🦋 **Signature** line — the one structural feature that tells you "this is pattern X" in a UML diagram. The diagrams follow Refactoring Guru's structure diagrams. No code is needed in the exam — learn to *draw* these.

How to read the UML in this guide

A: These are the six relationships you'll draw in every answer:



UML relationship legend. Triangle arrow = inheritance/implementation; diamond = the "whole" side of has-a (hollow = aggregation, filled = composition); open arrow = association; dashed open arrow = dependency.

Also: `«interface»` and italic *abstract* class names, `+` public, `-` private, `#` protected.

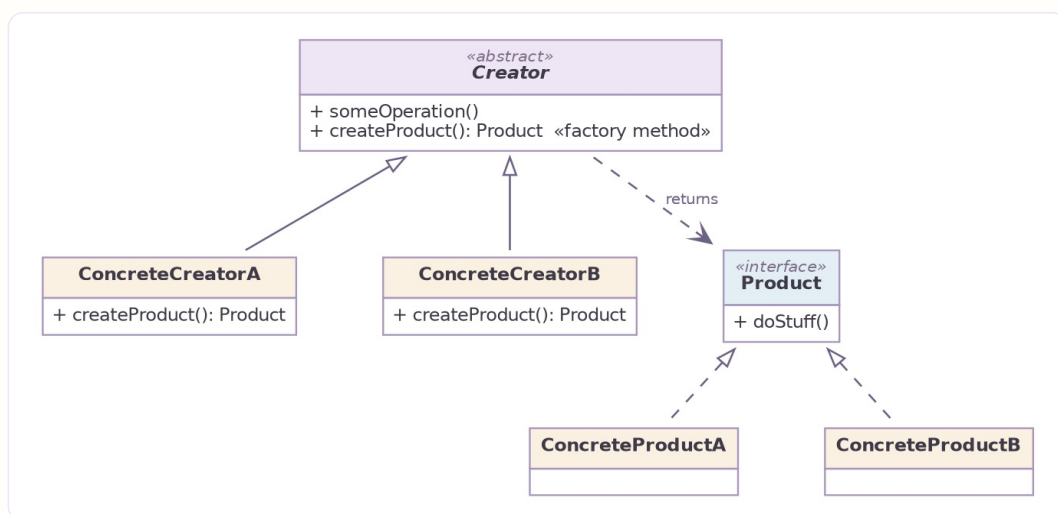
Creational Patterns

5.1 Factory Method

Intent: Define a method for creating objects in a superclass, and let **subclasses decide which class** to instantiate.

Problem (RG): A logistics app only created `Truck` s. Now it must support `Ship` s. Code full of `new Truck()` would need `if/else` everywhere.

🦋 **Signature:** a **creator class hierarchy parallel to a product hierarchy**; an abstract/overridable `createProduct()` method **returning the product interface**, overridden in each concrete creator.



Factory Method structure

Participants: Product (interface), ConcreteProducts, Creator (declares factory method, often has business logic that uses the product), ConcreteCreators (override it).

✓ **Use when:** you don't know ahead of time the exact types your code will work with; you want users of your framework to extend its internal components; you want to reuse existing objects (pooling) behind a creation method.

✗ **Don't use when:** only one product type exists and won't change (plain `new` is simpler); you'd need a new creator subclass for every product just to create it (class explosion).

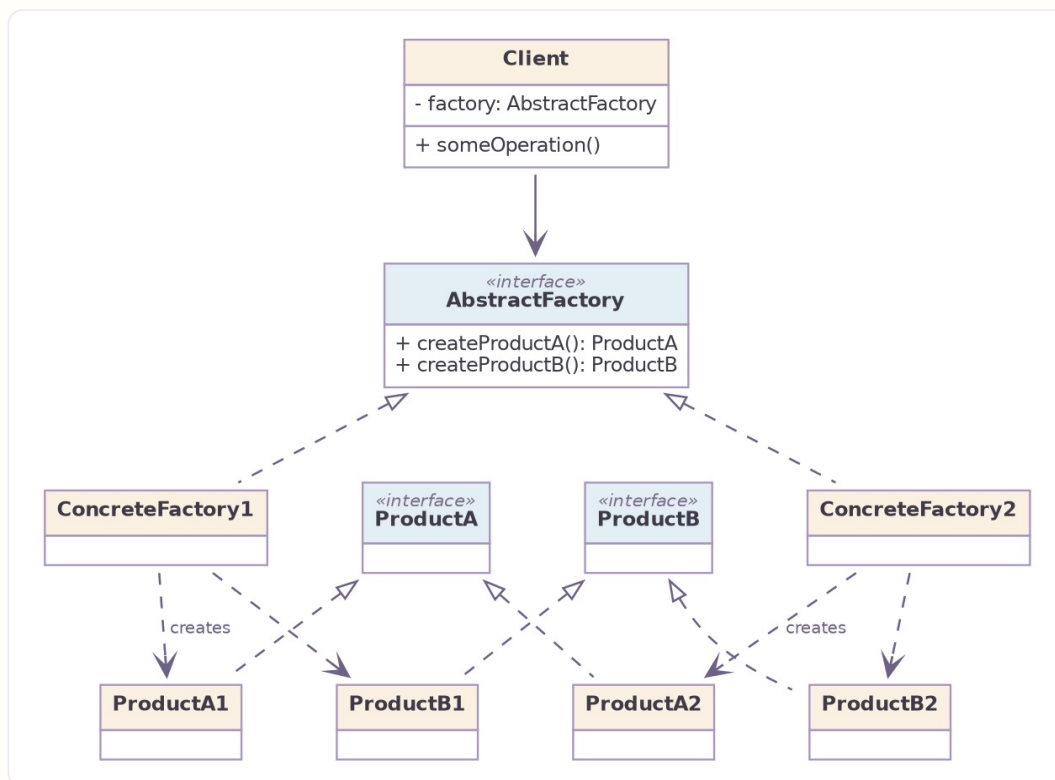
Similar: Abstract Factory (families, many factory methods), Template Method (Factory Method is a specialised Template Method step), Prototype (creates by cloning, no subclassing of creator).

5.2 Abstract Factory

Intent: Produce **families of related objects** without specifying their concrete classes.

Problem (RG): A furniture shop sells Chair + Sofa + CoffeeTable in Modern / Victorian / ArtDeco styles. A customer must never get a Modern chair with a Victorian sofa.

✎ **Signature:** one factory interface with several `createX()` methods — one per product type — and one concrete factory per family (a grid: families × product types).



Abstract Factory structure

Participants: AbstractProducts (one interface per product type), ConcreteProducts (per family), AbstractFactory, ConcreteFactories (one per family), Client (uses only interfaces).

✓ **Use when:** code must work with various families of related products and **consistency between products matters**; you want to switch a whole family at once (theme, OS, region).

✗ **Don't use when:** there's only one product type (use Factory Method); adding **new product types** is frequent — every factory must change (it's easy to add families, hard to add product kinds).

Similar: Factory Method (one product), Builder (one complex product, step by step), Prototype (factories can be built from prototypes). Concrete factories are often **Singletons**.

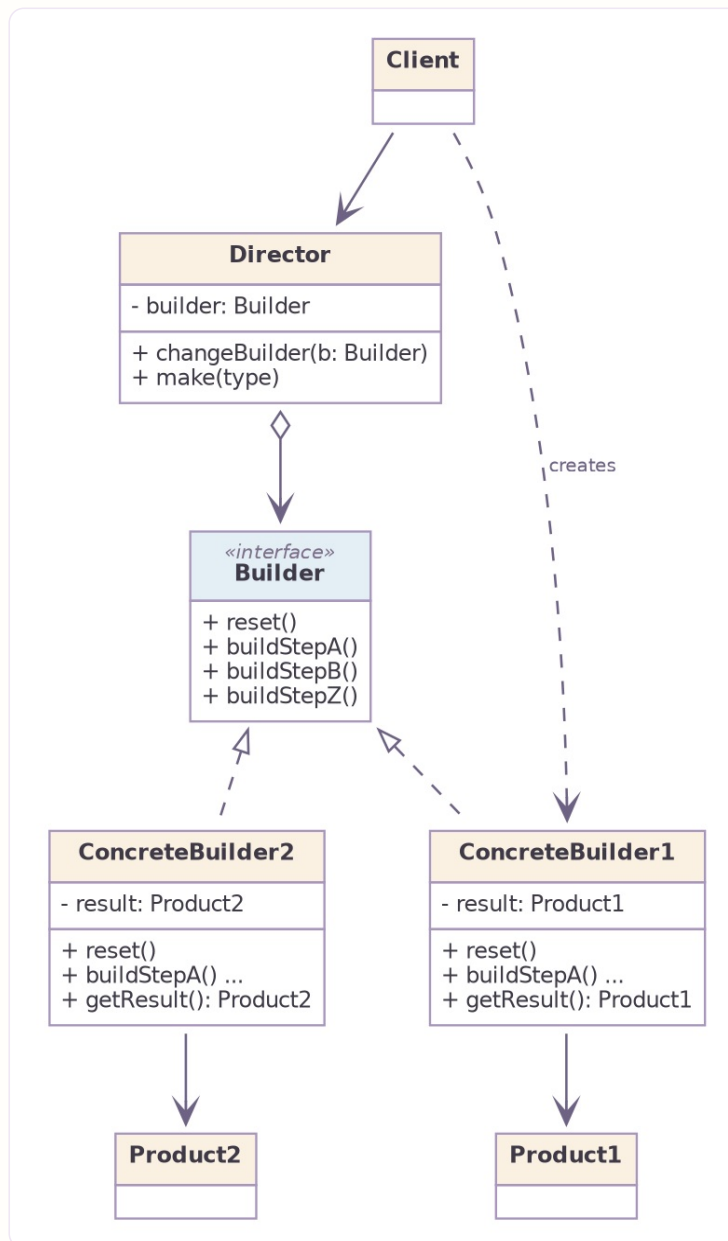
5.3 Builder

Intent: Construct **complex objects step by step**; the same construction process can create different representations.

Problem (RG): A `House` with walls, doors, windows, roof, optional garage, pool, garden... A constructor with 10 parameters (most `null`) — the "telescoping constructor".

✎ **Signature:** a **Builder** with many small `buildStep()` methods and a separate `getResult()`; an optional

Director that knows the step order. The product is returned **only at the end**.



Builder structure

Participants: Builder interface (steps), ConcreteBuilders (implement steps, hold the product), Product(s) (need not share an interface), Director (optional: fixed recipes), Client.

✓ **Use when:** many optional parts/parameters; you want different representations (stone house vs wooden house, or a car and its manual) from the same steps; building Composite trees step by step.

✗ **Don't use when:** the object is simple with 2-3 required fields — a constructor is clearer; it adds several new classes.

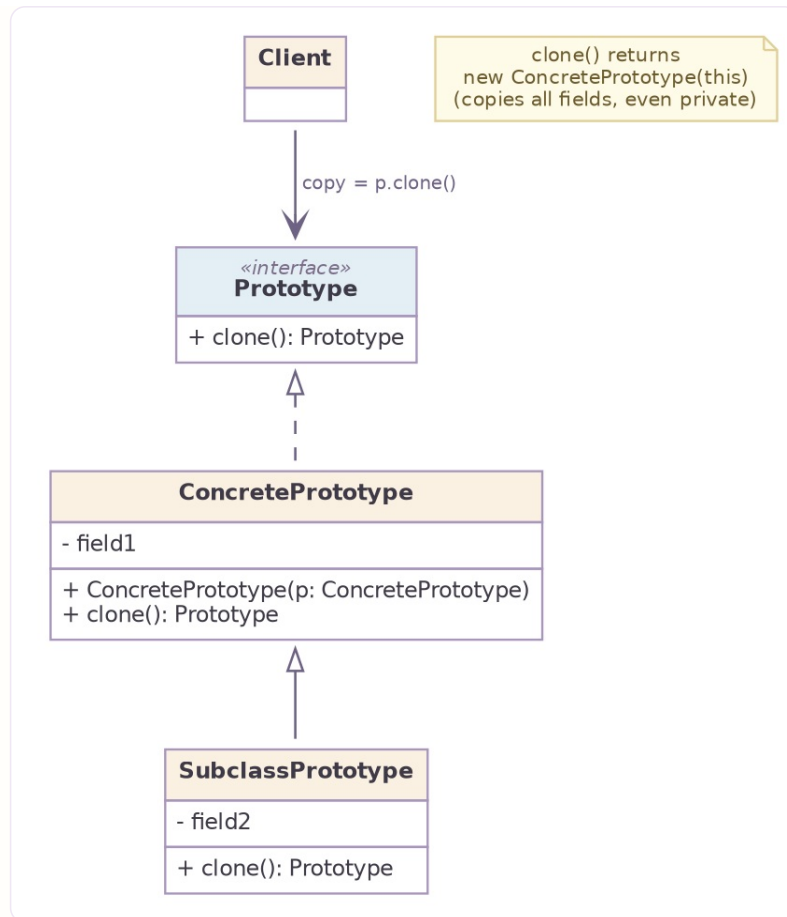
Similar: Abstract Factory (returns immediately, families), Composite (Builder often builds composite trees), Bridge (Director = abstraction, builders = implementations).

5.4 Prototype

Intent: Copy existing objects **without depending on their classes**.

Problem (RG): To copy an object from outside, you must know its concrete class and can't see its private fields.

✎ **Signature:** a `clone()` method declared in a common interface, implemented by each class to copy **itself** (usually via a copy-constructor). Optional **prototype registry** of ready-made objects.



Prototype structure

✓ **Use when:** code shouldn't depend on concrete classes of objects it copies (they come via an interface from 3rd-party code); objects are expensive to configure and you need many similar ones; you want to replace many subclasses that differ only in initial settings with pre-configured prototypes.

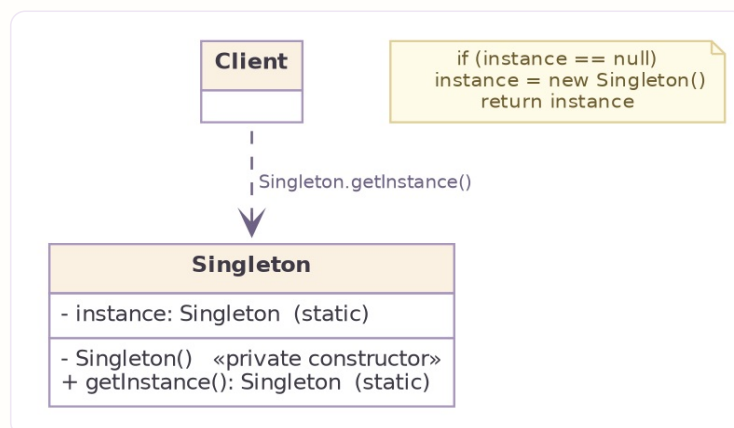
✗ **Don't use when:** objects have **circular references** or complex deep-copy needs (cloning is tricky); objects are cheap and simple to create.

Similar: Factory Method (inheritance-based creation; Prototype needs no creator subclasses), Memento (Prototype can be a simpler alternative for saving state).

5.5 Singleton

Intent: Ensure a class has **only one instance**, with a **global access point** to it.

🔗 **Signature:** private constructor + static field holding the only instance + static `getInstance()` .



Singleton structure

✓ **Use when:** exactly one shared object must exist (DB connection pool, configuration, logger) and you want stricter control than a global variable.

✗ **Don't use when:** you only want "easy access" — it becomes a hidden global, **hides dependencies**, is **hard to unit test** (can't swap with a fake), and needs care with **multithreading**. RG notes it breaks **Single Responsibility** (it solves two problems at once). Prefer passing a single instance via dependency injection.

Similar: Facade (often a Singleton), Flyweight (many shared immutable instances vs one mutable instance).

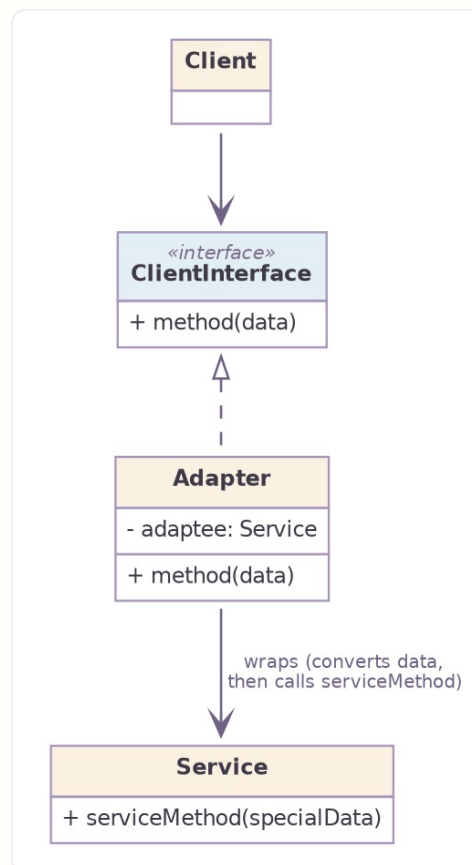
Structural Patterns

5.6 Adapter

Intent: Let objects with **incompatible interfaces** collaborate.

Problem (RG): Your stock app uses XML; a great analytics library only accepts JSON.

✎ **Signature:** the **Adapter implements the interface the client expects and holds (wraps) an object of a different interface** (the Service), translating calls and data.



Adapter (object adapter) structure

✓ **Use when:** you want to use an existing/3rd-party/legacy class whose interface doesn't match; reuse several subclasses that lack a common feature.

✗ **Don't use when:** you can simply change the service's code; interfaces already match (then it's just an extra layer).

Similar: Bridge (designed **up-front** to vary two sides; Adapter is **retro-fitted** to existing code), Decorator (same interface, adds behaviour), Proxy (same interface, controls access), Facade (new simple interface for a **whole subsystem**, Adapter wraps **one** object).

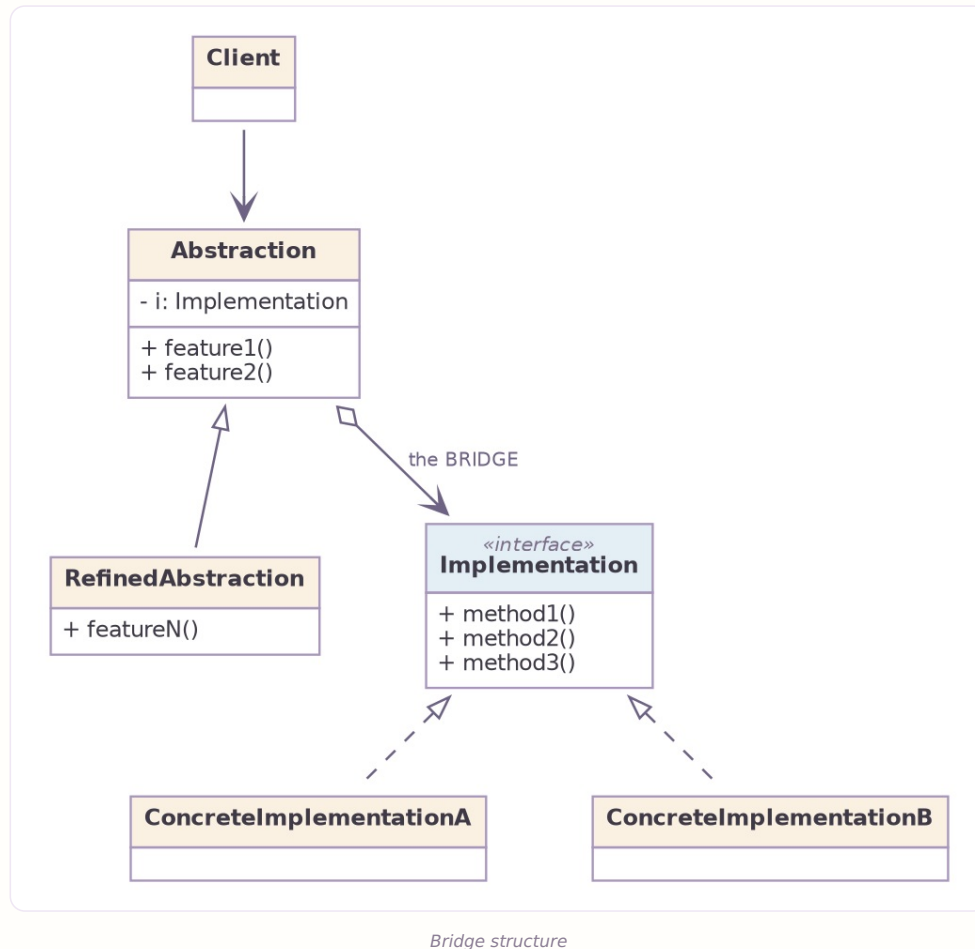
5.7 Bridge

Intent: Split a large class or set of closely related classes into **two separate hierarchies** — **abstraction and**

implementation — that can vary independently.

Problem (RG): `Shape` × `Color` → `RedCircle`, `BlueCircle`, `RedSquare`, `BlueSquare...` — subclasses grow as a product ($m \times n$).

🔗 **Signature:** two independent hierarchies connected by one has-a reference (the "bridge") from the abstraction to the implementation *interface*. The abstraction delegates real work to the implementation.



Bridge structure

✓ **Use when:** a class varies in **two (or more) independent dimensions** (shape/color, remote/device, UI/OS-API, message type/sender); you want to switch implementations at runtime.

✗ **Don't use when:** there's only one dimension of variation; the class is highly cohesive — splitting makes it harder to follow.

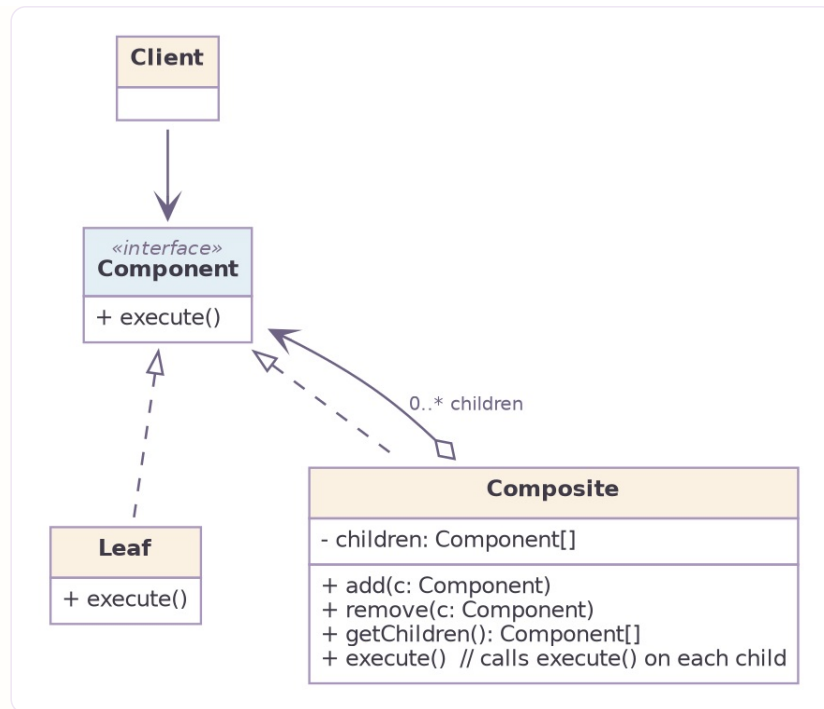
Similar: Strategy/State (also composition + interface, but they solve a *behaviour* problem; Bridge solves a *structure explosion* problem), Adapter (after the fact), Abstract Factory (can create matching abstraction-implementation pairs).

5.8 Composite

Intent: Compose objects into **tree structures** and treat individual objects and groups **uniformly**.

Problem (RG): Orders contain Products and Boxes; Boxes contain Products and smaller Boxes. How to compute the total price?

🔗 **Signature:** **Composite implements the Component interface AND holds a collection of Components** (aggregation arrow pointing back to its own interface, 0..*). Leaf and Composite share one interface; operations recurse down the tree.



Composite structure

✓ **Use when:** your model is a **tree / part-whole hierarchy** (files/folders, menus, org charts, packages); clients should treat simple and complex elements the same.

✗ **Don't use when:** the structure isn't a tree; leaves and groups have very different behaviour (a common interface becomes over-general — leaves get meaningless `add()` methods).

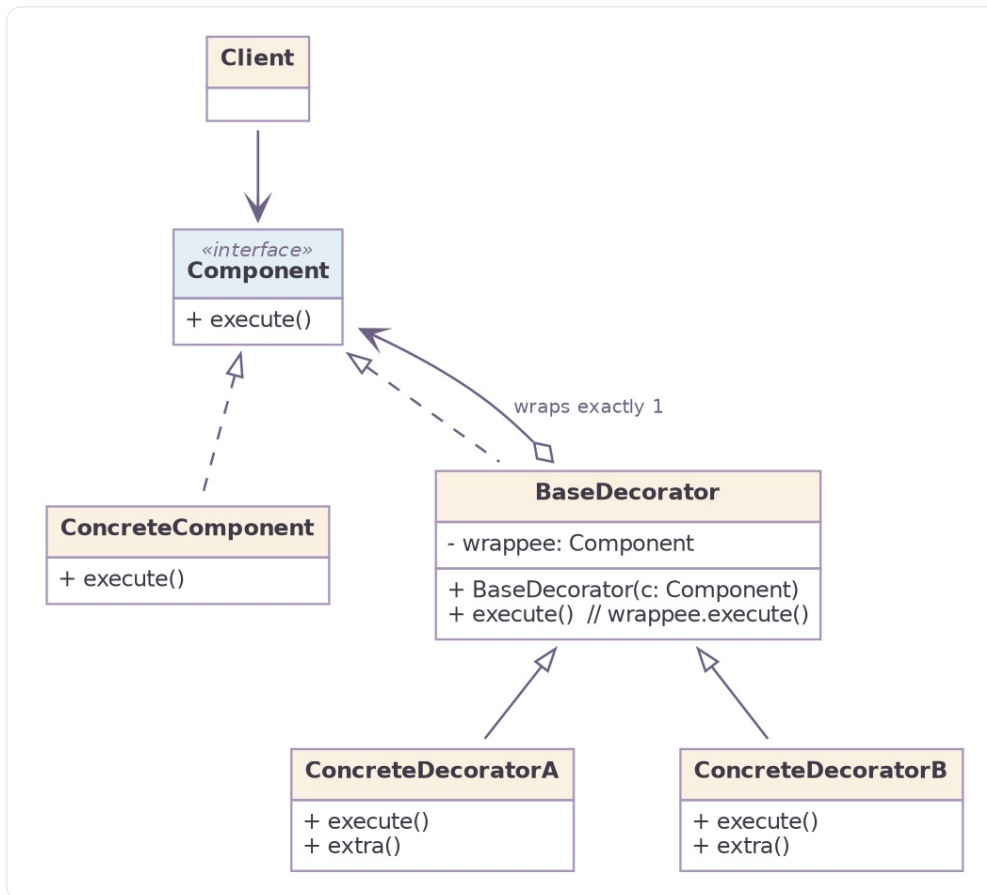
Similar: Decorator (also recursive, but wraps **exactly one** child and adds behaviour), Chain of Responsibility (often used on composite trees to pass requests up), Iterator/Visitor (traverse or operate on composite trees), Builder (constructs them), Flyweight (shared leaves save RAM).

5.9 Decorator

Intent: Attach **new behaviours to objects dynamically** by placing them inside wrapper objects that have the same interface.

Problem (RG): A `Notifier` sends email. Users want SMS, Facebook, Slack — and **combinations**. Subclasses for every combination explode.

✎ **Signature:** the **Decorator both implements the Component interface AND has a field of that same interface** (wraps exactly one). Concrete decorators call the wrapped object and add something before/after. Wrappers can be **stacked**.



Decorator structure

✓ **Use when:** you need to add responsibilities at runtime without breaking client code; extending by subclassing is impossible (final class) or causes explosion of combinations.

✗ **Don't use when:** the order of wrappers matters and is hard to control; you need to remove a specific wrapper from the middle of the stack; a lot of tiny similar objects hurts readability/debugging.

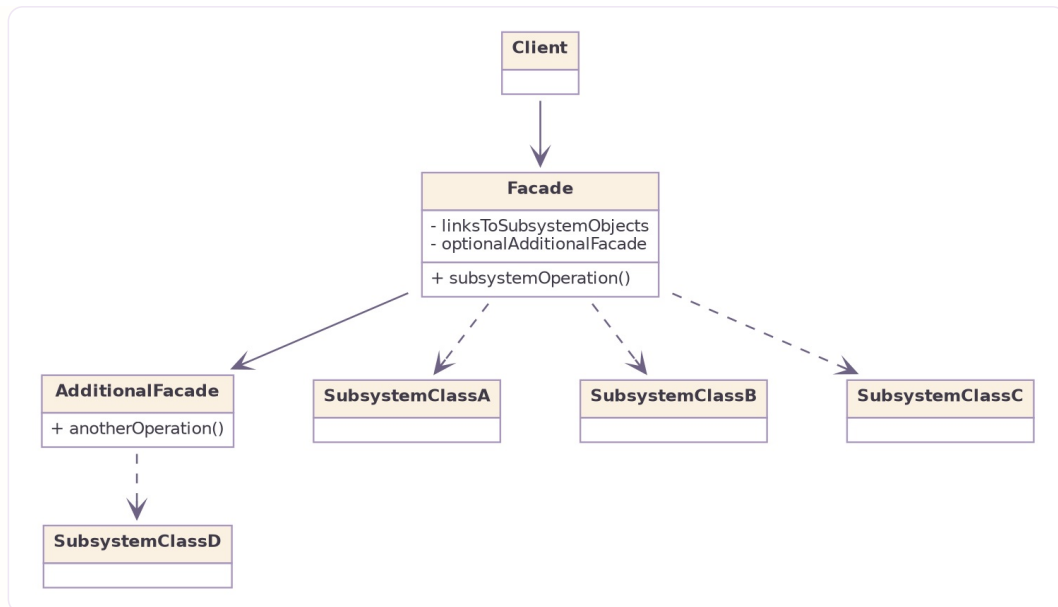
Similar: Adapter (changes interface), Proxy (same interface, but controls access and usually manages the service's life cycle itself; decorators are composed by the client), Composite (many children), Strategy (changes the *inside/guts*; Decorator changes the *skin*), Chain of Responsibility (handlers can stop the flow; decorators can't).

5.10 Facade

Intent: Provide a **simplified interface** to a library, framework, or any complex set of classes.

Problem (RG): To convert a video you'd have to initialise codecs, bitrate readers, audio mixers in the right order.

✎ **Signature: one class with a few high-level methods that call many subsystem classes;** the subsystem doesn't know the facade exists; clients mostly talk only to the facade.



Facade structure

✓ **Use when:** you need a limited, simple entry point to a complex subsystem; you want to layer a system (a facade per layer).

✗ **Don't use when:** clients genuinely need the full power of the subsystem; beware the facade becoming a **God object** coupled to everything.

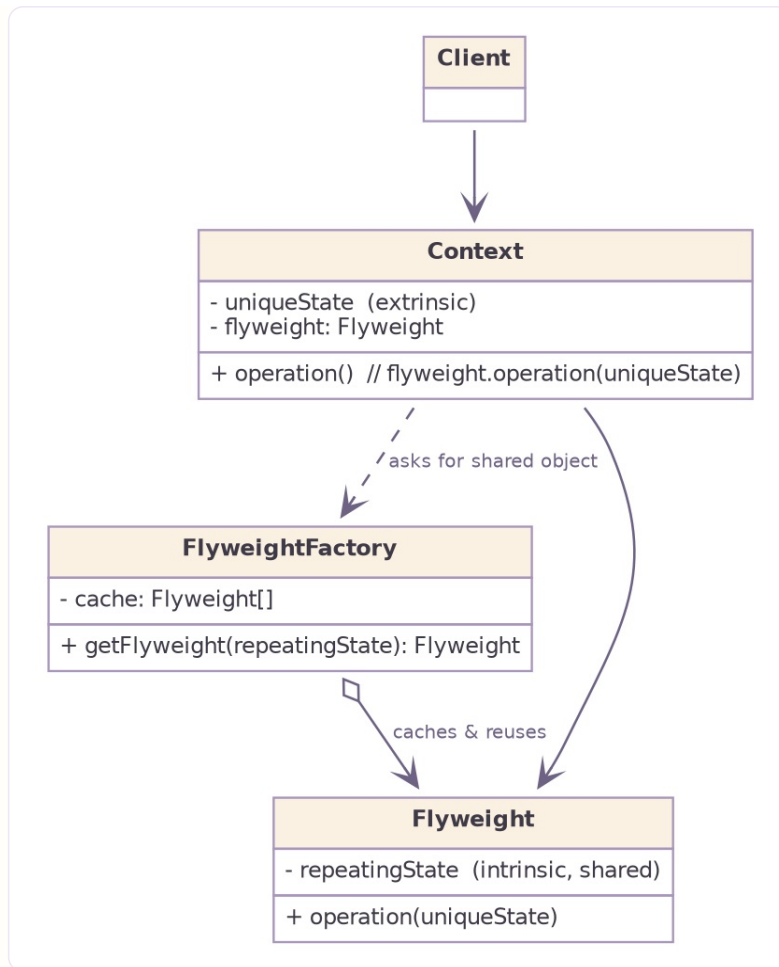
Similar: Adapter (makes an existing interface usable; Facade defines a new one for many objects), Mediator (Mediator *coordinates* two-way communication between peers who know it; Facade is one-way — subsystem classes don't know the facade), Abstract Factory (can hide subsystem object creation), Singleton (one facade object is usually enough).

5.11 Flyweight

Intent: Fit more objects into RAM by **sharing common parts of state** between many objects.

Problem (RG): A game with millions of particles (bullets, shrapnel) crashes; each particle stores the same sprite and color.

✎ **Signature:** a **Flyweight Factory with a cache (pool)** that returns existing shared objects; state is split into **intrinsic (shared, immutable, inside the flyweight)** and **extrinsic (unique, passed in as a method parameter / kept in a context object)**.



Flyweight structure

✓ **Use when:** the app must create a **huge number of similar objects** and RAM is the bottleneck; objects contain duplicate state that can be extracted and shared.

✗ **Don't use when:** there aren't many objects (complexity for nothing); you trade RAM for CPU (extrinsic data must be recalculated/passed each call); shared state must be mutable.

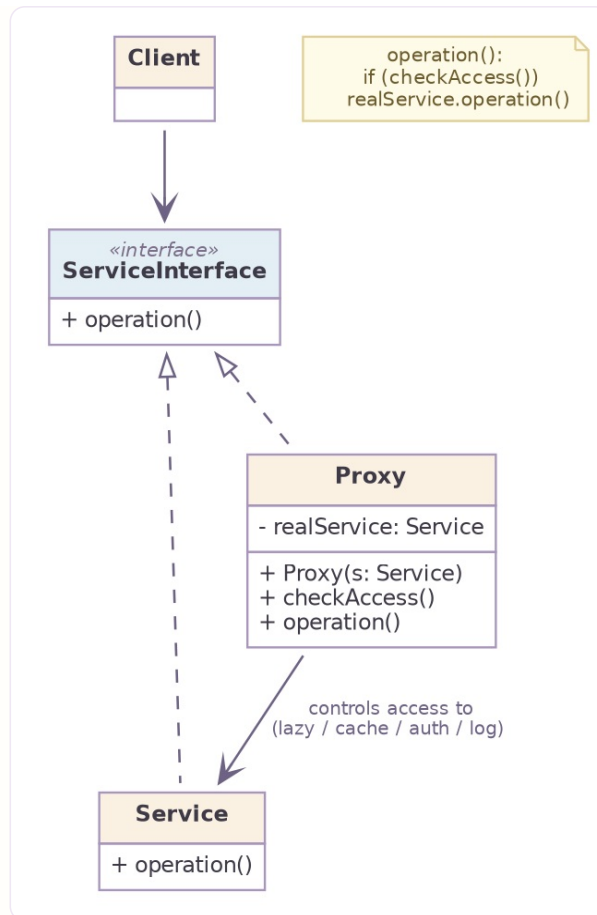
Similar: Singleton (one instance vs many shared instances; singleton can be mutable, flyweights are immutable), Composite (shared leaf nodes as flyweights), **Factory** (comes built into the pattern).

5.12 Proxy

Intent: Provide a **substitute or placeholder** for another object to **control access** to it.

Problem (RG): A huge object (DB, video library) consumes resources but is only needed sometimes — you want lazy loading, caching, access checks, or logging without changing it.

✎ **Signature:** the **Proxy implements the SAME interface as the real Service and holds a reference to it**, and usually **manages the service's life cycle itself** (creates it lazily). Client can't tell the difference.



Proxy structure

Kinds: Virtual (lazy init), Protection (access control), Remote (local stand-in for a remote object), Logging, Caching, Smart reference.

✓ **Use when:** lazy initialization of a heavy object; access control; caching results; logging requests; remote services.

✗ **Don't use when:** it only adds delay and complexity with no control needed; response time is critical and the extra layer hurts.

Similar: Decorator (same structure; Decorator adds behaviour and is composed by the client, Proxy controls access and manages the real object), Adapter (different interface), Facade (both buffer a complex entity, but Proxy keeps the same interface).

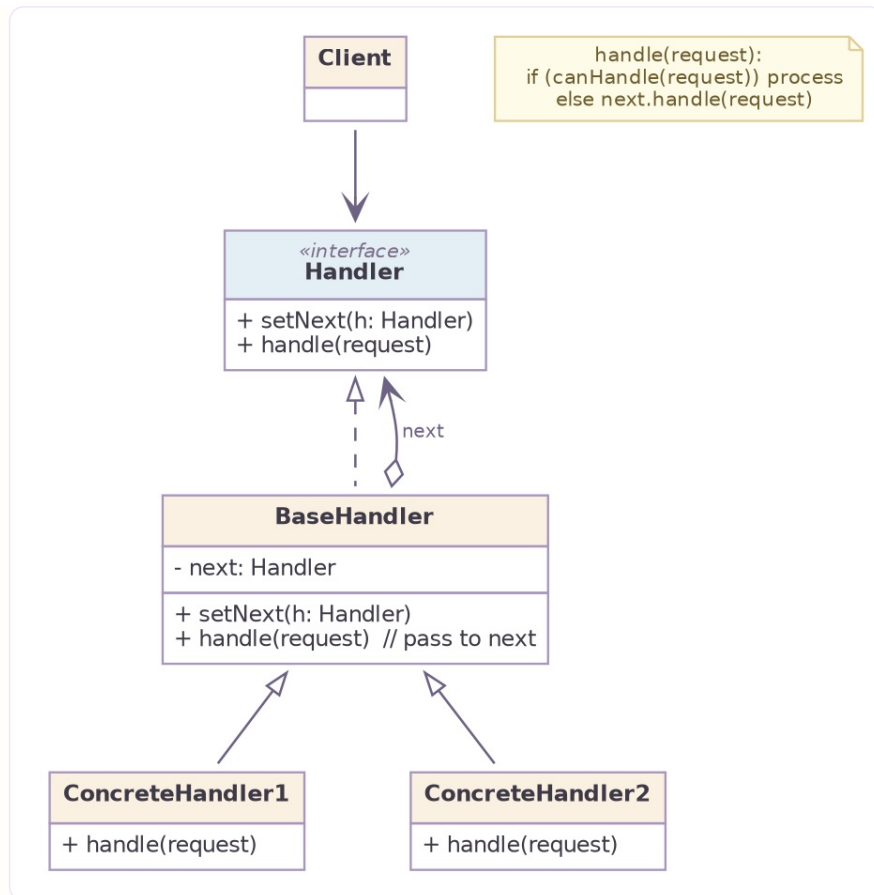
Behavioral Patterns

5.13 Chain of Responsibility

Intent: Pass requests along a **chain of handlers**. Each handler decides either to process the request or pass it to the next.

Problem (RG): An ordering system needs checks: authentication → validation → brute-force filter → cache. Putting them all in one method becomes a mess.

✎ **Signature:** a **Handler** with a **next** reference to another **Handler (self-association)** and **handle(request)**; handlers are linked into a chain **at runtime by the client**.



Chain of Responsibility structure

✓ **Use when:** different requests need different processing, but the types and order aren't known in advance; several handlers must run in a specific order; the set/order of handlers should change at runtime (approval levels, middleware, event bubbling in GUIs).

✗ **Don't use when:** every request must be handled by a known single receiver; **a request may end up unhandled** if nobody in the chain accepts it — you need a default/end handler.

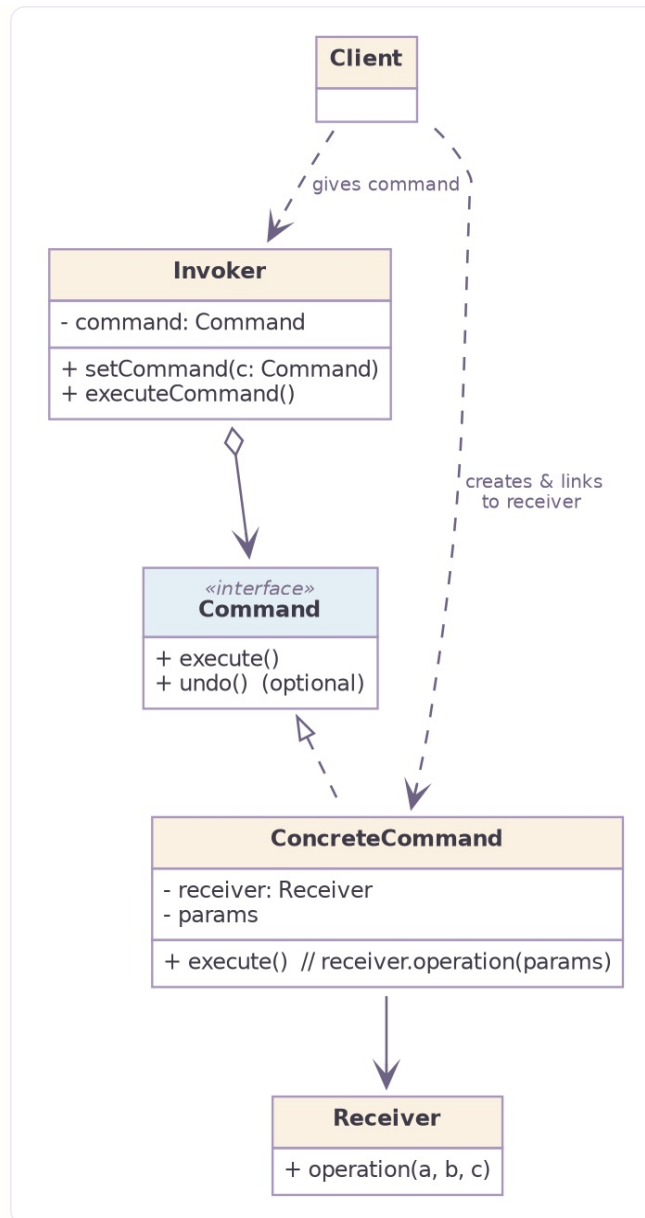
Similar: Decorator (nearly the same structure, but decorators **can't stop** the flow and all add behaviour), Command/Mediator/Observer (other ways to connect senders and receivers — CoR passes a request **sequentially** until someone handles it), Composite (a leaf passes a request up through its parents).

5.14 Command

Intent: Turn a request into a **stand-alone object** that contains all the information about the request.

Problem (RG): A text editor has buttons, menu items, and shortcuts that all "copy". GUI classes calling business logic directly causes duplication and coupling.

✎ **Signature:** a **Command interface** with `execute()` ; **Invoker** holds a **Command**; **ConcreteCommand** holds a **reference to the Receiver** and the parameters. Invoker never references the Receiver.



Command structure

✓ **Use when:** you want to parameterise objects with operations (buttons, menu items); queue, schedule, or send operations remotely; implement **undo/redo**, macros, or transaction logs.

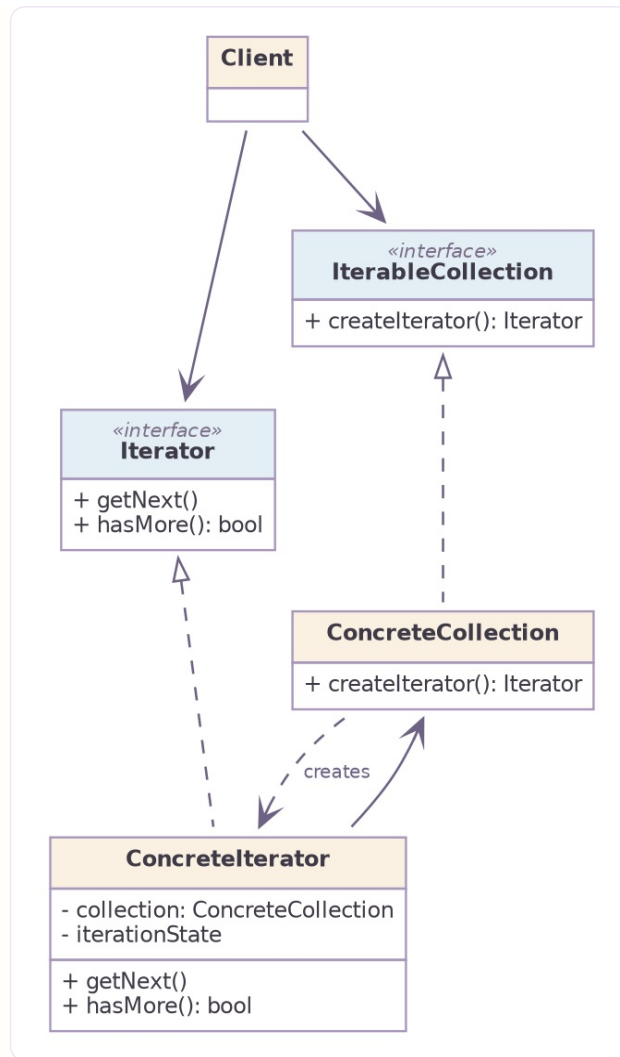
✗ **Don't use when:** a direct call is enough — it adds a whole layer of classes between senders and receivers.

Similar: Strategy (both parametrise with an object; Command turns *any operation* into an object that can be queued/undone; Strategy describes *different ways of doing the same thing*), Memento (for undo state), Chain of Responsibility (handlers can be commands), Prototype (copy commands into history).

5.15 Iterator

Intent: Traverse elements of a collection **without exposing its underlying representation** (list, stack, tree...).

🔗 **Signature:** a **separate Iterator object** with `getNext()` / `hasMore()` that keeps its own traversal state, and a collection with a **factory method** `createIterator()`.



Iterator structure

✓ **Use when:** the collection has a complex structure (tree, graph) you want to hide; you need several traversal ways (DFS/BFS, friends/coworkers); you want one traversal code for different collections.

✗ **Don't use when:** the collection is a simple list — a normal loop is enough; it can be less efficient than direct access.

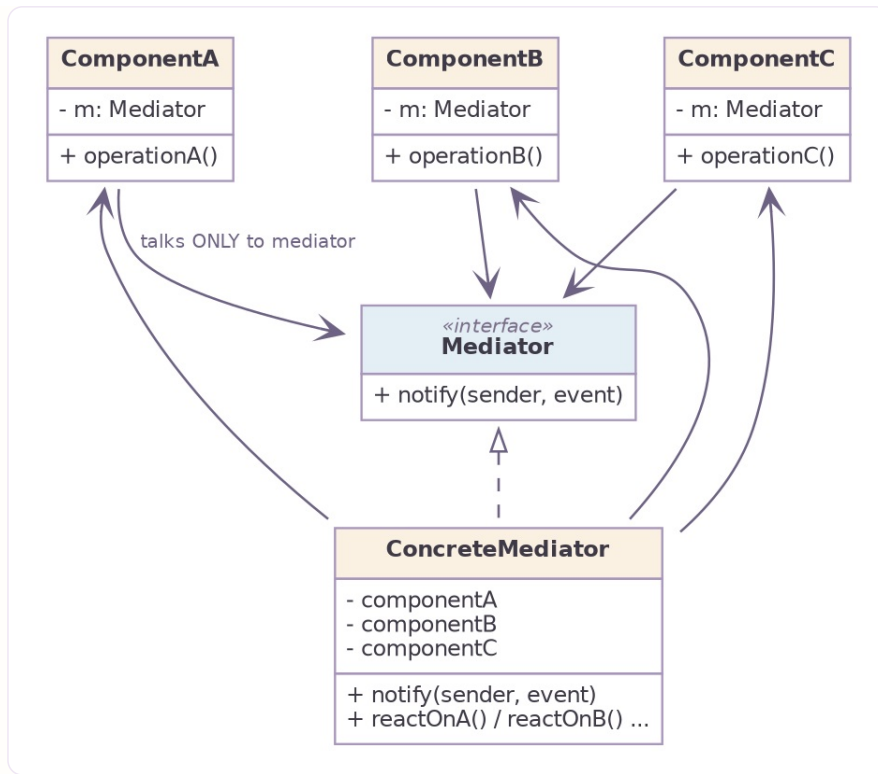
Similar: Composite (iterate its tree), Factory Method (`createIterator` is one), Memento (save iteration state), Visitor (Iterator *traverses*, Visitor *performs an operation* on each element).

5.16 Mediator

Intent: Reduce chaotic dependencies between objects: they **stop talking directly and communicate only through a mediator**.

Problem (RG): A profile dialog: a checkbox reveals a text field, the submit button validates all fields... every element knows every other.

✎ **Signature:** components hold a reference only to the **Mediator interface** (not to each other); the **ConcreteMediator holds references to all components** and contains the coordination logic in `notify(sender, event)` — a **star** shape instead of a **web**.



Mediator structure

✓ **Use when:** classes are hard to change because they're tightly coupled to many others; you can't reuse a component because it depends on too many others; you're creating many subclasses just to reuse basic behaviour in different contexts.

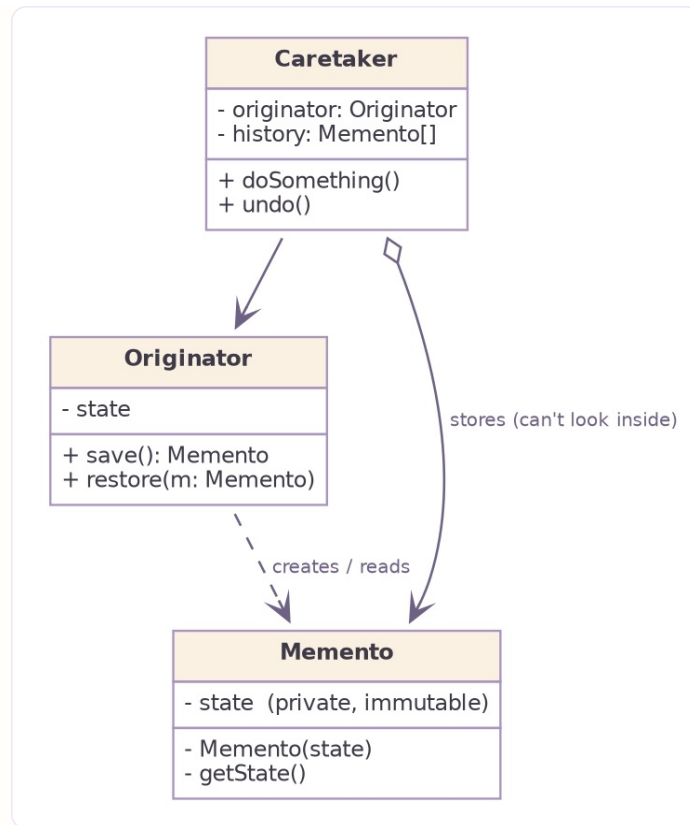
✗ **Don't use when:** only two or three objects interact simply; beware the mediator growing into a **God object**.

Similar: Facade (one-way, subsystem unaware), Observer (dynamic one-way subscriptions; a mediator is often *implemented* with Observer), Chain of Responsibility/Command (other sender-receiver connections).

5.17 Memento

Intent: Save and restore an object's previous state **without revealing its implementation details**.

✎ **Signature:** the **Originator creates and restores a Memento** (an immutable snapshot of its state); the **Caretaker stores a history of Mementos but cannot read inside them**.



Memento structure

✓ **Use when:** you need snapshots for undo, rollback, or transactions; direct access to fields/getters would break encapsulation.

✗ **Don't use when:** snapshots are large and frequent — it eats RAM; the language can't guarantee the memento stays private.

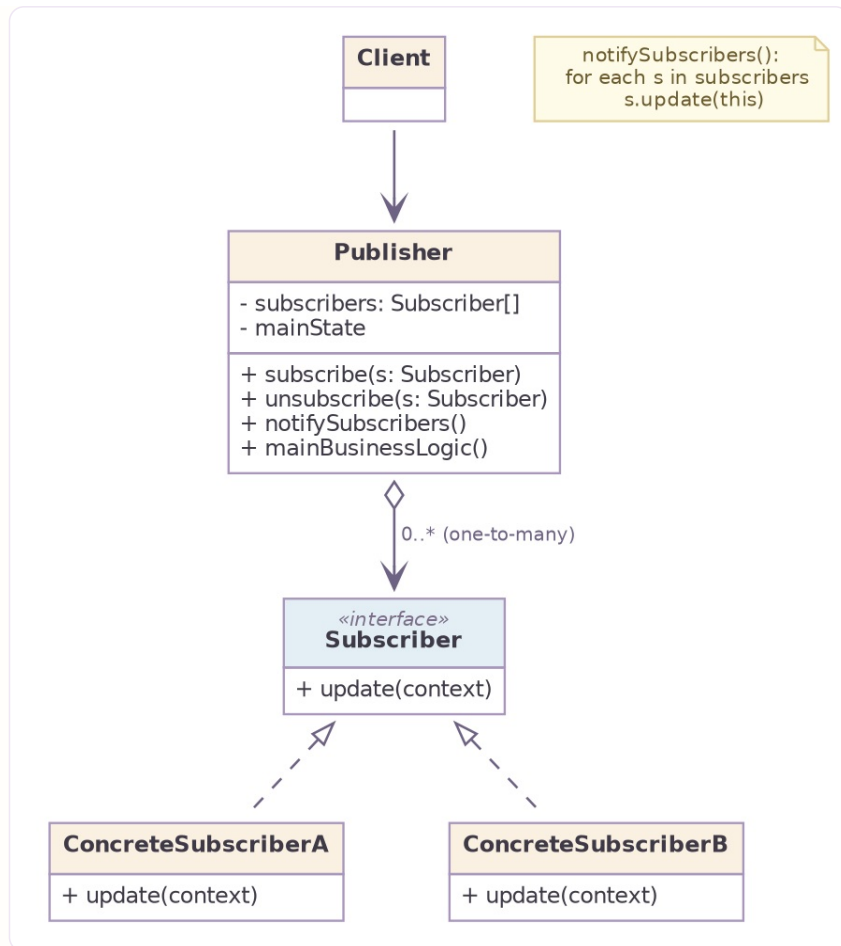
Similar: Command (Command performs operations, Memento saves state before them — used together for undo), Iterator (save traversal state), Prototype (simpler alternative when the object is simple to clone).

5.18 Observer

Intent: Define a **subscription mechanism** so many objects are notified about events happening to the object they observe.

Problem (RG): Customers keep visiting the store to check if a new iPhone arrived; or the store spams everyone. Better: let interested customers subscribe.

🔗 **Signature:** the **Publisher keeps a list of Subscriber interfaces** and has `subscribe()/unsubscribe()/notify()` ; `notify` loops over the list calling `update()` — **one-to-many, dynamic, one-way**.



Observer structure

✓ **Use when:** changes to one object require updating others, and the set of dependents isn't known in advance or changes at runtime (event listeners, UI updates, feeds).

✗ **Don't use when:** the notification order must be guaranteed (subscribers are notified in random order); there are many cascaded updates that become hard to trace; memory leaks from forgotten subscriptions are a risk.

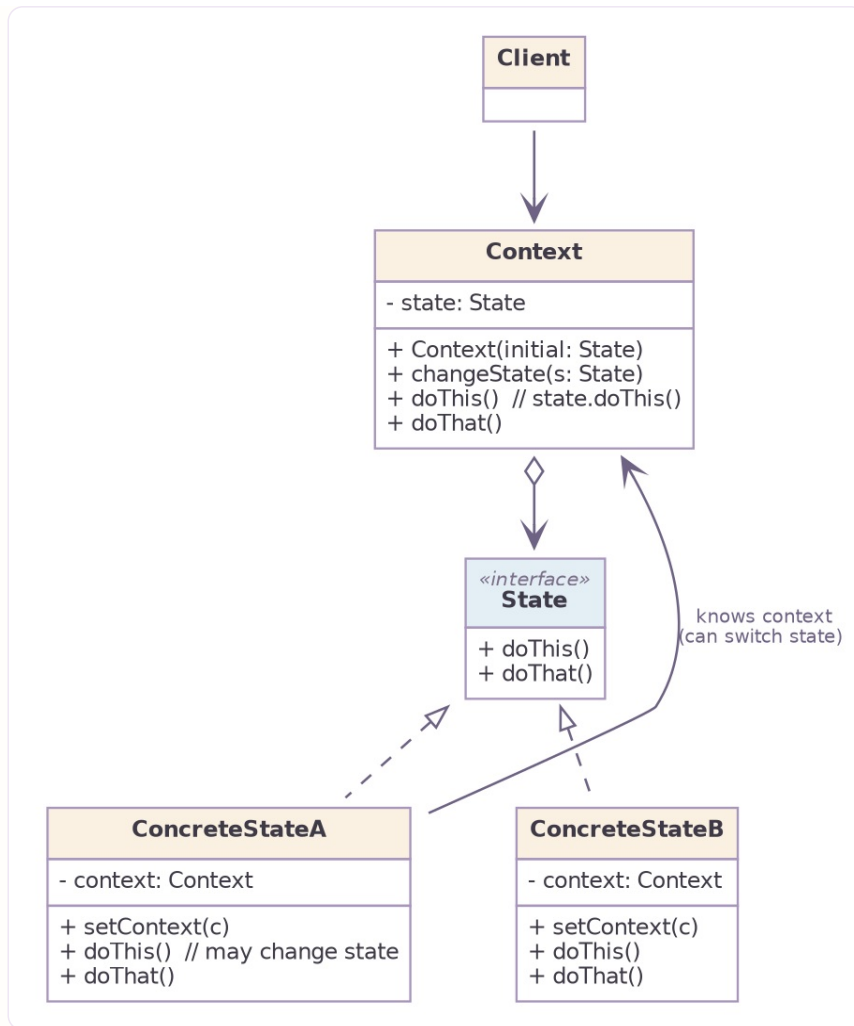
Similar: Mediator (removes mutual dependencies via a central object; Observer sets up one-way connections), Chain of Responsibility, Command.

5.19 State

Intent: Let an object **change its behaviour when its internal state changes** — it looks as if the object changed its class.

Problem (RG): A `Document` with Draft / Moderation / Published states: `publish()` does different things in each. Big `switch(state)` in every method.

✎ **Signature:** Context holds a State interface field and delegates to it; ConcreteStates know the Context and can switch it to another state (`context.changeState(new X())`). States are aware of each other.



State structure

✓ **Use when:** behaviour depends on the current state, there are many states, and state-specific code changes often; a class is polluted with large state conditionals (a finite-state machine).

✗ **Don't use when:** there are only a few states that rarely change — the pattern is overkill.

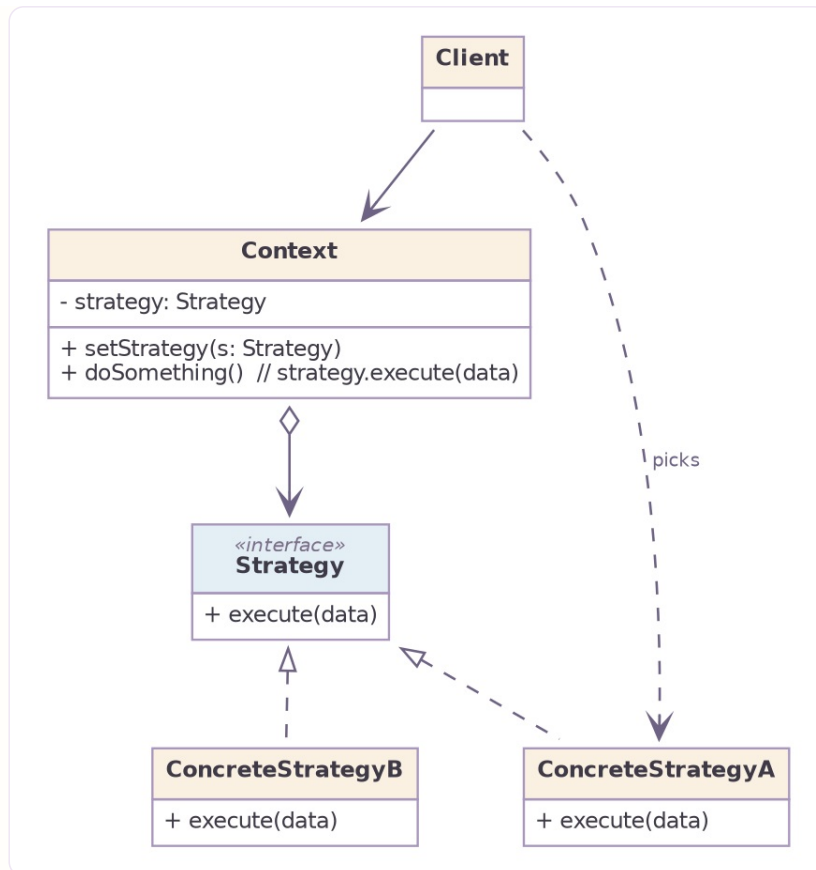
Similar: Strategy (same structure! but strategies are **independent and unaware of each other**, chosen by the client; states **know each other and trigger transitions** themselves), Bridge (also composition-based).

5.20 Strategy

Intent: Define a **family of algorithms**, put each in its own class, and make them **interchangeable**.

Problem (RG): A navigation app adds routing for cars, then walking, public transport, cycling... the main class becomes huge.

✎ **Signature:** Context holds a Strategy interface field with a setter; the client picks the concrete strategy; strategies are **independent and don't know about each other**.



Strategy structure

✓ **Use when:** you need different variants of an algorithm and want to switch at runtime; many similar classes differ only in behaviour; you want to isolate algorithm details; a class has a big conditional choosing between variants.

✗ **Don't use when:** there are only a couple of algorithms that rarely change; clients shouldn't need to know the differences between strategies; a simple lambda/function would do (modern languages).

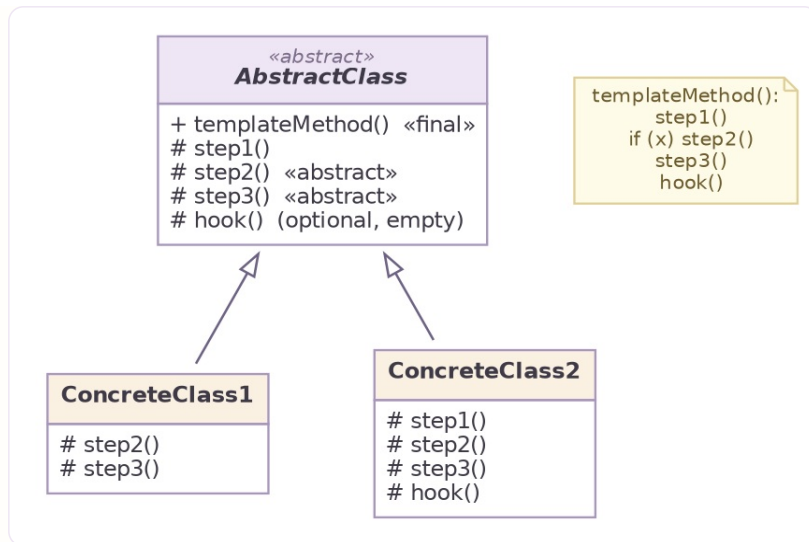
Similar: State (states switch themselves), Template Method (inheritance, compile-time, varies *parts* of an algorithm; Strategy is composition, runtime, swaps the *whole* algorithm), Decorator (skin vs guts), Command, Bridge.

5.21 Template Method

Intent: Define the **skeleton of an algorithm in the superclass**; subclasses override **specific steps** without changing the structure.

Problem (RG): A data-mining app for DOC, CSV, PDF files — three classes with lots of identical code (open, extract, parse, analyze, report).

🔗 **Signature:** an **abstract class with a (final) `templateMethod()`** that calls a sequence of step methods; some steps abstract, some default, some optional **hooks**; subclasses override only steps — **inheritance, not composition**.



Template Method structure

✓ **Use when:** several classes share almost the same algorithm with small differences (removes duplication); you want clients to extend only particular steps.

✗ **Don't use when:** subclasses need to change the step *order* (skeleton is fixed); many steps make it hard to maintain; overriding a default step may violate LSP.

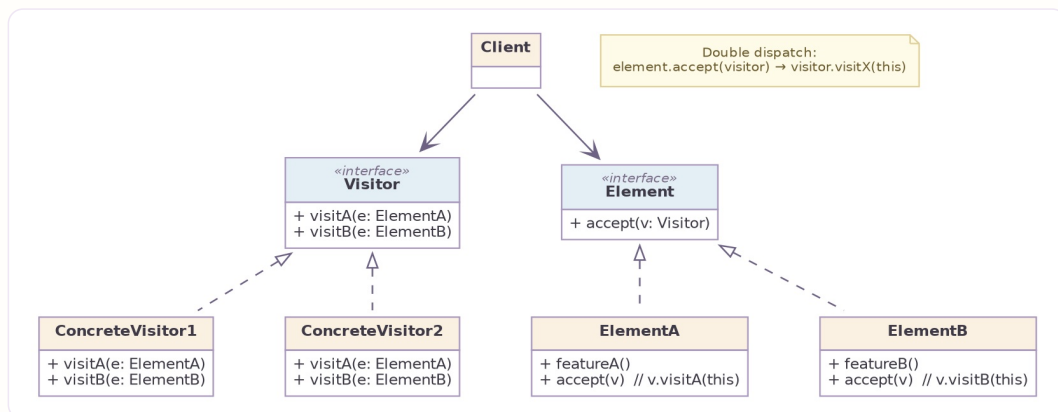
Similar: Factory Method (a specialised template step), Strategy (composition/runtime vs inheritance/compile-time).

5.22 Visitor

Intent: Separate algorithms from the objects they operate on — add **new operations to existing classes without changing them**.

Problem (RG): A geographic graph (City, Industry, Sightseeing nodes) needs an XML export — but you're not allowed to change the node classes, and more exports will come.

✎ **Signature:** a **Visitor interface with one visitX() method per element class**, and every element has **accept(visitor) that calls visitor.visitX(this)** — **double dispatch**.



Visitor structure

✓ **Use when:** you need to perform many unrelated operations on a complex object structure (e.g. a Composite tree) and don't want to pollute its classes; the element classes are stable but operations change often.

✗ **Don't use when:** element classes change often — **every new element class forces changes in all visitors**; visitors need private fields of elements.

Similar: Command (Visitor is a powerful Command that runs over many element types), Composite (visit a tree), Iterator (traverse while visiting).

PART 6 — Similar Patterns: Differences & When to Choose Which

★ Teacher focus

Many patterns have **almost the same UML** — the difference is **intent**. For every group below, learn the one-line **tie-breaker**.

6.1 Which patterns are similar to Proxy? Why? What makes them separate? asked 2x

A: Decorator, Adapter, and Facade — all are "wrappers" that stand between a client and another object and forward calls.

	Proxy	Decorator	Adapter	Facade
Interface vs wrapped object	Same	Same	Different (converts)	New simplified one
Wraps	one service	one component (stackable)	one service	a whole subsystem
Purpose	Control access (lazy load, cache, auth, log, remote)	Add behaviour	Make incompatible fit	Simplify
Who creates the wrapped object	Proxy usually manages its life cycle	Client composes it	Client passes it in	Facade uses subsystem objects
Stacking	not the idea	designed for stacking	no	no

Tie-breaker: *Proxy guards, Decorator decorates, Adapter translates, Facade simplifies.*

6.2 Which patterns are similar to Composite? Why? Differences? asked 2x

A: Decorator (and **Chain of Responsibility**). All three use **recursive composition** — an object holds object(s) of its own abstract type and forwards work to them.

	Composite	Decorator	Chain of Responsibility
Holds	many children (0..*)	exactly one wrappee	one next handler
Goal	represent a part-whole tree , treat leaf & group the same	add responsibilities to one object	pass a request until someone handles it
Result	sums/collects results from children	enhances one result	usually one handler processes

They cooperate: Decorator can extend a specific node of a Composite tree; CoR often passes requests **up** a Composite tree.

6.3 What are the similarities and differences between Composite and Decorator?

A: Similar: nearly identical structure — both implement the Component interface and hold Component reference(s); both rely on recursion; clients treat the whole thing through one interface.

Different: Composite *sums up* the results of **many** children to represent a hierarchy; Decorator wraps **one** object and *adds* extra behaviour. Composite is about **structure**; Decorator is about **responsibilities**.

6.4 Which patterns are similar to Decorator? Why? Differences?

A: Proxy (same interface + wraps an object — but controls access instead of adding features), **Adapter** (wraps, but changes the interface), **Composite** (recursive, but many children), **Chain of Responsibility** (same linked structure, but a handler can **stop** the request; a decorator **must** pass it on), **Strategy** (both change behaviour, but Strategy swaps the *inside*, Decorator wraps the *outside*).

Decorator's unique trait: stackable wrappers, chosen and combined **by the client at runtime**, all keeping the original

interface.

6.5 "Strategy changes the guts of an object, Decorator changes the skin" — explain. asked 3x

A:

- **Strategy = guts (inside):** the context object keeps a Strategy field and **delegates its core algorithm to it**. Swapping the strategy changes *how the object works internally*. The object **knows** about the strategy.
- **Decorator = skin (outside):** a wrapper with the same interface is placed **around** the object and adds behaviour before/after the call. The original object **doesn't know** it's being decorated; its inner code never changes.

Consequence: with Strategy, the component must be designed to accept strategies (you change the class). With Decorator, you can extend a class you can't modify (even a **final** one), and stack several extensions. Use Strategy when the **core algorithm** varies; Decorator when you need **optional add-ons** combined freely.

6.6 Strategy vs State vs Template Method

A: All three let behaviour vary.

	Strategy	State	Template Method
Mechanism	composition (has-a)	composition (has-a)	inheritance (is-a)
Chosen by	the client	the states themselves (transitions)	the subclass at compile time
Parts know each other?	no — independent	yes — a state switches to the next	n/a
Varies	the whole algorithm	behaviour per state	some steps of a fixed algorithm
Changes at runtime?	yes	yes (automatically)	no

Tie-breakers: "Who decides the switch?" Client → **Strategy**; the object's own status → **State**. "Is the order of steps fixed with a few steps varying?" → **Template Method**.

6.7 Observer vs Mediator vs Chain of Responsibility vs Command — connecting senders and receivers

A: Refactoring Guru groups these four as different ways to connect senders and receivers:

Pattern	How requests travel	Tie-breaker
Chain of Responsibility	sequentially along a chain until one handler processes it	"try handlers in order / escalate"
Command	one-way connection; request packed into an object	"queue, log, undo, parameterize an action"
Mediator	everything goes through a central object ; components don't know each other	"many objects tangled with each other"
Observer	receivers subscribe/unsubscribe dynamically ; one-to-many broadcast	"notify all interested parties when X changes"

Mediator vs Observer in depth: Mediator's goal is to **remove mutual dependencies** among a fixed set of components; Observer's goal is **dynamic one-way subscriptions**. A mediator is often *implemented* with Observer (components publish events, mediator subscribes). If components talk to each other both ways → Mediator. If one source broadcasts to changing listeners → Observer.

6.8 Factory Method vs Abstract Factory vs Builder vs Prototype

A:

	Factory Method	Abstract Factory	Builder	Prototype
Creates	one product	a family of related products	one complex product	a copy of an

				existing object
How	subclass overrides a creation method (inheritance)	factory object with many create methods (composition)	step-by-step calls, result at the end	<code>clone()</code>
Returns	immediately	immediately	after the last step	immediately
Easy to add	new product subclass	new family	new representation/builder	new pre-configured prototype
Hard to add	—	new product type	—	objects with circular refs

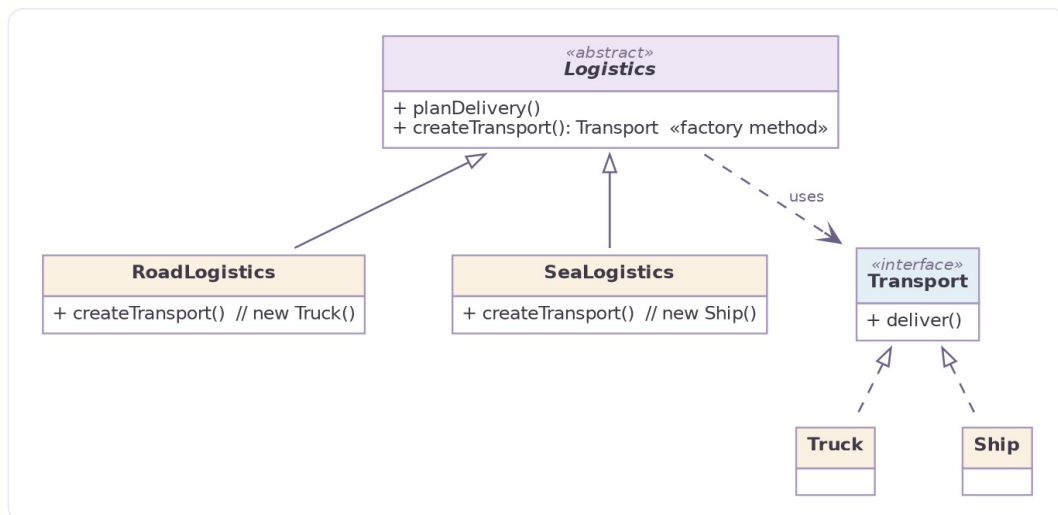
Tie-breakers: "one product, subclass decides" → Factory Method. "products must match each other" → Abstract Factory. "many optional parts / steps" → Builder. "copy a configured object" → Prototype. Designs often **start** with Factory Method and evolve into the others.

6.9 Compare Factory Method and Abstract Factory with examples. When to use each?

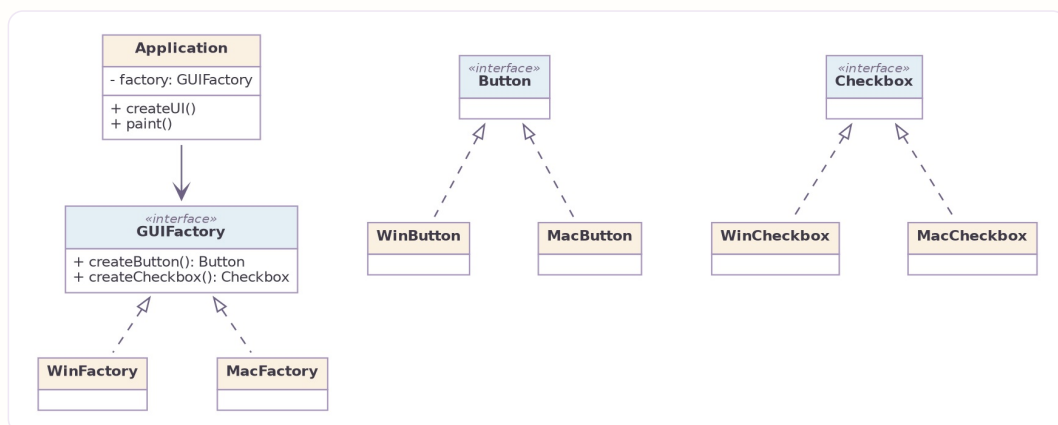
A: Factory Method — one creation *method* overridden in subclasses to decide *which* product class. Example:

`Logistics.createTransport()` → `RoadLogistics` makes `Truck`, `SeaLogistics` makes `Ship`.

Abstract Factory — one factory *object* with several creation methods, each concrete factory producing one consistent *family*. Example: `GUIFactory.createButton()/createCheckbox()` → `WinFactory` makes Windows widgets, `MacFactory` makes Mac widgets — never mixed.



Factory Method example — the creator subclass decides the product.



Abstract Factory example — one factory per family guarantees matching widgets.

Use Factory Method when there's a single product type and you want subclasses/frameworks to extend creation. **Use Abstract Factory** when you have several product types that must be used together consistently and you want to switch the whole family at once.

6.10 Briefly describe the differences between Builder and Abstract Factory.

A:

- **Builder** constructs **one complex object step by step** and returns it **at the end**; the client (or a Director) controls the steps. Good for many optional parts.
- **Abstract Factory** creates **families of related (usually simpler) objects** and returns each product **immediately** in a single call; the client doesn't control how it's built.
- Builder focuses on **how** an object is assembled; Abstract Factory focuses on **which family** of objects is produced.

6.11 Adapter vs Bridge

A: Both put an interface between two sides and use composition.

- **Adapter** is applied **after** the fact to make two **existing, incompatible** classes work together.
- **Bridge** is designed **up-front** to let an abstraction and its implementation **develop independently** (to avoid $m \times n$ subclasses).

Tie-breaker: "Can't change the other class and interfaces don't match" → Adapter. "Two dimensions of variation causing class explosion" → Bridge.

6.12 Facade vs Mediator vs Adapter

A:

- **Facade**: one-way simplification. Clients → Facade → subsystem. **Subsystem objects don't know the facade** and can still talk to each other directly.
- **Mediator**: two-way coordination. Components **know the mediator** and send events to it; the mediator tells other components what to do. Components **can't** talk directly.
- **Adapter**: wraps **one** object to make its existing interface usable; Facade defines a **new** interface over **many** objects.

6.13 Singleton vs Flyweight vs static class

A:

- **Singleton**: exactly **one** instance, can be **mutable**, global access point.
- **Flyweight**: **many** shared instances (one per unique intrinsic state), must be **immutable**.
- **Static class**: no instance at all — can't implement an interface, can't be passed around or replaced (worse for polymorphism and testing).

6.14 Command vs Strategy vs Memento

A: **Command** packs *any action* into an object (to queue, log, undo). **Strategy** gives *different ways of doing the same task* (swap algorithms). **Memento** captures *state*, not actions. For undo: Command decides *what* to reverse; Memento stores *the state to go back to* — often used together.

6.15 Iterator vs Visitor

A: **Iterator** walks through elements (*how to reach each one*) without exposing the collection. **Visitor** adds an **operation** to run on each element type (*what to do with each one*) without changing element classes. Often used together: iterate the composite tree, call `accept(visitor)` on each node.

6.16 Name two patterns used to eliminate if-else. Explain with an example each. asked 2x

A:

1. **Strategy** — replaces `if (method == "card") ... else if (method == "bkash") ...` for choosing an algorithm. Each branch becomes a strategy class; the context calls `strategy.pay(amount)`. The client picks the strategy once. (See scenario 8.1.)
2. **State** — replaces `if (state == DRAFT) ... else if (state == PUBLISHED) ...` inside every method. Each state becomes a class implementing all state-dependent methods; the context calls `state.publish()` and states switch the context to the next state. (See scenario 8.2.)

Also valid: **Chain of Responsibility** (a chain of `if` checks in order), **Factory Method / Replace Type Code with Subclasses** (type-based creation), **Null Object** (`if (x == null)`), **Command** (a `switch` on action names → map of commands).

6.17 When should you NOT use a design pattern at all?

A: When it adds complexity without solving a real, current problem. Signs of over-use: only one implementation will ever exist; the "flexibility" is speculative (YAGNI → Speculative Generality); a simple function/lambda or plain `if` is clearer; the team can't follow it. A pattern is justified by a **concrete change you expect** (new type, new family, new channel). The best exam answers say *why* the pattern fits **and** why a simpler/alternative solution doesn't.

PART 7 — Past-Paper Design Pattern Questions (answered with UML)

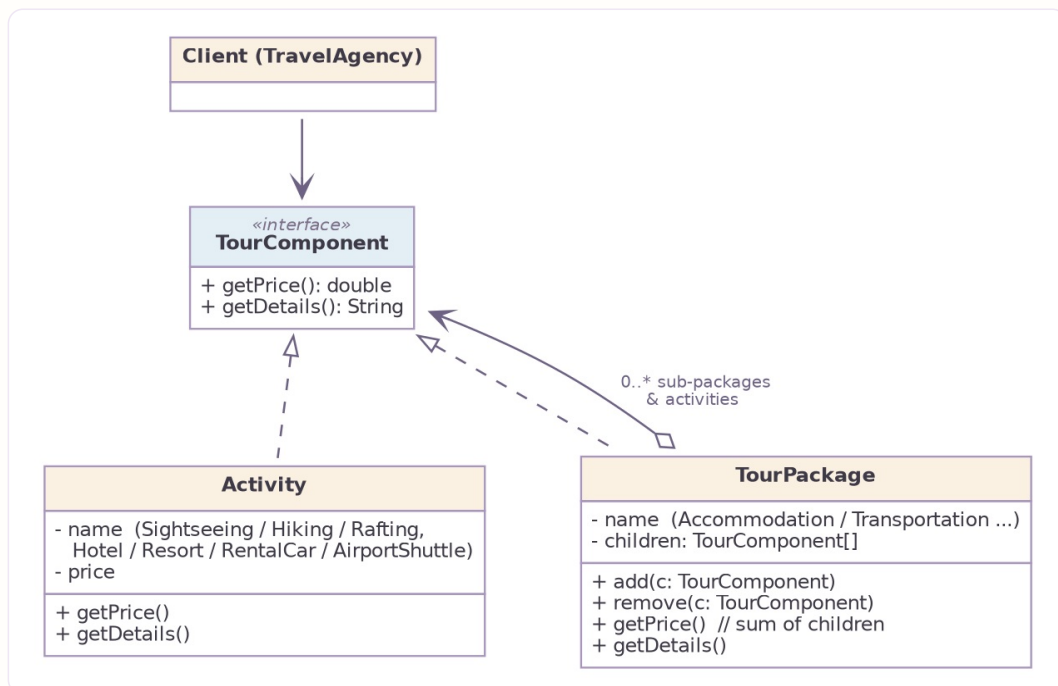
Exam answer template for any scenario question (use it every time): **1. Problem** in the scenario → **2. Pattern** name → **3. Why** (map scenario parts to pattern roles/signature) → **4. UML** class diagram → **5. Why not the look-alike** alternative.

7.1 Travel agency: tour packages with activities and nested sub-packages (accommodation → hotel/resort; transportation → rental car/airport shuttle). Suggest a pattern and draw UML.

A: Pattern: Composite.

Why: it's a **part-whole tree** — a package contains activities (leaves) and other packages (composites) to any depth, and the agency wants to treat one activity and a whole package the same way (e.g. `getPrice()` of a package = sum of its children).

Role mapping: Component = `TourComponent`; Leaf = `Activity` (Sightseeing, Hiking, Rafting, Hotel, Resort, Rental Car, Airport Shuttle); Composite = `TourPackage` (Full Tour, Accommodation, Transportation).

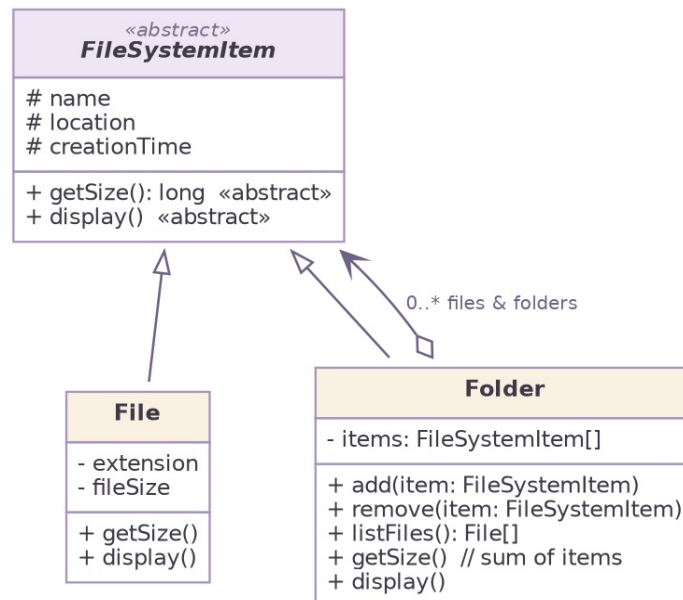


Composite design for tour packages.

Why not Decorator: a package holds **many** children, not one wrapped object, and nothing is being "added on" to a single activity.

7.2 File system: files and folders; a folder holds files and folders, can add/remove, list its files. Design it. asked 2x

A: Pattern: Composite. File = Leaf, Folder = Composite, `FileSystemItem` = Component holding the shared attributes (`name`, `location`, `creationTime`). Operations like `getSize()` recurse: a folder's size is the sum of its items.



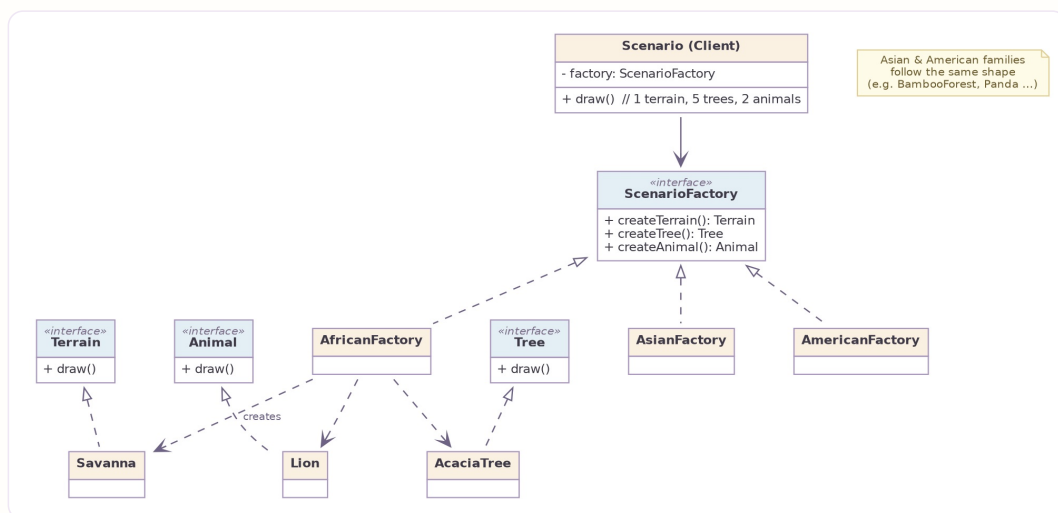
Composite design for the file system.

7.3 Game with African / Asian / American scenarios; each has a terrain, 5 trees of one kind, 2 animals of one kind. Design it. asked 2x

A: Pattern: Abstract Factory.

Why: each scenario is a **family** of related objects (terrain + tree + animal) that must **match** (no lions in an Asian bamboo forest). The drawing code should work with any scenario without knowing concrete classes.

Role mapping: AbstractFactory = `ScenarioFactory`; ConcreteFactories = `AfricanFactory`, `AsianFactory`, `AmericanFactory`; AbstractProducts = `Terrain`, `Tree`, `Animal`; Client = `Scenario` (calls `createTree()` 5 times, `createAnimal()` 2 times).

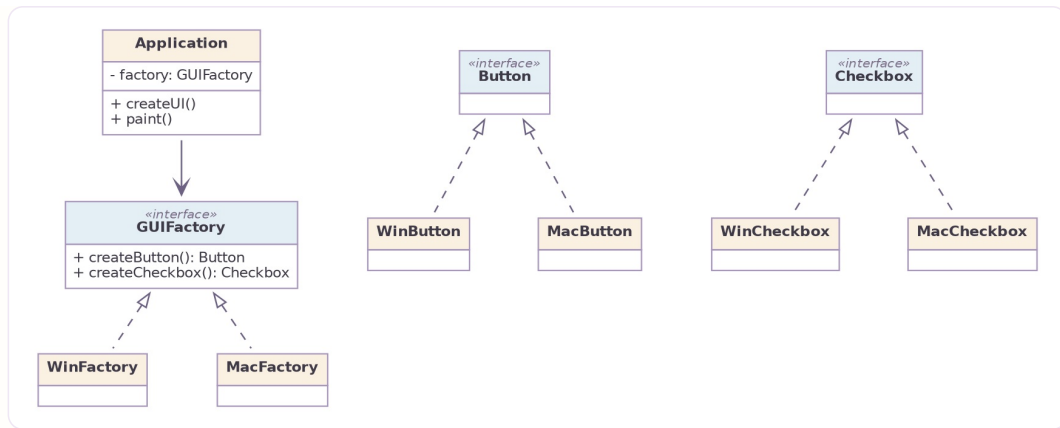


Abstract Factory design for game scenarios.

Why not Builder: we don't assemble one complex object step by step with optional parts; we need consistent *families*. (Flyweight could additionally share the 5 identical tree objects.)

7.4 Which pattern when you have a group of related objects designed to work together and must ensure they're used together? Justify with a class diagram.

A: Abstract Factory. Each concrete factory creates only products of **one family**, so the client can't accidentally mix a Windows button with a Mac checkbox — consistency is guaranteed by construction, and the client depends only on interfaces.



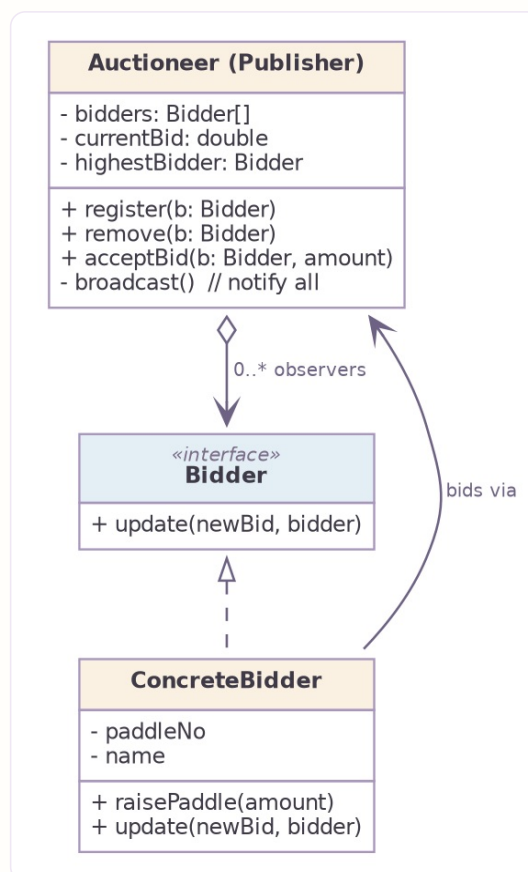
GUI example: WinFactory and MacFactory each produce a matching Button + Checkbox.

7.5 Auction: auctioneer takes bids; a bidder raises a numbered paddle; the price changes and all other bidders must be notified; auctioneer broadcasts the new bid. Design it.

A: Pattern: Observer.

Why: one object (auctioneer) changes state (current bid) and **many** objects (bidders) must be notified automatically; bidders can join/leave.

Role mapping: Publisher = **Auctioneer** (**register** , **remove** , **broadcast**); Subscriber = **Bidder** interface with **update(newBid, bidder)** ; ConcreteSubscriber = **ConcreteBidder** (has a **paddleNo** , calls **auctioneer.acceptBid(this, amount)** when raising the paddle).



Observer design for the auction.

7.6 Purchase approval: Manager approves $\leq 10,000$; Director $> 10,000$. Design it (class diagram + client). Then Assistant Director is added ($\leq 1,00,000$). Show minimal changes. asked 3x

A: Pattern: Chain of Responsibility.

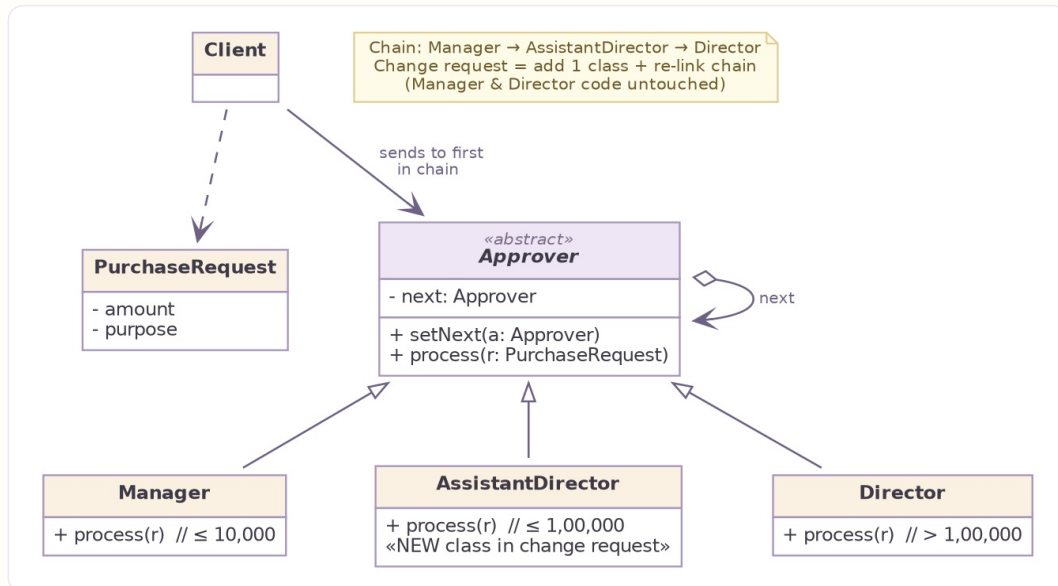
Why: a request must travel through a sequence of approvers until one has enough authority; the levels may change later.

Client wiring (design level): create `Manager`, `Director`; `manager.setNext(director)`; send every request to `manager.process(request)`.

Change request — minimal changes:

1. **Add one new class** `AssistantDirector` extends `Approver` (handles $\leq 1,00,000$).
2. Director's condition becomes $> 1,00,000$ (or Director simply handles anything reaching it).
3. **Re-link the chain** in client code: `manager.setNext(assistantDirector)`; `assistantDirector.setNext(director)`.

`Manager` and the `Approver` base class are untouched — the design is **open for extension (OCP)**.



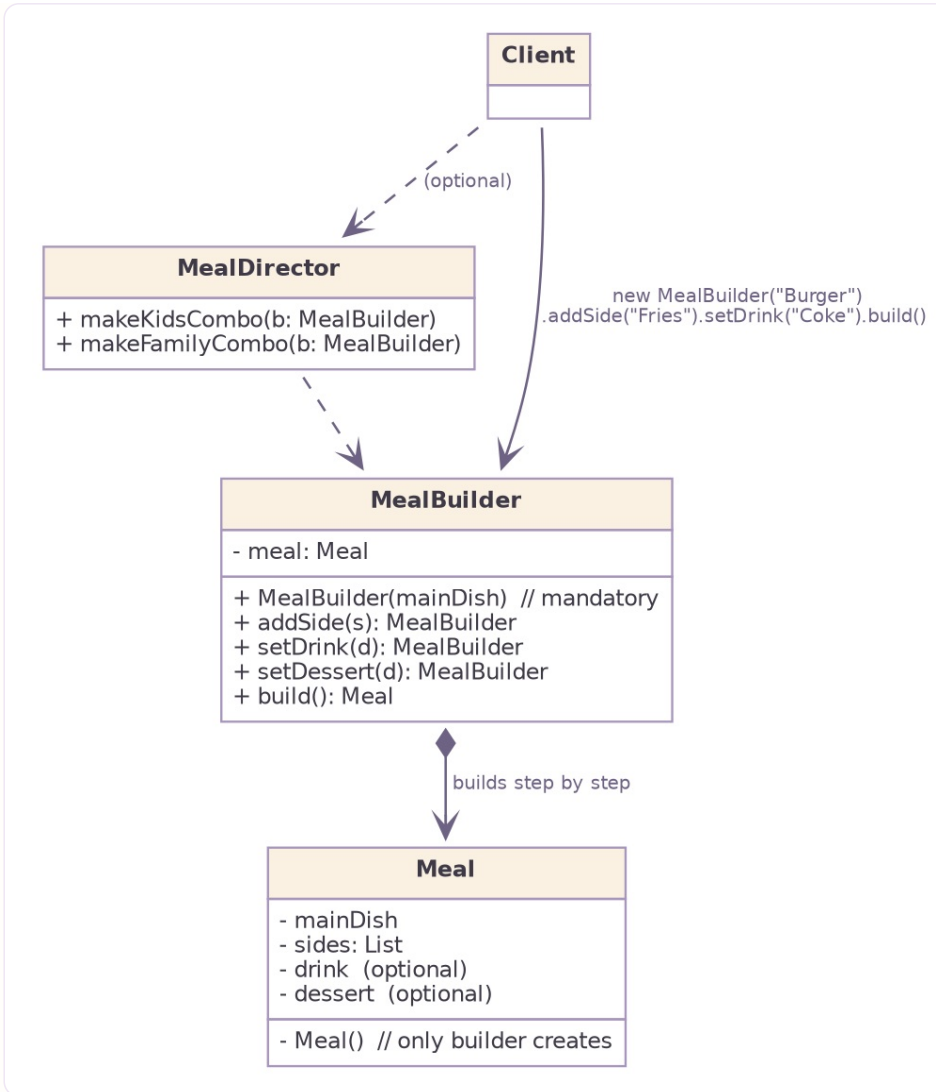
Chain of Responsibility design with the new AssistantDirector.

Why not Strategy/if-else: approvers are a **sequence with fall-through**, and the chain order is configured at runtime.

7.7 Meal ordering: main dish (mandatory), one or more sides, optional drink and dessert. Flexible, readable, step-by-step/chained construction. UML.

A: Pattern: Builder.

Why: many optional parts → a telescoping constructor would be unreadable. A builder takes the **mandatory part in its constructor**, offers **chainable** methods for optional parts (each returns the builder), and `build()` validates and returns the finished `Meal`. An optional **Director** (`MealDirector`) can hold standard combos.

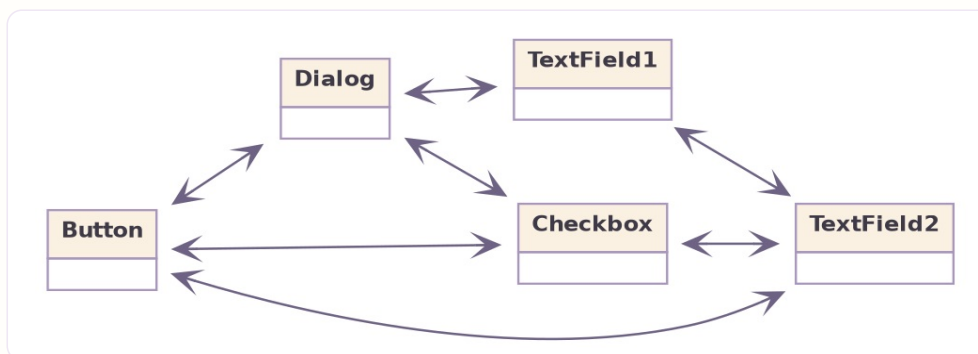


Builder design for custom meals.

Why not Abstract Factory: we're not creating families of separate products; we're assembling **one** object from optional pieces.

7.8 UI components (Button, Dialog, two TextFields, Checkbox) all depend on each other. Problems? Which pattern? Redraw. asked 2x

A: The current design:

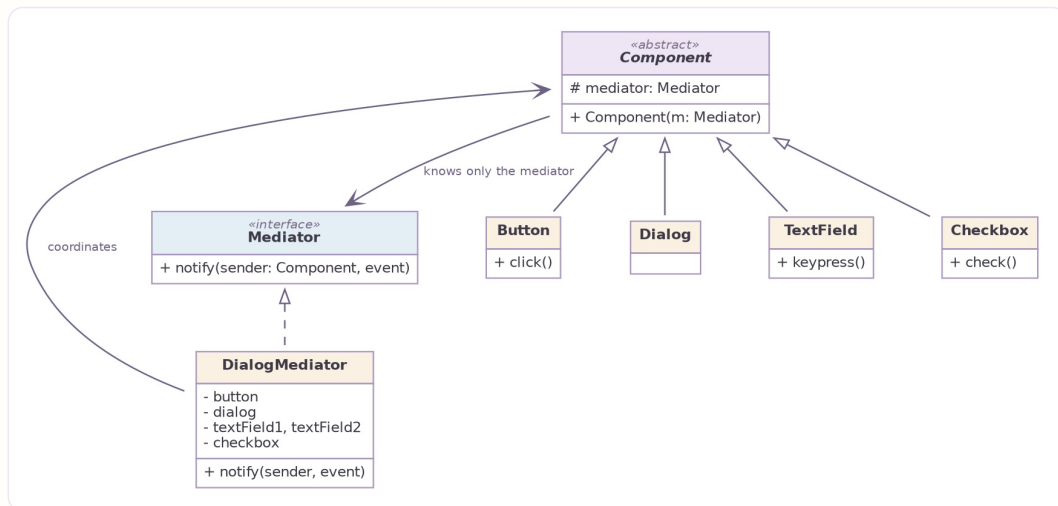


Before: a many-to-many web of dependencies.

Problems: tight **many-to-many coupling**; a change in one component ripples to others; components **can't be reused** in another dialog because they drag their neighbours along; interaction logic is **scattered** across all classes; adding a component means editing several others.

Pattern: Mediator. Components know only the mediator and report events (`mediator.notify(this, "check")`); the

DialogMediator holds all components and contains all the coordination logic.



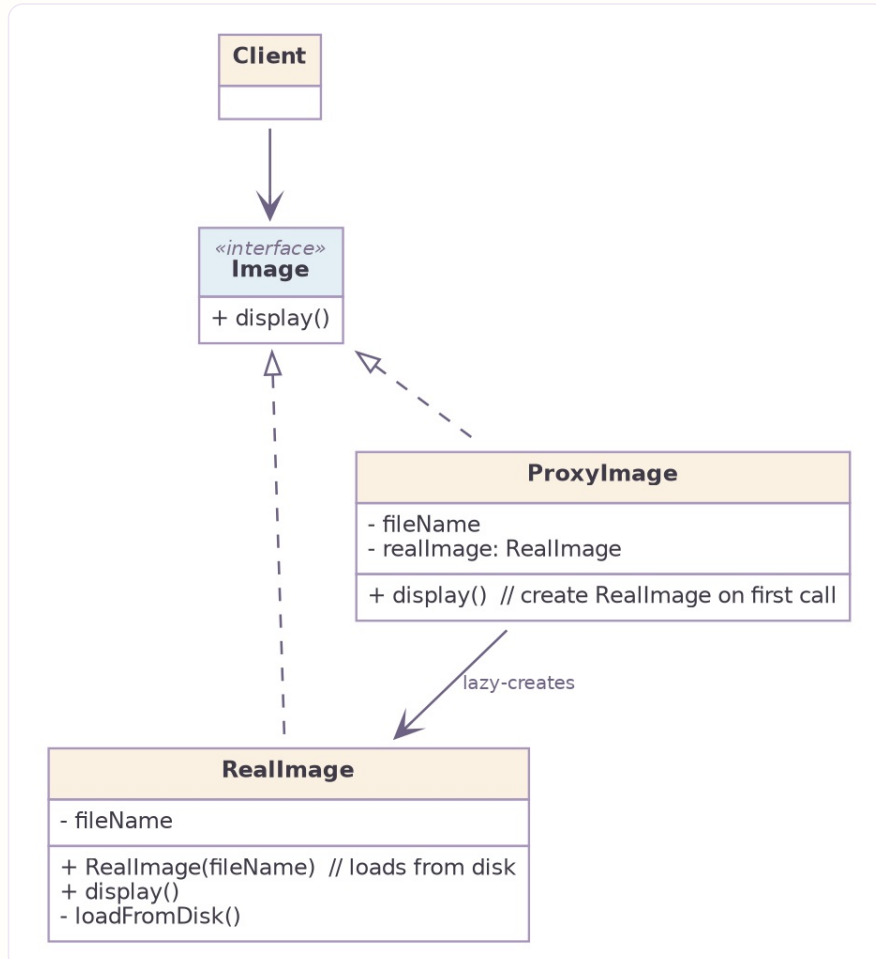
After: a star around the mediator.

Why not Observer alone: the interactions are two-way and specific between known components; a central coordinator (which may use Observer internally) removes the mutual dependencies.

7.9 Image loading: **RealImage** loads from disk in its constructor. Problems? Which pattern? UML.

A: Problems: the image is loaded from disk **as soon as the object is created**, even if it's never displayed → wasted memory and slow start-up; the client can't control when loading happens; adding caching or access checks would mean modifying **RealImage**.

Pattern: Proxy (virtual proxy). **ProxyImage** implements the same **Image** interface, stores only the file name, and creates the **RealImage** on the **first display()** call, then reuses it.

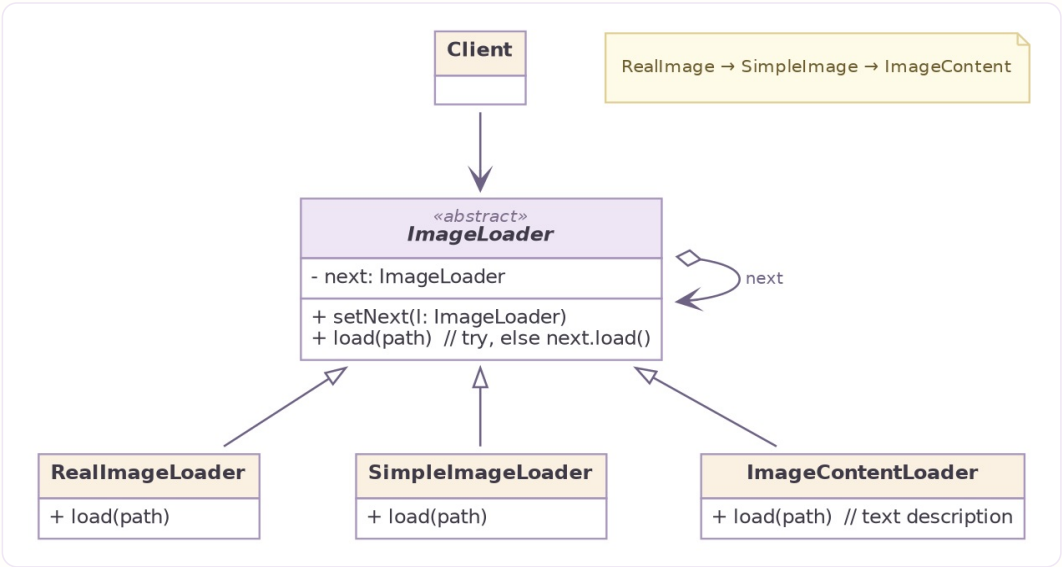


Virtual Proxy for lazy image loading.

Why not Decorator: we aren't adding features; we're **controlling when/whether** the real object is created — and the proxy manages its life cycle.

7.10 If ReallImage fails to load, SimpleImage is tried; if that fails, ImageContent loads a text description. Which pattern? UML.

A: Pattern: Chain of Responsibility. Each loader tries to handle the request; on failure it passes it to `next`. The chain is `RealImageLoader → SimpleImageLoader → ImageContentLoader` (the last one always succeeds — a safe end of chain).



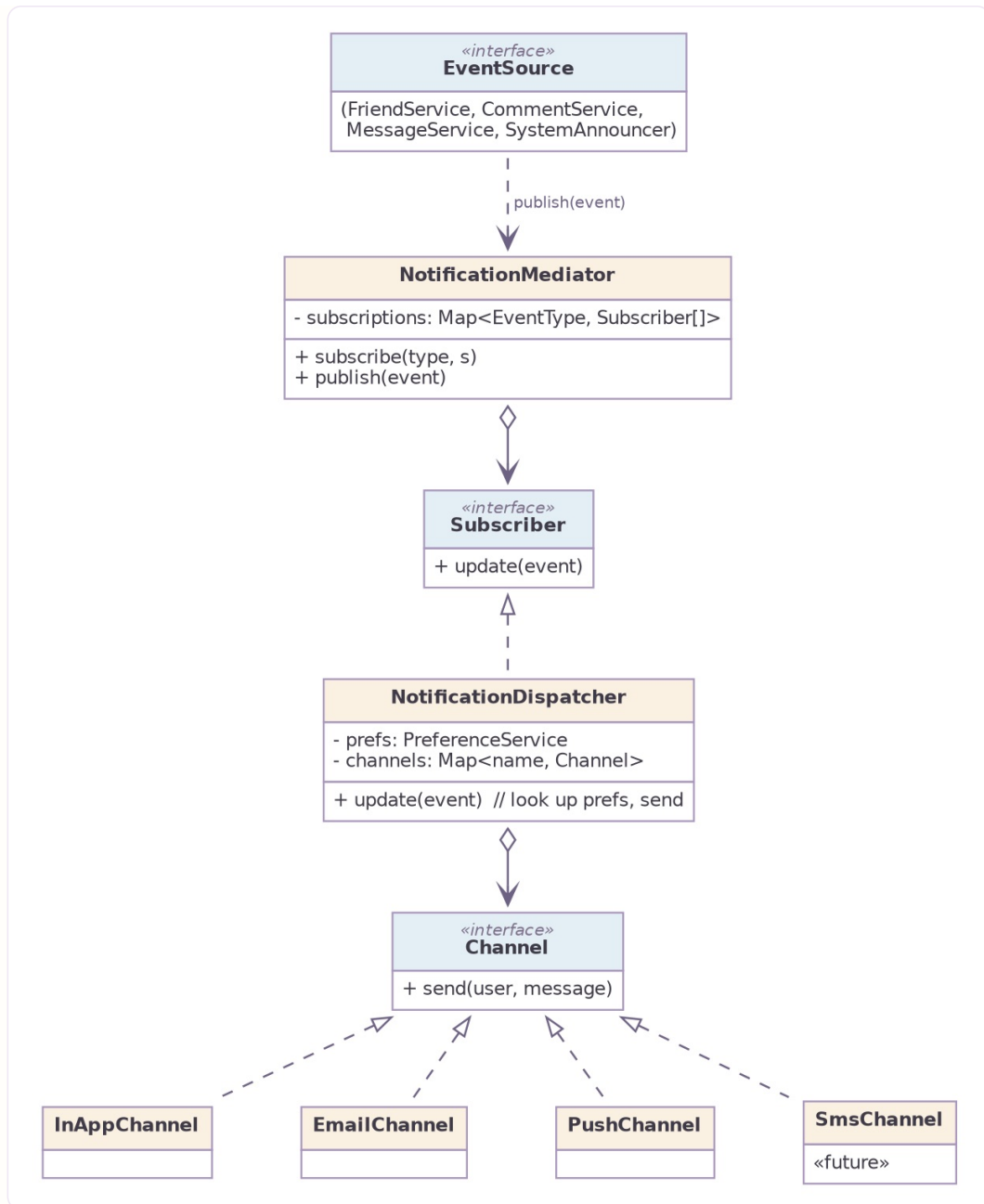
Chain of Responsibility for fallback loading.

7.11 Notification system (friend requests, comments, DMs, announcements; channels in-app/email/push; more events and channels later). Observer (User observes Post) vs Mediator (central NotificationMediator). Compare on scalability, flexibility, performance. Recommend.

A:

	Observer (User observes each source)	Mediator (central hub)
New event types (scalability)	Each new source becomes a new Subject; every user must subscribe to many subjects → subscription explosion (millions of users × posts)	New event type = register it with the hub; sources just publish
New channels (flexibility)	Channel logic sits inside <code>User.update()</code> → every channel change edits User (breaks SRP/OCP)	Channel choice lives in one dispatcher; new channel = new class
Coupling	Sources hold lists of users; users hold channel logic	Sources and channels know only the hub
Performance	Notification fan-out runs inside each source's call (e.g. posting a comment waits for emails)	Hub can queue/batch/async-send — but it's a single point of load/failure and can become a God object

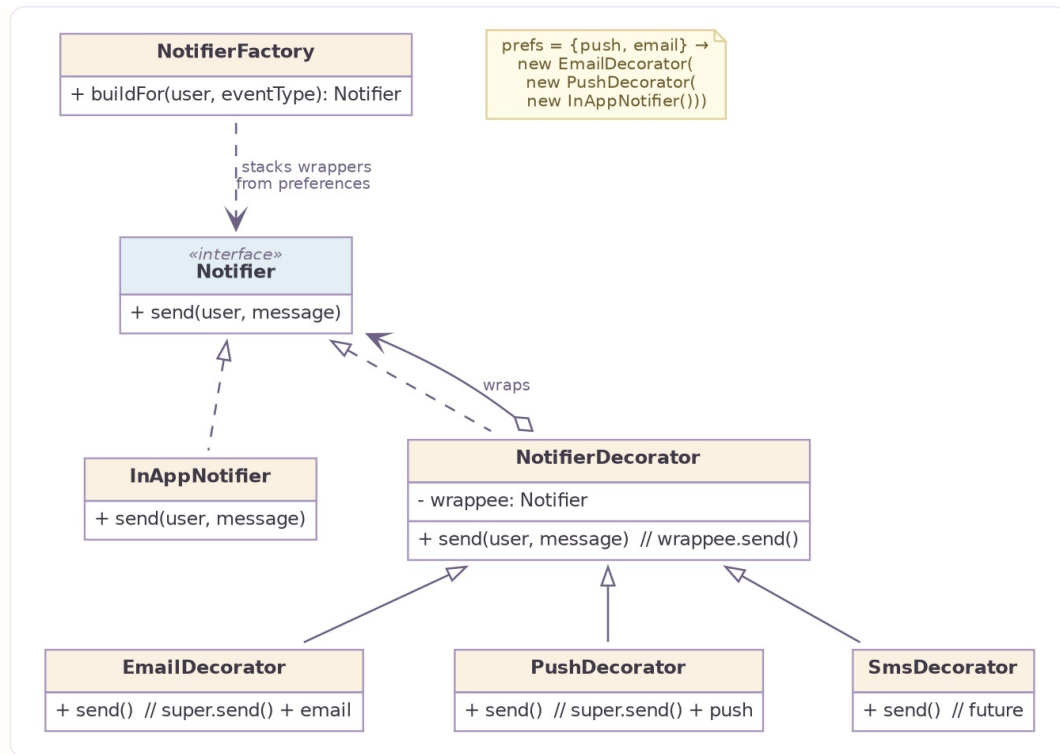
Recommendation: a **Mediator-style notification hub built on Observer**: sources publish events to the hub; the hub keeps subscriptions per **event type** (Observer mechanics) and hands events to a dispatcher that reads preferences and uses **Channel** objects (Strategy-like). This gives Observer's dynamic subscriptions *and* Mediator's central, decoupled routing. Make the hub asynchronous (queue) to avoid the bottleneck.



Recommended design: event hub + dispatcher + pluggable channels.

7.12 Same system: avoid big conditional logic when checking a user's channel preferences; dynamically build a "pipeline" of notification actions per event. Which pattern(s)?

A: Decorator — this is exactly Refactoring Guru's own Decorator example (a `Notifier` extended with SMS, Facebook, Slack decorators). Each channel is a decorator wrapping a base notifier; a small factory **stacks only the wrappers the user enabled** for that event type, so there's **no** `if (wantsEmail) ... if (wantsPush) ...`. Every enabled layer sends and passes the call on.



Alternative: Chain of Responsibility — a pipeline of channel handlers linked at runtime; each handler checks the preference and always passes on. Decorator fits better because every layer should act (no handler "stops" the request). A **Builder/factory** assembles either pipeline from preferences.

7.13 What is the motivation of Abstract Factory? Participants and roles? Example. asked 3x

A: Motivation: create **families of related objects** that must be used together, without the client depending on concrete classes — and switch an entire family by changing one factory object.

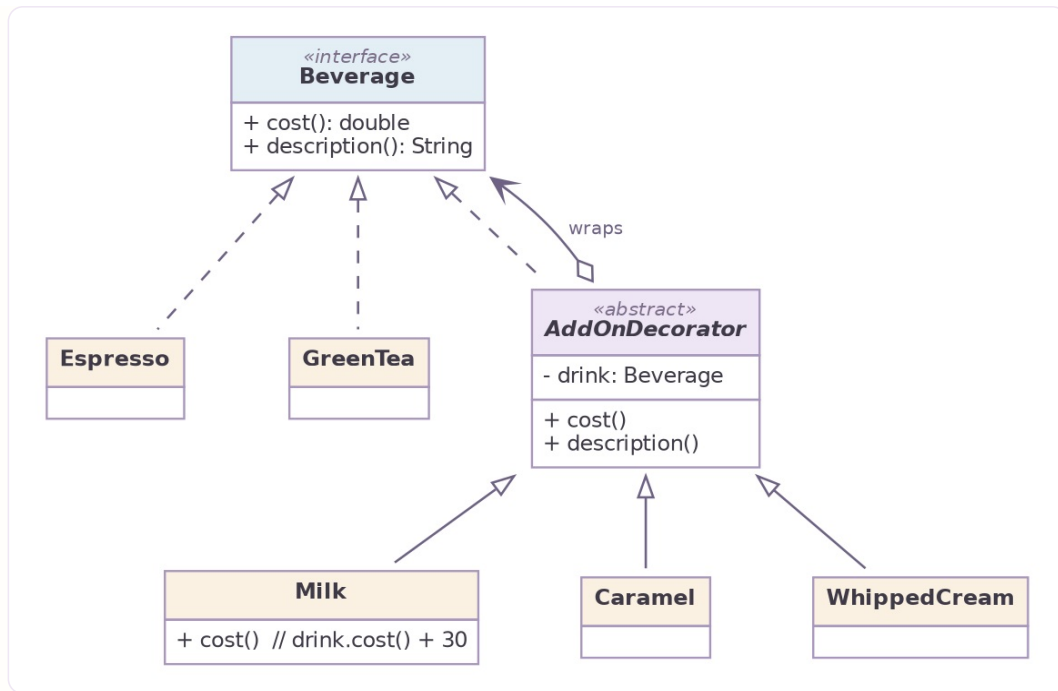
Participants: **AbstractFactory** (declares a create method per product type), **ConcreteFactory** (creates products of one family), **AbstractProduct** (interface per product type), **ConcreteProduct** (family-specific implementation), **Client** (uses only the abstract interfaces; gets a factory at start-up).

Example: the GUI design in 7.4 (WinFactory/MacFactory) or the game in 7.3.

7.14 What is the motivation of Decorator? Participants and roles? Example. asked 1x — full question

A: Motivation: add responsibilities to individual objects **dynamically and in combinations**, without creating a subclass for every combination and without changing the original class.

Participants: **Component** (common interface), **ConcreteComponent** (the basic object), **BaseDecorator** (implements Component and holds a Component — delegates to it), **ConcreteDecorators** (add behaviour before/after delegating), **Client** (wraps components in any order).



Example: beverages with add-ons. Espresso + Milk + Caramel = new Caramel(new Milk(new Espresso())); cost adds up through the layers.

7.15 What is the motivation of Command? Draw its UML and explain the participants. asked 2x

A: Motivation: turn a request into an object so we can **decouple the sender from the receiver**, pass actions as parameters (buttons, menus, shortcuts), **queue/schedule/log** them, and support **undo/redo**.

Participants: **Command** (interface with `execute()`), **ConcreteCommand** (knows the receiver + parameters, calls the receiver), **Receiver** (does the real work), **Invoker** (holds a command and triggers it; knows nothing about the receiver), **Client** (creates commands, links them to receivers, gives them to invokers).

See the Command structure in 5.14 and the text-editor design in 4.8.

7.16 What is the motivation of Strategy? What are the participants in Observer? Draw the UML for Command and explain each participant.

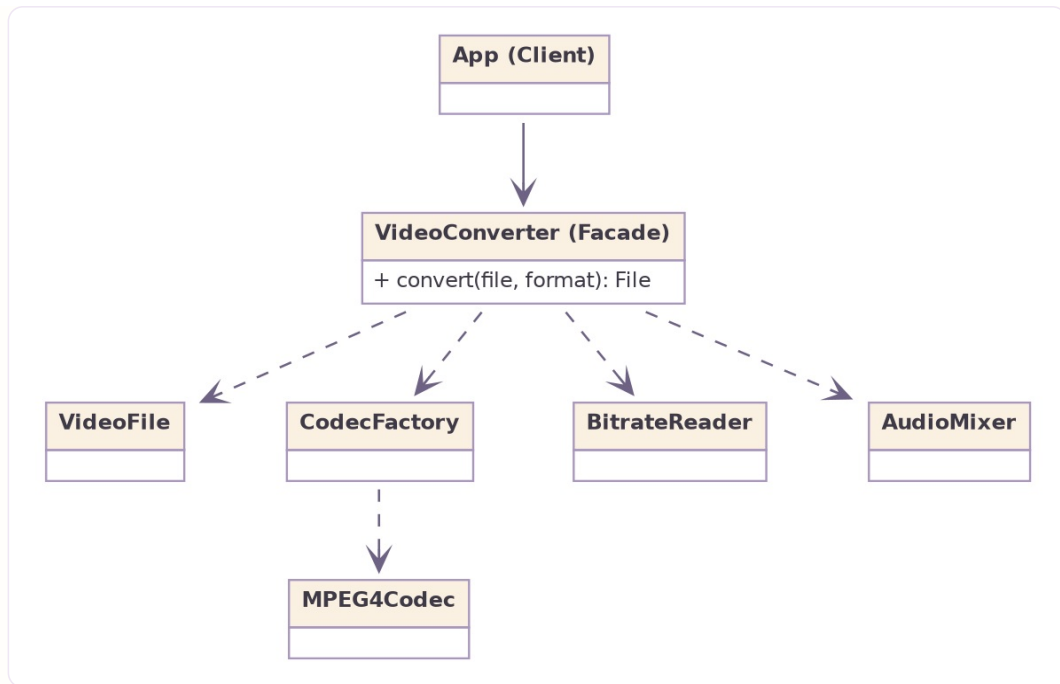
A: Strategy motivation: a class has many variants of one algorithm (often in a big `if/switch`). Strategy extracts each variant into its own class behind a common interface so they're **interchangeable at runtime**, new ones can be added without editing the context (OCP), and algorithm details are isolated.

Observer participants: **Publisher/Subject** (keeps subscriber list, `subscribe/unsubscribe/notify`), **Subscriber/Observer** interface (`update()`), **ConcreteSubscribers** (react to updates), **Client** (creates both and registers subscribers).

Command UML & roles: see 5.14 / 7.15.

7.17 Which pattern hides the complexity of a complex subsystem? UML, participants, roles.

A: Facade. Participants: **Facade** (simple methods that delegate to the right subsystem objects in the right order), **Additional Facade** (optional, keeps one facade from getting bloated), **Subsystem classes** (do the real work, unaware of the facade), **Client** (uses the facade instead of the subsystem).

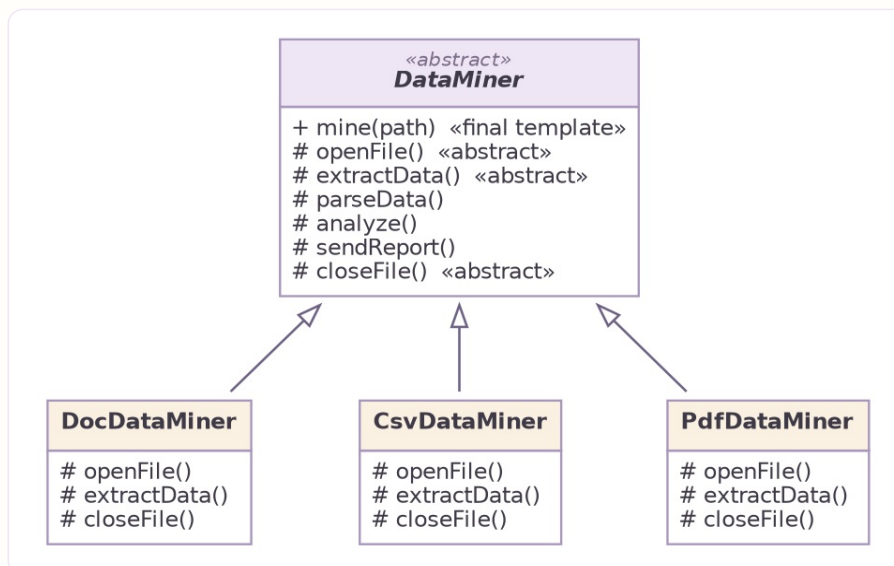


Example: VideoConverter facade over a complex video library.

7.18 Which pattern removes duplication in a common workflow? UML, participants, roles. asked 2x

A: Template Method. The shared workflow is written **once** as a template method in an abstract class; subclasses override only the steps that differ.

Participants: **AbstractClass** (declares the template method — fixed order of steps — plus abstract steps, default steps, and optional hooks), **ConcreteClass** (overrides the varying steps, never the template itself).

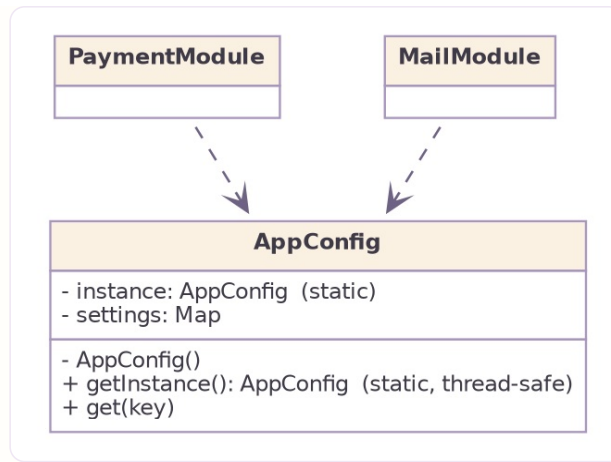


Example: DOC/CSV/PDF data miners share open → extract → parse → analyze → report → close.

7.19 What are the benefits of Singleton?

A: Guarantees a **single instance** (e.g. one config or connection pool); provides a **global access point** without a raw global variable (the instance is protected from being overwritten); **lazy initialization** — created only when first needed; saves resources.

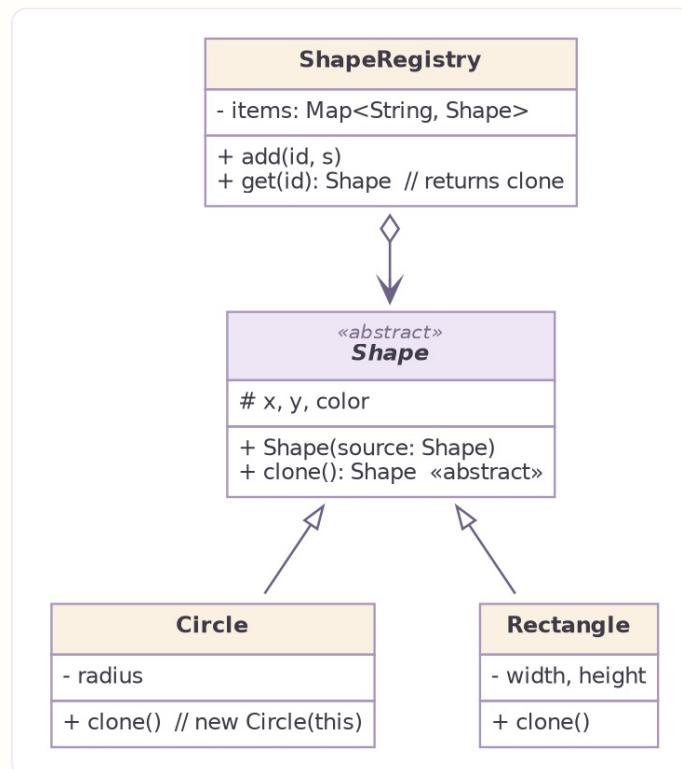
Mention the costs too (RG): violates SRP, hides dependencies, hard to test, needs thread-safety care.



Example: one shared AppConfig used by many modules.

7.20 Which pattern provides object cloning? Scenario. asked 2x

A: Prototype. Scenario: a graphics editor where users copy complex shapes (position, color, gradients, nested sub-shapes). Instead of rebuilding and re-configuring, each shape implements `clone()` and copies itself (including private fields). A **ShapeRegistry** can store pre-configured shapes ("red button", "large circle") to clone on demand.



Prototype with a registry of ready-made shapes.

7.21 Which pattern reduces memory requirement? How does it work? Which pattern automatically comes with it? asked 2x

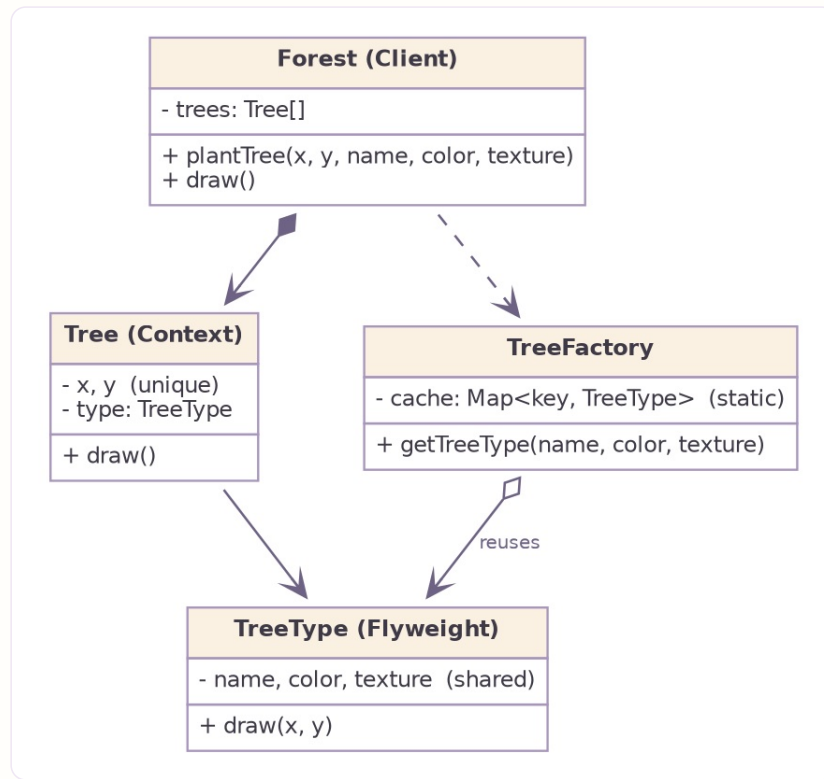
A: Flyweight. It splits object state into **intrinsic** (shared, same for many objects, stored once inside the flyweight, immutable) and **extrinsic** (unique per object, kept outside and passed in when needed). Thousands of objects then share a handful of flyweights instead of each storing the heavy data.

Comes with it: Factory — a **Flyweight Factory** manages a pool/cache and returns an existing flyweight or creates one if missing (so clients never create duplicates). (RG also notes Flyweight resembles **Singleton** if you could reduce all shared state to one object.)

7.22 What is Flyweight? Purpose, performance/memory benefit, intrinsic vs extrinsic state with examples, typical scenarios.

A: Purpose: support **huge numbers of fine-grained objects** efficiently by sharing identical state.

Example (RG forest): millions of trees. `TreeType` (name, color, texture — **intrinsic**) is stored once per species; each `Tree` keeps only `x, y` (**extrinsic**) and a reference to its `TreeType`.



Flyweight design for a forest.

Benefit: memory drops from *(heavy data × millions)* to *(heavy data × species) + (small data × millions)*; fewer objects also means less garbage collection. **Cost:** some CPU to pass/compute extrinsic data, more complex code.

Typical scenarios: particles/bullets in games, characters/glyphs in text editors, map tiles, icons in large lists.

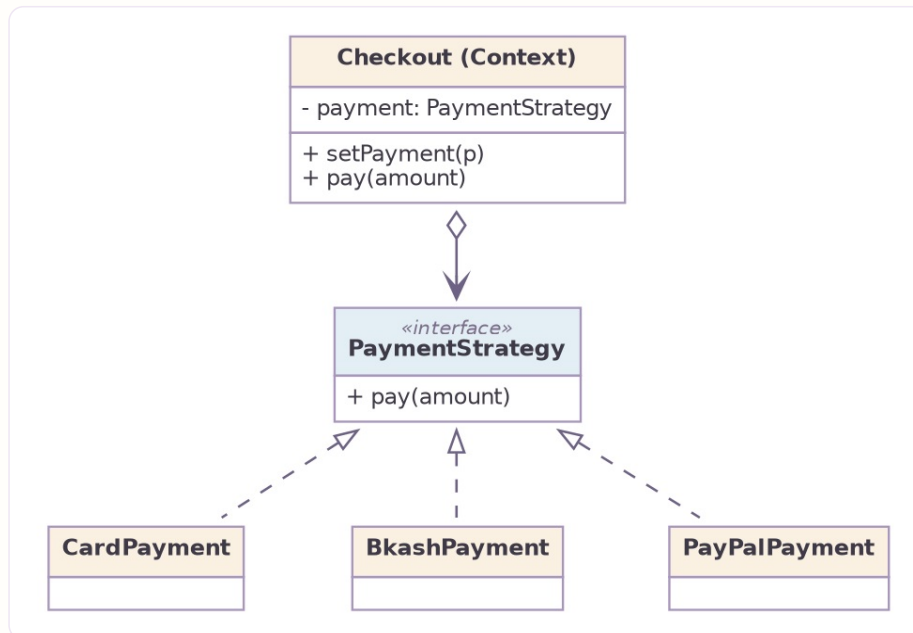
PART 8 — Scenario Practice (new, exam-style)

★ Teacher focus

Teacher: "Expect scenario based questions requiring you to suggest appropriate pattern(s) and design the core UML class design. No code needs to be written." Cover the answer, read the scenario, decide the pattern, then check.

8.1 Checkout supports card, bKash, and PayPal. The customer picks one at payment time; more methods will come. The current code is a long if-else on the method name.

A: Strategy. Each payment method is an interchangeable algorithm chosen by the **client** at runtime; the checkout just calls `payment.pay(amount)`.

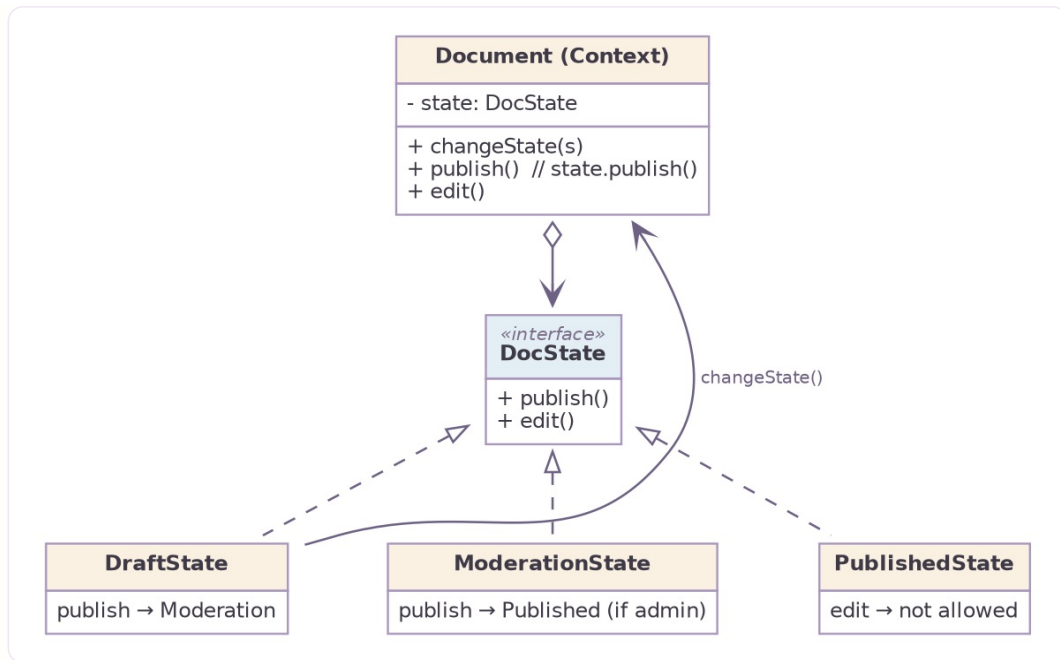


Strategy: Checkout holds a PaymentStrategy chosen by the client.

Why not State: the payment method doesn't switch itself based on internal status — the customer chooses it.

8.2 A document is Draft → Moderation → Published. `publish()` and `edit()` behave differently in each stage, and the stage changes automatically after actions. Methods are full of `switch(status)`.

A: State. Each stage is a class; the document delegates to its current state, and states trigger transitions (`Draft.publish()` moves the document to Moderation).

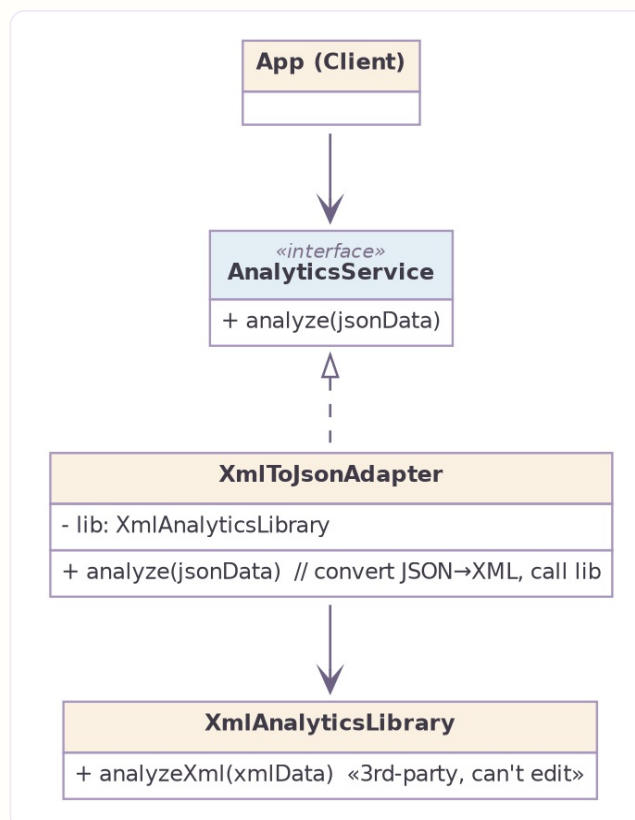


State: states know the context and switch it.

Why not Strategy: here the objects **change each other** based on internal status; a strategy is picked from outside and never switches itself.

8.3 Your app works with JSON. You must use a third-party analytics library that only accepts XML and you can't modify it.

A: Adapter. `XmlToJsonAdapter` implements the interface your app expects, converts JSON to XML, and calls the library.

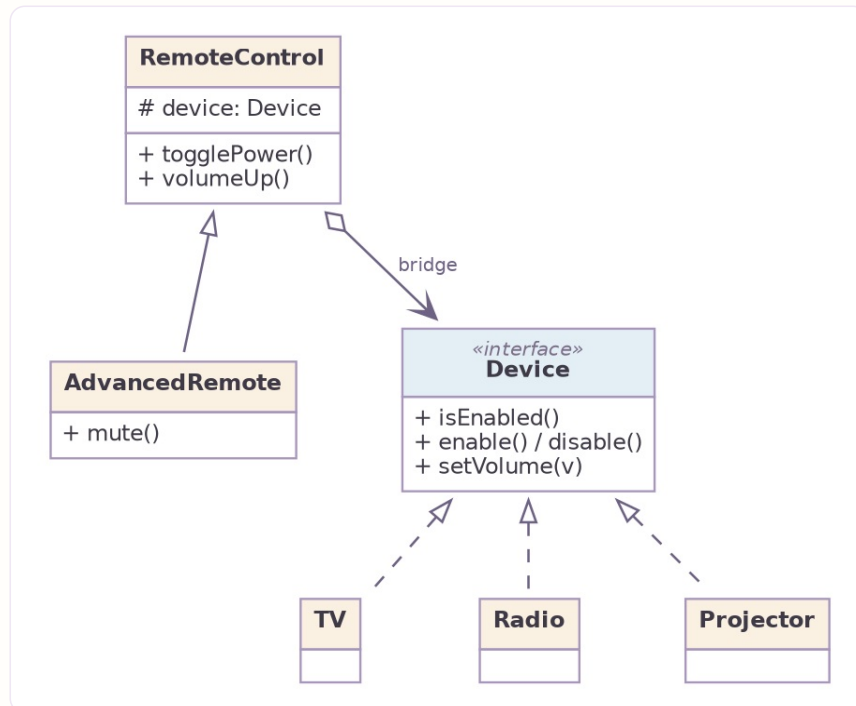


Adapter: translate between the app's interface and the library's.

Why not Facade: we're not simplifying a big subsystem — we're converting **one** incompatible interface.

8.4 Remote controls (basic, advanced) must work with many devices (TV, radio, projector). Subclasses like `AdvancedTVRemote`, `BasicRadioRemote` are exploding.

A: Bridge. Two independent dimensions (remote type × device type) → two hierarchies joined by a `device` reference. $m + n$ classes instead of $m \times n$.

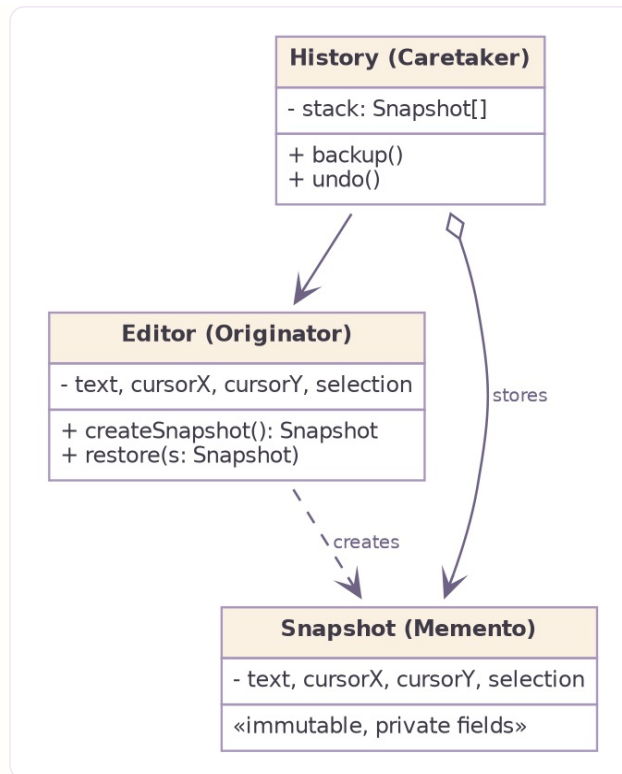


Bridge: remotes (abstraction) hold a Device (implementation).

Why not Adapter: nothing is incompatible; we're designing to let two sides vary independently.

8.5 A text editor must support multiple undo, restoring text, cursor position, and selection — without other classes reading the editor's private fields.

A: Memento (often with **Command**). The editor creates immutable snapshots of its own state; a History (caretaker) stores them without seeing inside.

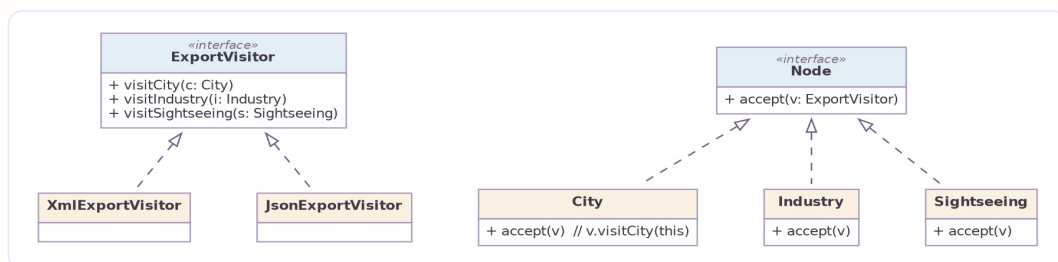


Memento: Editor = originator, Snapshot = memento, History = caretaker.

Why not only Command: reversing each command by hand is hard for complex state; a snapshot restores everything safely. Command decides *when* to save and restore.

8.6 A map app's node classes (City, Industry, Sightseeing) are stable and you must not change them, but you keep needing new operations: export to XML, to JSON, compute statistics...

A: Visitor. Each operation is a visitor with one `visitX()` per node class; nodes only get one `accept(visitor)` method (double dispatch).

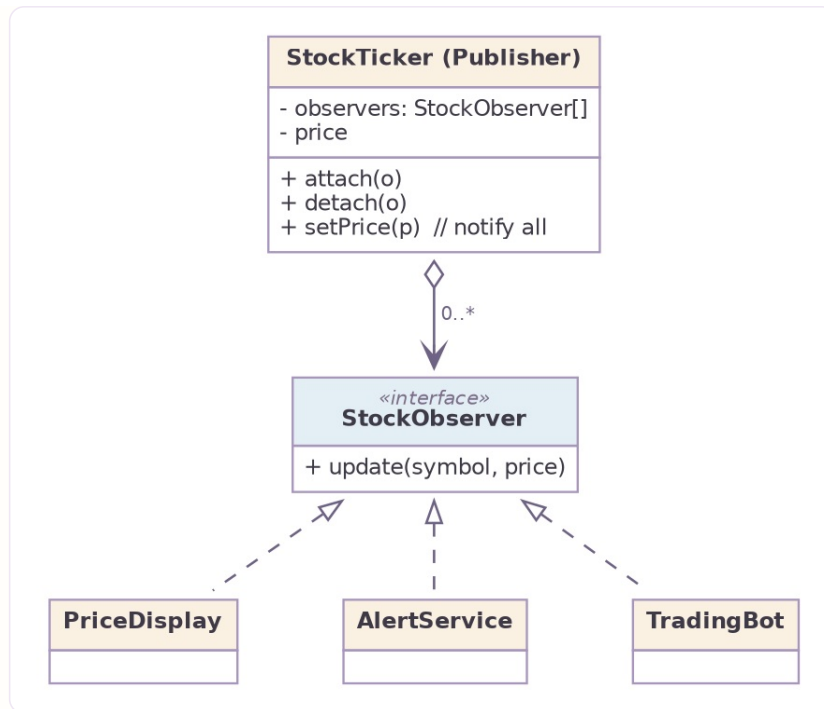


Visitor: new operations = new visitor classes.

When NOT: if new node types were added often, every visitor would need changing — then Visitor is a bad choice.

8.7 A stock ticker price changes; a price display, an alert service, and a trading bot must react. Listeners can be added/removed at runtime.

A: Observer. Ticker = publisher with a subscriber list; each listener implements `update(symbol, price)`.

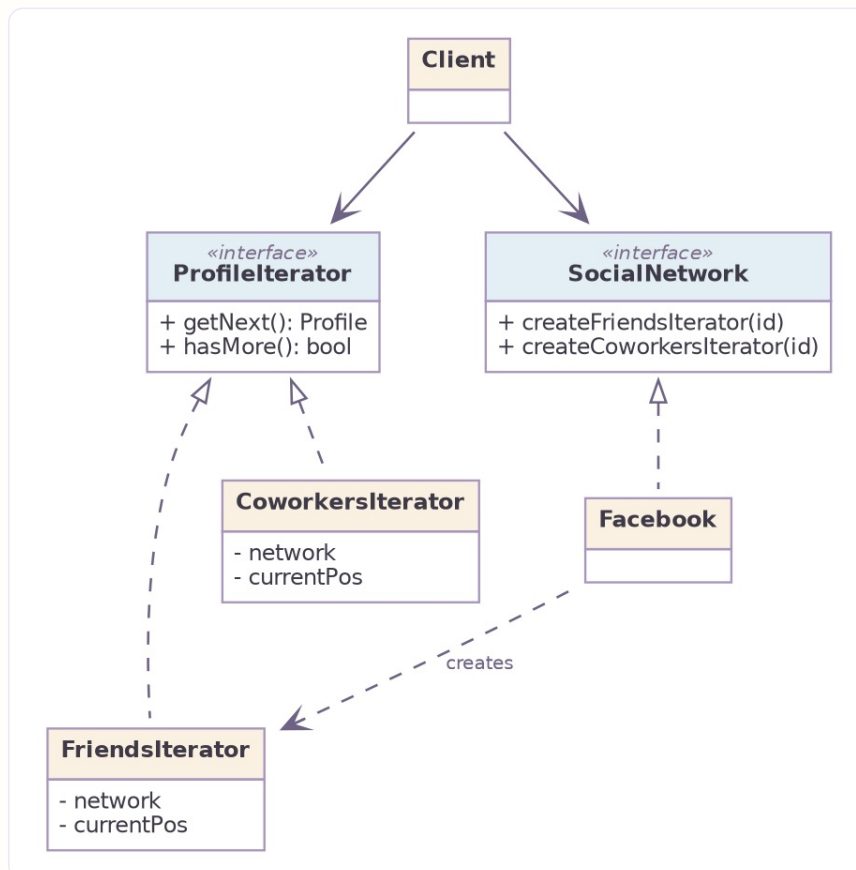


Observer: one publisher, many dynamic subscribers.

Why not Mediator: listeners don't interact with each other; it's a one-to-many broadcast.

8.8 A social network app must walk a user's friends list and coworkers list (loaded lazily from the network), without exposing how profiles are stored.

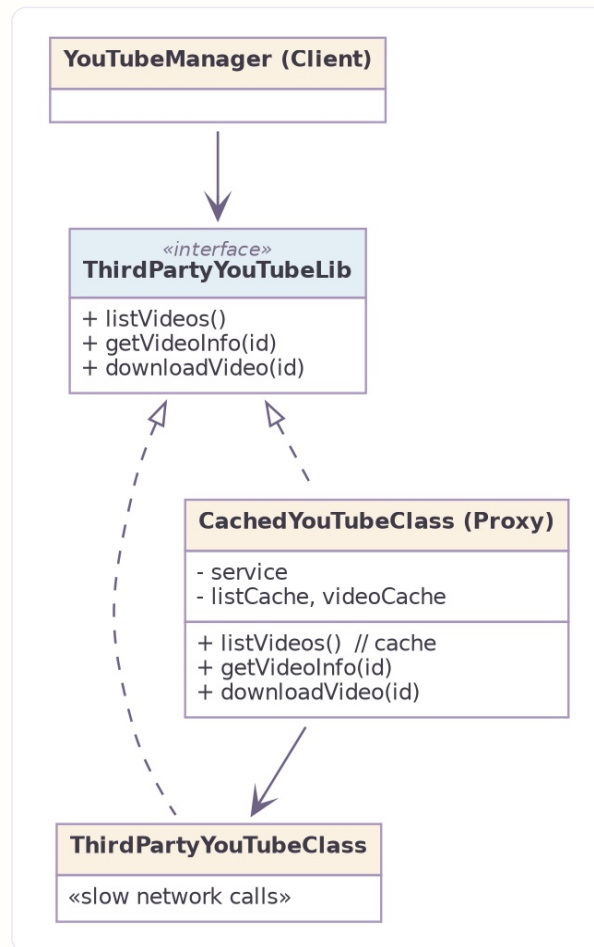
A: Iterator. Separate iterator classes keep the traversal state; the network creates the right iterator.



Iterator: different traversals over the same collection.

8.9 A slow third-party YouTube library downloads the same video info again and again. You want caching and lazy loading without changing the library or its clients.

A: Proxy (caching proxy). Same interface as the library; keeps caches; calls the real service only when needed.

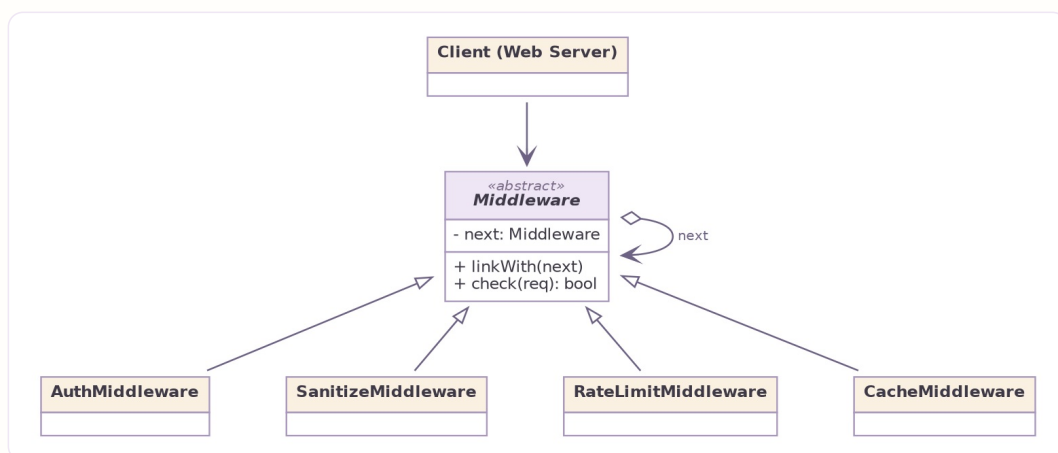


Proxy: caching stand-in with the same interface.

Why not Decorator: the purpose is **controlling access** (cache, laziness) and the proxy manages the real service; we're not stacking optional features.

8.10 Every web request must pass authentication, input sanitising, rate limiting, and caching checks. Some checks can stop the request. The order must be configurable.

A: Chain of Responsibility (middleware). Each check is a handler linked to the next; any handler can stop the request.

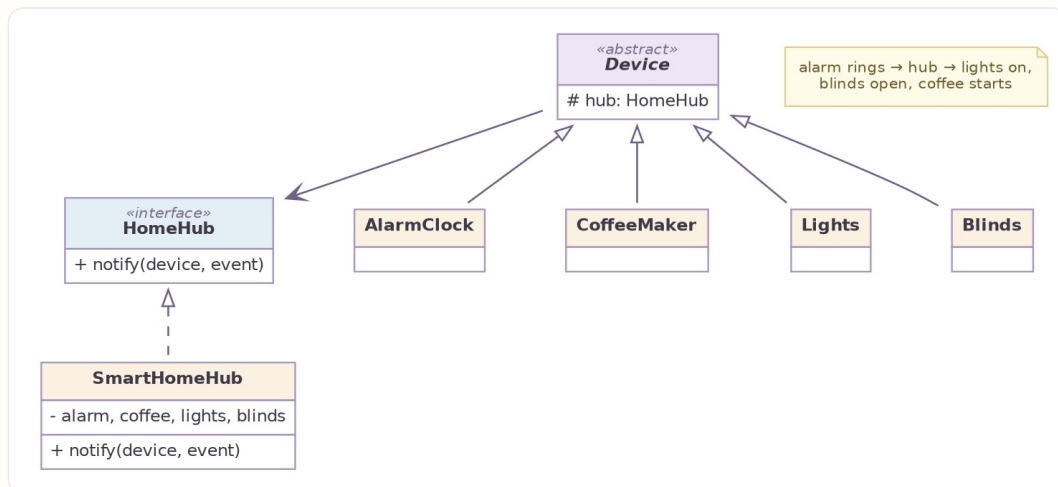


Chain of Responsibility: configurable middleware.

Why not Decorator: handlers must be able to **stop** the flow; decorators always pass on.

8.11 In a smart home, when the alarm rings the lights turn on, the blinds open, and the coffee machine starts; turning off the TV dims the lights... Devices shouldn't reference each other.

A: Mediator. Devices only notify the hub; the hub holds the rules.



Mediator: devices know only the hub.

Why not Observer: these are **many-to-many coordinated rules** between known devices, not one source broadcasting to anonymous listeners (the hub may use Observer internally).

8.12 Quick-fire: name the pattern

A:

Scenario	Pattern
Road vs sea logistics — the subclass decides whether a Truck or Ship is created	Factory Method
Furniture sets (chair + sofa + table) in Modern / Victorian styles, never mixed	Abstract Factory
SQL query builder with optional <code>where</code> , <code>orderBy</code> , <code>limit</code>	Builder
Game spawns many enemies that are copies of a configured template enemy	Prototype
One shared logger/configuration across the app	Singleton
Menu with sub-menus and menu items, treated the same	Composite
Add encryption and compression layers to a data stream in any combination	Decorator
One <code>placeOrder()</code> method hiding inventory, payment, shipping subsystems	Facade
Characters in a word processor share font/format objects	Flyweight
Access-control check before calling a sensitive service	Proxy
Tech support escalation: bot → operator → engineer	Chain of Responsibility
Toolbar button, menu item, and Ctrl+C all trigger "copy"; support undo	Command
Traverse a tree in DFS or BFS without exposing nodes	Iterator
Air traffic control tower coordinates planes	Mediator
Save game checkpoints and restore them	Memento
Newsletter subscribers get new posts	Observer
Vending machine: idle / has coin / dispensing / sold out	State
Route planning by car / walk / bike / transit	Strategy
Beverage making: boil → brew → pour → add condiments (tea vs coffee differ in brew/condiments)	Template Method
Calculate tax and export reports over many different product classes without changing them	Visitor
Old payment gateway class plugged into a new <code>PaymentProcessor</code> interface	Adapter

PART 9 — Memory Tricks 🧠

9.1 The three families

A: "Birth, Body, Brain" — Creational = how objects are *born*; Structural = how they're *built into bodies*; Behavioral = how they *think and talk*.

Counts on Refactoring Guru: **5 creational, 7 structural, 10 behavioral = 22**.

- Creational (5): "**Fat Apes Build Pretty Songs**" → Factory Method, Abstract Factory, Builder, Prototype, Singleton
- Structural (7): "**A Big Cat Does Fancy Flying Poses**" → Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
- Behavioral (10): "**Chef Cooks In My Mom's Old Stove, Slowly Tasting Veggies**" → Chain, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

9.2 Signatures as pictures — say the picture, name the pattern

A:

Picture in the UML	Pattern
Private constructor + static <code>getInstance()</code>	Singleton
Creator hierarchy parallel to product hierarchy, one <code>createX()</code>	Factory Method
Factory interface with many <code>createX()</code> methods, one factory per family	Abstract Factory
Many <code>buildStep()</code> methods + <code>getResult()</code> at the end (+ Director)	Builder
<code>clone()</code> in the interface	Prototype
Implements interface A, wraps object of interface B	Adapter
Two hierarchies joined by one has-a arrow	Bridge
Class implements interface and holds a list of that interface	Composite
Class implements interface and holds one of that interface; stackable	Decorator
One class calling many subsystem classes	Facade
Factory with cache + intrinsic/extrinsic split	Flyweight
Same interface as the real object + reference to it + access check	Proxy
Handler with a <code>next</code> handler (self-arrow)	Chain of Responsibility
<code>execute()</code> object holding a receiver; invoker holds command	Command
<code>getNext()/hasMore()</code> object + <code>createIterator()</code>	Iterator
Star: everyone points to one mediator	Mediator
Originator → Memento ← Caretaker (can't peek)	Memento
Publisher with list of subscribers + <code>notify()</code> loop	Observer
Context → State interface; states point back to context	State
Context → Strategy interface with setter; client picks	Strategy
Abstract class with final <code>templateMethod()</code> calling steps	Template Method
<code>visitX()</code> per element + <code>accept(v)</code> everywhere (double dispatch)	Visitor

9.3 Everyday analogies

A: Adapter = **plug converter** · Bridge = **remote + device** · Composite = **folder tree** · Decorator = **pizza toppings / clothing layers** · Facade = **hotel receptionist** · Flyweight = **shared crayon box** · Proxy = **credit card for bank**

account / bodyguard · Chain = **support escalation** · Command = **restaurant order slip** · Iterator = **TV channel-up button** · Mediator = **air traffic tower** · Memento = **game save** · Observer = **newsletter** · State = **traffic light / phone modes** · Strategy = **choose route (car/walk/bus)** · Template Method = **recipe with blanks / house plan** · Visitor = **insurance agent visiting different buildings**.

9.4 Tie-breaker one-liners

A:

- *Adapter translates, Proxy guards, Decorator decorates, Facade simplifies.*
- *Strategy — client switches; State — it switches itself.*
- *Strategy swaps the guts; Decorator adds skin.*
- *Template Method = inheritance, steps; Strategy = composition, whole algorithm.*
- *Composite = many children; Decorator = one child.*
- *Chain can stop; Decorator can't.*
- *Mediator = two-way hub; Facade = one-way front door; Observer = one-to-many broadcast.*
- *Adapter = after the fact; Bridge = before the fact.*
- *Factory Method = one product via subclass; Abstract Factory = matching family via object; Builder = one complex product step by step.*
- *Singleton = one mutable; Flyweight = many immutable shared.*

9.5 SOLID in one breath

A: **S** — one actor, one reason to change · **O** — add classes, don't edit tested ones (except wiring/config/bug fixes) · **L** — subclass never surprises the caller · **I** — small interfaces, no dummy methods · **D** — business logic owns the interface; details plug in.

9.6 Code-smell families (BOCDC)

A: **B**loaters (too big) · **OO** Abusers (OOP misused) · **C**hange Preventers (one change → many edits) · **D**ispensables (useless) · **C**ouplers (too connected). Think **effect** → **fix**, not the list.

PART 10 — Final Cheat Sheet 📄

10.1 The 22 Refactoring Guru patterns — intent + signature

A:

#	Pattern	Type	Intent (one line)	🔗 Signature
1	Factory Method	C	Subclass decides which product to create	overridable <code>createProduct()</code> returning interface
2	Abstract Factory	C	Create matching families of products	factory with a <code>createX()</code> per product type
3	Builder	C	Build a complex object step by step	step methods + <code>getResult()</code> ; optional Director
4	Prototype	C	Copy objects without knowing their class	<code>clone()</code> in common interface
5	Singleton	C	Exactly one instance, global access	private ctor + static <code>getInstance()</code>
6	Adapter	S	Make incompatible interfaces work	implements target, wraps different service
7	Bridge	S	Split abstraction and implementation	two hierarchies + one has-a link
8	Composite	S	Treat tree leaves and groups uniformly	implements Component + holds list of Component
9	Decorator	S	Add behaviour dynamically by wrapping	implements Component + holds one Component
10	Facade	S	Simple interface to a complex subsystem	one class → many subsystem classes
11	Flyweight	S	Share common state to save RAM	factory cache; intrinsic vs extrinsic
12	Proxy	S	Control access to an object	same interface + reference to real service
13	Chain of Responsibility	B	Pass request along handlers	handler has <code>next</code> handler
14	Command	B	Turn request into an object	<code>execute()</code> ; command holds receiver
15	Iterator	B	Traverse without exposing structure	<code>getNext()/hasMore()</code> + <code>createIterator()</code>
16	Mediator	B	Centralize communication	components → mediator only (star)
17	Memento	B	Save/restore state, keep encapsulation	originator / memento / caretaker
18	Observer	B	Notify many on change	subscriber list + <code>notify()</code>
19	State	B	Behaviour changes with internal state	context → state; states switch context
20	Strategy	B	Interchangeable algorithms	context → strategy; client picks
21	Template Method	B	Skeleton in base, steps in subclasses	final template calls abstract steps
22	Visitor	B	Add operations without changing classes	<code>visitX()</code> per element + <code>accept()</code>

(Interpreter is the 23rd GoF pattern, but Refactoring Guru doesn't cover it — the teacher follows RG, so focus on these 22.)

10.2 Scenario keyword → pattern

A:

If the scenario says...	Think
"only one instance", "shared config/logger"	Singleton
"subclass decides what to create", "framework users extend"	Factory Method
"family", "matching set", "theme/OS/region", "must be used together"	Abstract Factory
"optional parts", "step by step", "chained construction", "many constructor params"	Builder
"copy/clone a configured object"	Prototype
"third-party/legacy class, incompatible interface"	Adapter
"two dimensions vary", "m × n subclasses"	Bridge
"tree", "contains items and sub-groups", "part-whole", "hierarchical"	Composite

"add features/responsibilities at runtime", "combinations", "layers"	Decorator
"hide complexity", "simple entry point to a library"	Facade
"millions of similar objects", "memory"	Flyweight
"lazy load", "cache", "access control", "placeholder", "remote"	Proxy
"escalate", "fallback", "approval levels", "middleware", "try X then Y"	Chain of Responsibility
"undo/redo", "queue/schedule actions", "decouple invoker and receiver"	Command
"traverse without exposing internals", "different traversal orders"	Iterator
"components all depend on each other", "central coordinator"	Mediator
"snapshot", "restore previous state", "checkpoint"	Memento
"notify all subscribers/listeners when X changes"	Observer
"behaviour depends on status", "state machine", "switch(state)"	State
"choose algorithm at runtime", "if-else choosing a method"	Strategy
"same steps, some steps differ", "common workflow duplication"	Template Method
"new operations on stable classes", "export/report over many classes"	Visitor

10.3 Smell → effect → fix (top 12)

A:

Smell	Effect	Fix
Long Method	hard to read/test	Extract Method
Large Class	many reasons to change	Extract Class
Long Parameter List	confusing calls	Parameter Object
Primitive Obsession	scattered rules	Replace Data Value with Object
Switch Statements	edit every switch per new type	Replace Conditional with Polymorphism
Duplicate Code	fix one copy, forget other	Extract/Pull Up Method, Template Method
Feature Envy	logic far from data	Move Method
Refused Bequest	broken inheritance (LSP)	Push Down / Replace Inheritance with Delegation
Shotgun Surgery	one change, many edits	Move Method/Field
Divergent Change	one class, many reasons	Extract Class
Comments (deodorant)	hide unclear code	Extract Method + Rename
Lazy Class / Speculative Generality	cost for nothing	Inline Class / Collapse Hierarchy

10.4 Exam strategy (from the teacher's note)

A:

1. **Design patterns ≈ 60% of marks** — spend most revision time on Parts 5–8.
2. For scenario questions: **Problem → Pattern → Role mapping → UML → Why not the look-alike**. No code needed.
3. Draw UML with correct arrows: triangle for inheritance/implementation, diamond on the "whole" side, «interface» labels. Make the **signature** part of the pattern clearly visible.
4. For "similar patterns" questions: similarity first, then the **one intent difference**, in a table.
5. For SOLID: say **what it targets**, not only the slogan. SRP = one actor; OCP isn't for wiring/config/bug fixes.
6. For code smells: name the smell, **explain its effect**, name the refactoring.

10.5 Last-minute revision order

A:

1. Part 5 signatures + Part 9.2 picture table
2. Part 6 comparisons (Strategy/State, Proxy/Decorator/Adapter, Composite/Decorator, Observer/Mediator, factories)
3. Part 7 past-paper scenarios (Composite ×3, Chain ×2, Mediator, Proxy, Abstract Factory, Builder, Observer, Decorator)
4. Part 8 new scenarios
5. Part 2 SOLID (SRP meaning, OCP limits)
6. Part 3 smell effects + Part 4 exercises
7. Part 1 good-design & abstraction